Virtual Machine Image Guide

OpenStack contributors

Apr 01, 2026

CONTENTS

1 Abstract **1**

2 Contents **3**

2.1 Conventions 3
2.2 Introduction 4
2.3 Get images 7
2.4 Image requirements 11
2.5 Modify images 18
2.6 Create images manually 24
2.7 Tools to automate image creation 48
2.8 Converting between image formats 53
2.9 Image sharing 54
2.10 Appendix 55

Index **109**

CHAPTER

ONE

ABSTRACT

This guide describes how to obtain, create, and modify virtual machine images that are compatible with OpenStack.

CHAPTER

TWO

CONTENTS

2.1 Conventions

The OpenStack documentation uses several typesetting conventions.

2.1.1 Notices

Notices take these forms:

Note

A comment with additional information that explains a part of the text.

Important

Something you must be aware of before proceeding.

Tip

An extra but helpful piece of practical advice.

Caution

Helpful information that prevents the user from making mistakes.

Warning

Critical information about the risk of data loss or security issues.

2.1.2 Command prompts

```
$ command
```

Any user, including the `root` user, can run commands that are prefixed with the `$` prompt.

```
# command
```

The `root` user must run commands that are prefixed with the # prompt. You can also prefix these commands with the **sudo** command, if available, to run them.

2.2 Introduction

An OpenStack Compute cloud is not very useful unless you have virtual machine images (which some people call virtual appliances). This guide describes how to obtain, create, and modify virtual machine images that are compatible with OpenStack.

To keep things brief, we will sometimes use the term `image` instead of virtual machine image.

What is a virtual machine image?

A virtual machine image is a single file which contains a virtual disk that has a bootable operating system installed on it.

2.2.1 Disk and container formats for images

Virtual machine images come in different *formats*. A format describes the way the bits making up a file are arranged on the storage medium. Knowledge of a format is required in order for a consumer to interpret the content of the file correctly (rather than to simply view it as a bunch of bits).

When considering a stored virtual machine image, there are two types of format that can come into play.

container format
: The stored file may be a *container* that contains the virtual disk. For example, the virtual disk may be contained in a `tar` file which must be opened before the disk can be retrieved. Its possible, however, that the virtual disk is not contained in a file, but is just stored as-is by the Image Service.

disk format
: The virtual disk itself has its bits arranged in some format. A consuming service must know what this format is before it can effectively use the virtual disk.

2.2.2 Image metadata

Image metadata (also known as image properties) provide information about the virtual disk stored by the Image service. The metadata is stored as part of the image record associated with the image data by the Image service. Image metadata can help end users determine the nature of an image, and is used by associated OpenStack components and drivers which interface with the Image service.

So that image consumers can easily identify the container and disk format of images, the image service has set aside particular metadata keys for these. Not surprisingly, these are named `container_format` and `disk_format`. The legal values for each of these are specified in the Image services Image schema, which you can obtain in any OpenStack installation by making the following API call:

```
GET /v2/schemas/image
```

The supported formats may vary across OpenStack clouds. The formats accepted by a particular cloud will be specified in that clouds get-schema response to the Images API.

Note

The image schema lists the legal identifiers for container and disk formats. To understand what these identifiers refer to, consult the Disk and Container Formats section of the Glance User Guide.

Image metadata can also determine the scheduling of hosts. If specific metadata are set on an image (possible metadata are architecture, hypervisor type, and virtual machine mode), and Compute is configured so that the `ImagePropertiesFilter` scheduler filter is enabled (default), then the scheduler only considers compute hosts that satisfy the specified properties.

Note

Computes `ImagePropertiesFilter` value is specified in the `enabled_filters` value in the `[filter_scheduler]` section of the `/etc/nova/nova.conf` file.

Other Compute scheduler filters may also be affected by image metadata. For a complete list of valid property keys and values, refer to the Useful image properties section of the Glance Administration Guide.

2.2.3 Adding metadata to an image

You can add metadata to Image service images by using the `--property key=value` parameter with the **`openstack image create`** or **`openstack image set`** command. More than one property can be specified. For example:

```
$ openstack image set --property architecture=arm \
  --property hypervisor_type=qemu image_name_or_id
```

Common image properties are also specified in the `/etc/glance/schema-image.json` file. Other useful property keys and values, are listed in the Useful image properties section of the Glance Administration Guide.

All associated properties for an image can be displayed using the **`openstack image show`** command. For example:

```
$ openstack image show cirros
+------------------+------------------------------------------------------+
| Field            | Value                                                |
+------------------+------------------------------------------------------+
| checksum         | ee1eca47dc88f4879d8a229cc70a07c6                     |
| container_format | bare                                                 |
| created_at       | 2016-04-15T13:57:38Z                                 |
| disk_format      | qcow2                                                |
| file             | /v2/images/55f0907f-70a5-4376-a346-432e4ec509ed/file |
| id               | 55f0907f-70a5-4376-a346-432e4ec509ed                 |
| min_disk         | 0                                                    |
| min_ram          | 0                                                    |
| name             | cirros                                               |
| owner            | f9574e69042645d6b5539035cb8c00bf                     |
| properties       | architecture='arm', hypervisor_type='qemu'           |
| protected        | False                                                |
```

(continues on next page)

(continued from previous page)

```
| schema           | /v2/schemas/image                                    |
| size             | 13287936                                             |
| status           | active                                               |
| tags             |                                                      |
| updated_at       | 2016-04-15T13:57:57Z                                 |
| virtual_size     | None                                                 |
| visibility       | public                                               |
+------------------+------------------------------------------------------+
```

Note

Volume-from-Image properties

When creating Block Storage volumes from images, also consider your configured image properties. If you alter the core image properties, you should also update your Block Storage configuration. Amend `glance_core_properties` in the `/etc/cinder/cinder.conf` file on all controller nodes to match the core properties you have set in the Image service.

2.2.4 Metadata definition (metadefs) service

Images are not the only OpenStack resource that can have metadata associated with them. Many other resources (for example, volumes) support setting metadata on the resources. As with images, the metadata may be consumed by humans to understand something about the resource, or may be used by other OpenStack services so that they can make efficient use of the resource (for example, the nova filter scheduler using the image `architecture` property to determine an appropriate host on which to build an instance from that image). Thus it is important that there be a discoverable way for people and services to determine what metadata properties and values are available throughout an OpenStack cloud.

To facilitate this, Glance (the OpenStack Image service) hosts a metadata definition service, which is also known as the *OpenStack metadefs catalog*.

With this service you can define:

Namespace

- Contains metadata definitions.
- Specifies the access controls for everything defined in the namespace. These access controls determine who can define and use the definitions in the namespace.
- Associates the definitions with different types of resources.

Property
: A single property and its primitive constraints. Each property can only be a primitive type. For example, string, integer, number, boolean, or array.

Object
: Describes a group of one to many properties and their primitive constraints. Each property in the group can only be a primitive type. For example, string, integer, number, boolean, or array.

  The object may optionally define required properties under the semantic understanding that if you use the object, you should provide all required properties.

Resource type association
: Specifies the relationship between resource types and the namespaces that are applicable to them.

This information can be used to drive UI and CLI views. For example, the same namespace of objects, properties, and tags may be used for images, snapshots, volumes, and flavors. Or a namespace may only apply to images.

The Image service has predefined namespaces for the metadata definitions catalog. To load files from this directory into the database:

```
$ glance-manage db_load_metadefs
```

To unload the files from the database:

```
$ glance-manage db_unload_metadefs
```

To export the definitions in JSON format:

```
$ glance-manage db_export_metadefs
```

Note

By default, files are loaded from and exported to the Image services `/etc/glance/metadefs` directory.

There is no special relationship between the Image service and the Metadefs service. If you want to apply the keys and values defined in the Metadefs service to images, you must use the Image service API or client tools just as you would for any other OpenStack service.

For more information about the OpenStack Metadefs catalog, see:

- Using Glances Metadata Definitions Catalog Public APIs in the Glance User Guide
- The Metadata Definitions Service API Reference

2.3 Get images

Contents

- *Get images*
  - *AlmaLinux*
  - *Alpine Linux*
  - *Arch Linux*
  - *BSD: DragonFlyBSD, FreeBSD, NetBSD, OpenBSD*
  - *CentOS*
  - *CirrOS (test)*
  - *Debian*
  - *Fedora*
  - *Kali Linux*

- *Microsoft Windows*
- *openSUSE and SUSE Linux Enterprise Server*
- *Red Hat Enterprise Linux*
- *Rocky Linux*
- *Ubuntu*

The simplest way to obtain a virtual machine image that works with OpenStack is to download one that someone else has already created. Most of the images contain the `cloud-init` package to support the SSH key pair and user data injection. Because many of the images disable SSH password authentication by default, boot the image with an injected key pair. You can `SSH` into the instance with the private key and default login account. See Configure access and security for instances for more information on how to create and inject key pairs with OpenStack.

2.3.1 AlmaLinux

AlmaLinux provides cloud images for download, more detail on the get AlmaLinux project page

Note

In an AlmaLinux cloud image, the login account is `almalinux`.

2.3.2 Alpine Linux

Alpine Linux provides cloud images for download, more detail on the Alpine Linux cloud page

Note

In an Alpine Linux cloud image, the login account is `alpine`.

2.3.3 Arch Linux

Arch Linux provides a cloud image for download. More details can be found on the arch-boxes project page.

Note

In an Arch Linux image, the login account is `arch`.

2.3.4 BSD: DragonFlyBSD, FreeBSD, NetBSD, OpenBSD

Unofficial images for BSD are available on BSD-Cloud-Image.org.

Note

The login accounts are `freebsd` for FreeBSD, `openbsd` for OpenBSD, and `netbsd` for NetBSD.

2.3.5 CentOS

The CentOS project maintains official images for direct download:

- CentOS 9 Stream images
- CentOS 10 Stream images

Note

In a CentOS cloud image, the login account is `cloud-user`.

2.3.6 CirrOS (test)

CirrOS is a minimal Linux distribution that was designed for use as a test image on clouds such as OpenStack Compute. You can download a CirrOS image in various formats from the CirrOS download page.

If your deployment uses QEMU or KVM, we recommend using the images in qcow2 format. The most recent 64-bit qcow2 image as of this writing is cirros-0.6.2-x86_64-disk.img.

Note

In a CirrOS image, the login account is `cirros`. The password is `gocubsgo`. Since the fixed PW allows anyone to login, you should not run this image with a public IP attached.

2.3.7 Debian

Debian provides images for direct download. They are made at the same time as the CD and DVD images of Debian. Therefore, images are available on each point release of Debian. Also, weekly images of the testing distribution are available.

Note

In a Debian image, the login account is `debian`.

2.3.8 Fedora

The Fedora project maintains a list of official cloud images at Fedora download page.

Note

In a Fedora cloud image, the login account is `fedora`.

2.3.9 Kali Linux

Kali Linux provides cloud images for download at the Get Kali/Cloud page. See also the page Common Cloud Based Setup Information for cloud setup information.

Note

In a Kali Linux cloud image, the login account is `kali`.

2.3.10 Microsoft Windows

Cloudbase Solutions provides the last available trial version of Windows Server 2012 R2. This image includes cloudbase-init plus VirtIO drivers on KVM. You can build your own image based on Windows Server 2016, 2019, Windows 10 etc) with Cloudbase Imaging Tools.

ISO files for Windows 10 are available on Microsoft Windows 10 Downloadpage and Microsoft Evaluation Center.

Fedora Virtio provides also Windows images.

2.3.11 openSUSE and SUSE Linux Enterprise Server

The openSUSE community provides images for openSUSE under `Alternative Downloads / Cloud image`

SUSE maintains official SUSE Linux Enterprise Server cloud images. Go to the SUSE Linux Enterprise Server download page, select the `AMD64 / Intel 64` architecture and search for `Cloud`.

Note

In an openSUSE cloud image, the login account is `opensuse`.

2.3.12 Red Hat Enterprise Linux

Red Hat maintains official Red Hat Enterprise Linux cloud images. A valid Red Hat Enterprise Linux subscription is required to download these images.

- Red Hat Enterprise Linux 7 KVM Guest Image
- Red Hat Enterprise Linux 8 KVM Guest Image
- Red Hat Enterprise Linux 9 KVM Guest Image

Note

In a RHEL cloud image, the login account is `cloud-user`.

2.3.13 Rocky Linux

Rocky Linux provides cloud images for download, more detail on the Rocky Linux download page

Note

In an Rocky Linux cloud image, the login account is `rocky`.

2.3.14 Ubuntu

Canonical maintains an official set of Ubuntu-based images.

Images are arranged by Ubuntu release, and by image release date, with `current` being the most recent. For example, the page that contains the most recently built image for Ubuntu 24.04 Noble Numbat is Ubuntu 24.04 LTS (Noble Numbat) Daily Build. Scroll to the bottom of the page for links to the images that can be downloaded directly.

If your deployment uses QEMU or KVM, we recommend using the images in qcow2 format, with name ending in `.img`. The most recent version of the 64-bit amd64-arch QCOW2 image for Ubuntu 24.04 is noble-server-cloudimg-amd64.img.

Note

In an Ubuntu cloud image, the login account is `ubuntu`.

2.4 Image requirements

2.4.1 Linux

For a Linux-based image to have full functionality in an OpenStack Compute cloud, there are a few requirements. For some of these, you can fulfill the requirements by installing the cloud-init package. Read this section before you create your own image to be sure that the image supports the OpenStack features that you plan to use.

- Disk partitions and resize root partition on boot (`cloud-init`)
- No hard-coded MAC address information
- SSH server running
- Disable firewall
- Access instance using ssh public key (`cloud-init`)
- Process user data and other metadata (`cloud-init`)
- Paravirtualized Xen support in Linux kernel (Xen hypervisor only with Linux kernel version < 3.0)

2.4.2 Disk partitions and resize root partition on boot (cloud-init)

When you create a Linux image, you must decide how to partition the disks. The choice of partition method can affect the resizing functionality, as described in the following sections.

The size of the disk in a virtual machine image is determined when you initially create the image. However, OpenStack lets you launch instances with different size drives by specifying different flavors. For example, if your image was created with a 5 GB disk, and you launch an instance with a flavor of `m1.small`. The resulting virtual machine instance has, by default, a primary disk size of 20 GB. When the disk for an instance is resized up, zeros are just added to the end.

Your image must be able to resize its partitions on boot to match the size requested by the user. Otherwise, after the instance boots, you must manually resize the partitions to access the additional storage to which you have access when the disk size associated with the flavor exceeds the disk size with which your image was created.

Xen: one ext3/ext4 partition (no LVM)

If you use the OpenStack XenAPI driver, the Compute service automatically adjusts the partition and file system for your instance on boot. Automatic resize occurs if the following conditions are all true:

- `auto_disk_config=True` is set as a property on the image in the image registry.
- The disk on the image has only one partition.
- The file system on the one partition is ext3 or ext4.

Therefore, if you use Xen, we recommend that when you create your images, you create a single ext3 or ext4 partition (not managed by LVM). Otherwise, read on.

Non-Xen with cloud-init/cloud-tools: one ext3/ext4 partition (no LVM)

You must configure these items for your image:

- The partition table for the image describes the original size of the image.
- The file system for the image fills the original size of the image.

Then, during the boot process, you must:

- Modify the partition table to make it aware of the additional space:
  - If you do not use LVM, you must modify the table to extend the existing root partition to encompass this additional space.
  - If you use LVM, you can add a new LVM entry to the partition table, create a new LVM physical volume, add it to the volume group, and extend the logical partition with the root volume.
- Resize the root volume file system.

Depending on your distribution, the simplest way to support this is to install in your image:

- the cloud-init package,
- the cloud-utils package, which, on Ubuntu and Debian, also contains the `growpart` tool for extending partitions,
- if you use Fedora, CentOS 7, or RHEL 7, the `cloud-utils-growpart` package, which contains the `growpart` tool for extending partitions,
- if you use Ubuntu or Debian, the cloud-initramfs-growroot package , which supports resizing root partition on the first boot.

With these packages installed, the image performs the root partition resize on boot. For example, in the `/etc/rc.local` file.

If you cannot install `cloud-initramfs-tools`, Robert Plestenjak has a GitHub project called linux-rootfs-resize that contains scripts that update a ramdisk by using `growpart` so that the image resizes properly on boot.

If you can install the `cloud-init` and `cloud-utils` packages, we recommend that when you create your images, you create a single ext3 or ext4 partition (not managed by LVM).

Non-Xen without cloud-init/cloud-tools: LVM

If you cannot install `cloud-init` and `cloud-tools` inside of your guest, and you want to support resize, you must write a script that your image runs on boot to modify the partition table. In this case, we recommend using LVM to manage your partitions. Due to a limitation in the Linux kernel (as of this writing), you cannot modify a partition table of a raw disk that has partitions currently mounted, but you can do this for LVM.

Your script must do something like the following:

1. Detect if any additional space is available on the disk. For example, parse the output of `parted /dev/sda --script "print free"`.
2. Create a new LVM partition with the additional space. For example, `parted /dev/sda --script "mkpart lvm ..."`.
3. Create a new physical volume. For example, `pvcreate /dev/sda6`.
4. Extend the volume group with this physical partition. For example, `vgextend vg00 /dev/sda6`.
5. Extend the logical volume contained the root partition by the amount of space. For example, `lvextend /dev/mapper/node-root /dev/sda6`.
6. Resize the root file system. For example, `resize2fs /dev/mapper/node-root`.

You do not need a `/boot` partition unless your image is an older Linux distribution that requires that `/boot` is not managed by LVM.

2.4.3 No hard-coded MAC address information

You must remove the network persistence rules in the image because they cause the network interface in the instance to come up as an interface other than eth0. This is because your image has a record of the MAC address of the network interface card when it was first installed, and this MAC address is different each time the instance boots. You should alter the following files:

- Replace `/etc/udev/rules.d/70-persistent-net.rules` with an empty file (contains network persistence rules, including MAC address).
- Replace `/lib/udev/rules.d/75-persistent-net-generator.rules` with an empty file (this generates the file above).
- Remove the HWADDR line from `/etc/sysconfig/network-scripts/ifcfg-eth0` on Fedora-based images.

Note

If you delete the network persistent rules files, you may get a `udev kernel` warning at boot time, which is why we recommend replacing them with empty files instead.

2.4.4 Ensure ssh server runs

You must install an ssh server into the image and ensure that it starts up on boot, or you cannot connect to your instance by using ssh when it boots inside of OpenStack. This package is typically called `openssh-server`.

2.4.5 Disable firewall

In general, we recommend that you disable any firewalls inside of your image and use OpenStack security groups to restrict access to instances. The reason is that having a firewall installed on your instance can make it more difficult to troubleshoot networking issues if you cannot connect to your instance.

2.4.6 Access instance by using ssh public key (cloud-init)

The typical way that users access virtual machines running on OpenStack is to ssh using public key authentication. For this to work, your virtual machine image must be configured to download the ssh public key from the OpenStack metadata service or config drive, at boot time.

If both the XenAPI agent and `cloud-init` are present in an image, `cloud-init` handles ssh-key injection. The system assumes `cloud-init` is present when the image has the `cloud_init_installed` property.

Use cloud-init to fetch the public key

The `cloud-init` package automatically fetches the public key from the metadata server and places the key in an account. The account varies by distribution. On Ubuntu-based virtual machines, the account is called `ubuntu`, on Fedora-based virtual machines, the account is called `fedora`, and on CentOS-based virtual machines, the account is called `centos`.

You can change the name of the account used by `cloud-init` by editing the `/etc/cloud/cloud.cfg` file and adding a line with a different user. For example, to configure `cloud-init` to put the key in an account named `admin`, use the following syntax in the configuration file:

```
users:
  - name: admin
    (...)
```

Write a custom script to fetch the public key

If you are unable or unwilling to install `cloud-init` inside the guest, you can write a custom script to fetch the public key and add it to a user account.

To fetch the ssh public key and add it to the root account, edit the `/etc/rc.local` file and add the following lines before the line `touch /var/lock/subsys/local`. This code fragment is taken from the rackerjoe oz-image-build CentOS 6 template.

```
if [ ! -d /root/.ssh ]; then
  mkdir -p /root/.ssh
  chmod 700 /root/.ssh
fi

# Fetch public key using HTTP
ATTEMPTS=30
FAILED=0
while [ ! -f /root/.ssh/authorized_keys ]; do
  curl -f http://169.254.169.254/latest/meta-data/public-keys/0/openssh-key >␣
↪/tmp/metadata-key 2>/dev/null
  if [ $? -eq 0 ]; then
    cat /tmp/metadata-key >> /root/.ssh/authorized_keys
    chmod 0600 /root/.ssh/authorized_keys
```

(continues on next page)

(continued from previous page)

```
    restorecon /root/.ssh/authorized_keys
    rm -f /tmp/metadata-key
    echo "Successfully retrieved public key from instance metadata"
    echo "*****************"
    echo "AUTHORIZED KEYS"
    echo "*****************"
    cat /root/.ssh/authorized_keys
    echo "*****************"
  else
    FAILED=`expr $FAILED + 1`
    if [ $FAILED -ge $ATTEMPTS ]; then
      echo "Failed to retrieve public key from instance metadata after
↪$FAILED attempts, quitting"
      break
    fi
    echo "Could not retrieve public key from instance metadata (attempt #
↪$FAILED/$ATTEMPTS), retrying in 5 seconds..."
    sleep 5
  fi
done
```

Note

Some VNC clients replace : (colon) with ; (semicolon) and _ (underscore) with - (hyphen). If editing a file over a VNC session, make sure it is http: not http; and authorized_keys not authorized-keys.

2.4.7 Process user data and other metadata (cloud-init)

In addition to the ssh public key, an image might need additional information from OpenStack, such as to povide user data to instances, that the user submitted when requesting the image. For example, you might want to set the host name of the instance when it is booted. Or, you might wish to configure your image so that it executes user data content as a script on boot.

You can access this information through the metadata service or referring to Store metadata on the configuration drive. As the OpenStack metadata service is compatible with version 2009-04-04 of the Amazon EC2 metadata service, consult the Amazon EC2 documentation on Using Instance Metadata for details on how to retrieve the user data.

The easiest way to support this type of functionality is to install the `cloud-init` package into your image, which is configured by default to treat user data as an executable script, and sets the host name.

2.4.8 Ensure image writes boot log to console

You must configure the image so that the kernel writes the boot log to the `ttyS0` device. In particular, the `console=tty0 console=ttyS0,115200n8` arguments must be passed to the kernel on boot.

If your image uses `grub2` as the boot loader, there should be a line in the grub configuration file. For example, `/boot/grub/grub.cfg`, which looks something like this:

```
linux /boot/vmlinuz-3.2.0-49-virtual root=UUID=6d2231e4-0975-4f35-a94f-
↪56738c1a8150 ro console=tty0 console=ttyS0,115200n8
```

If `console=tty0 console=ttyS0,115200n8` does not appear, you must modify your grub configuration. In general, you should not update the `grub.cfg` directly, since it is automatically generated. Instead, you should edit the `/etc/default/grub` file and modify the value of the `GRUB_CMDLINE_LINUX_DEFAULT` variable:

```
GRUB_CMDLINE_LINUX_DEFAULT="console=tty0 console=ttyS0,115200n8"
```

Next, update the grub configuration. On Debian-based operating systems such as Ubuntu, run this command:

```
# update-grub
```

On Fedora-based systems, such as RHEL and CentOS, and on openSUSE, run this command:

```
# grub2-mkconfig -o /boot/grub2/grub.cfg
```

2.4.9 Paravirtualized Xen support in the kernel (Xen hypervisor only)

Prior to Linux kernel version 3.0, the mainline branch of the Linux kernel did not have support for paravirtualized Xen virtual machine instances (what Xen calls DomU guests). If you are running the Xen hypervisor with paravirtualization, and you want to create an image for an older Linux distribution that has a pre 3.0 kernel, you must ensure that the image boots a kernel that has been compiled with Xen support.

2.4.10 Manage the image cache

Use options in the `nova.conf` file to control whether, and for how long, unused base images are stored in the `/var/lib/nova/instances/_base/`. If you have configured live migration of instances, all your compute nodes share one common `/var/lib/nova/instances/` directory.

For information about the libvirt images in OpenStack, see The life of an OpenStack libvirt image from Pádraig Brady.

Table 1: Image cache management configuration options

Configuration option=Default value	(Type) Description
preallocate_images=none	(StrOpt) VM image preallocation mode: **none** No storage provisioning occurs up front. **space** Storage is fully allocated at instance start. The `$instance_dir/` images are fallocated to immediately determine if enough space is available, and to possibly improve VM I/O performance due to ongoing allocation avoidance, and better locality of block allocations.
remove_unused_base_images=True	(BoolOpt) Should unused base images be removed? When set to True, the interval at which base images are removed are set with the following two settings. If set to False base images are never removed by Compute.
remove_unused_original_minimum_age_seconds=86400	(IntOpt) Unused unresized base images younger than this are not removed. Default is 86400 seconds, or 24 hours.
remove_unused_resized_minimum_age_seconds=3600	(IntOpt) Unused resized base images younger than this are not removed. Default is 3600 seconds, or one hour.

To see how the settings affect the deletion of a running instance, check the directory where the images are stored:

```
# ls -lash /var/lib/nova/instances/_base/
```

In the `/var/log/compute/compute.log` file, look for the identifier:

```
2012-02-18 04:24:17 41389 WARNING nova.virt.libvirt.imagecache [-] Unknown␣
↪base file: /var/lib/nova/instances/_base/
↪06a057b9c7b0b27e3b496f53d1e88810a0d1d5d3_20
2012-02-18 04:24:17 41389 INFO nova.virt.libvirt.imagecache [-] Removable␣
↪base files: /var/lib/nova/instances/_base/
↪06a057b9c7b0b27e3b496f53d1e88810a0d1d5d3 /var/lib/nova/instances/_base/
↪06a057b9c7b0b27e3b496f53d1e88810a0d1d5d3_20
2012-02-18 04:24:17 41389 INFO nova.virt.libvirt.imagecache [-] Removing base␣
↪file: /var/lib/nova/instances/_base/06a057b9c7b0b27e3b496f53d1e88810a0d1d5d3
```

Because 86400 seconds (24 hours) is the default time for `remove_unused_original_minimum_age_seconds`, you can either wait for that time interval to see the base image removed, or set the value to a shorter time period in the `nova.conf` file. Restart all nova services after changing a setting in the `nova.conf` file.

2.5 Modify images

Once you have obtained a virtual machine image, you may want to make some changes to it before uploading it to the Image service. Here we describe several tools available that allow you to modify images.

Warning

Do not attempt to use these tools to modify an image that is attached to a running virtual machine. These tools are designed only to modify the images that are not currently running.

2.5.1 guestfish

The `guestfish` program is a tool from the libguestfs project that allows you to modify the files inside of a virtual machine image.

Note

`guestfish` does not mount the image directly into the local file system. Instead, it provides you with a shell interface that enables you to view, edit, and delete files. Many of **`guestfish`** commands, such as **`touch`**, **`chmod`**, and **`rm`**, resemble traditional bash commands.

Example guestfish session

Sometimes you must modify a virtual machine image to remove any traces of the MAC address that was assigned to the virtual network interface card when the image was first created. This is because the MAC address is different when the virtual machine images boots. This example shows how to use the `guestfish` to remove references to the old MAC address by deleting the `/etc/udev/rules.d/70-persistent-net.rules` file and removing the `HWADDR` line from the `/etc/sysconfig/network-scripts/ifcfg-eth0` file.

Assume that you have a CentOS qcow2 image called `centos63_desktop.img`. Mount the image in read-write mode as root, as follows:

```
# guestfish --rw -a centos63_desktop.img

Welcome to guestfish, the libguestfs filesystem interactive shell for
editing virtual machine filesystems.

Type: 'help' for help on commands
'man' to read the manual
'quit' to quit the shell

><fs>
```

This starts a guestfish session.

Note

the guestfish prompt looks like a fish: `><fs>`.

We must first use the **run** command at the guestfish prompt before we can do anything else. This will launch a virtual machine, which will be used to perform all of the file manipulations.

```
><fs> run
```

1. We can now view the file systems in the image using the **list-filesystems** command:

   ```
   ><fs> list-filesystems
   /dev/vda1: ext4
   /dev/vg_centosbase/lv_root: ext4
   /dev/vg_centosbase/lv_swap: swap
   ```

2. We need to mount the logical volume that contains the root partition:

   ```
   ><fs> mount /dev/vg_centosbase/lv_root /
   ```

3. Next, we want to delete a file. We can use the **rm** guestfish command, which works the same way it does in a traditional shell.

   ```
   ><fs> rm /etc/udev/rules.d/70-persistent-net.rules
   ```

4. We want to edit the `ifcfg-eth0` file to remove the `HWADDR` line. The **edit** command will copy the file to the host, invoke your editor, and then copy the file back.

   ```
   ><fs> edit /etc/sysconfig/network-scripts/ifcfg-eth0
   ```

5. If you want to modify this image to load the 8021q kernel at boot time, you must create an executable script in the `/etc/sysconfig/modules/` directory. You can use the **touch** guestfish command to create an empty file, the **edit** command to edit it, and the **chmod** command to make it executable.

   ```
   ><fs> touch /etc/sysconfig/modules/8021q.modules
   ><fs> edit /etc/sysconfig/modules/8021q.modules
   ```

6. We add the following line to the file and save it:

   ```
   modprobe 8021q
   ```

7. Then we set to executable:

   ```
   ><fs> chmod 0755 /etc/sysconfig/modules/8021q.modules
   ```

8. We are done, so we can exit using the **exit** command:

   ```
   ><fs> exit
   ```

Go further with guestfish

There is an enormous amount of functionality in guestfish and a full treatment is beyond the scope of this document. Instead, we recommend that you read the guestfs-recipes documentation page for a sense of what is possible with these tools.

2.5.2 guestmount

For some types of changes, you may find it easier to mount the images file system directly in the guest. The `guestmount` program, also from the libguestfs project, allows you to do so.

1. For example, to mount the root partition from our `centos63_desktop.qcow2` image to `/mnt`, we can do:

```
# guestmount -a centos63_desktop.qcow2 -m /dev/vg_centosbase/lv_root --rw␣
↪/mnt
```

2. If we did not know in advance what the mount point is in the guest, we could use the `-i` (inspect) flag to tell guestmount to automatically determine what mount point to use:

```
# guestmount -a centos63_desktop.qcow2 -i --rw /mnt
```

3. Once mounted, we could do things like list the installed packages using rpm:

```
# rpm -qa --dbpath /mnt/var/lib/rpm
```

4. Once done, we unmount:

```
# umount /mnt
```

2.5.3 virt-* tools

The libguestfs project has a number of other useful tools, including:

- virt-edit for editing a file inside of an image.
- virt-df for displaying free space inside of an image.
- virt-resize for resizing an image.
- virt-sysprep for preparing an image for distribution (for example, delete SSH host keys, remove MAC address info, or remove user accounts).
- virt-sparsify for making an image sparse.
- virt-p2v for converting a physical machine to an image that runs on KVM.
- virt-v2v for converting Xen and VMware images to KVM images.

Modify a single file inside of an image

This example shows how to use **`virt-edit`** to modify a file. The command can take either a filename as an argument with the `-a` flag, or a domain name as an argument with the `-d` flag. The following examples shows how to use this to modify the `/etc/shadow` file in instance with libvirt domain name `instance-000000e1` that is currently running:

```
# virsh shutdown instance-000000e1
# virt-edit -d instance-000000e1 /etc/shadow
# virsh start instance-000000e1
```

Resize an image

Here is an example of how to use **virt-resize** to resize an image. Assume we have a 16 GB Windows image in qcow2 format that we want to resize to 50 GB.

1. First, we use **virt-filesystems** to identify the partitions:

```
# virt-filesystems --long --parts --blkdevs -h -a /data/images/win2012.
↪qcow2
Name       Type       MBR  Size  Parent
/dev/sda1  partition  07   350M  /dev/sda
/dev/sda2  partition  07   16G   /dev/sda
/dev/sda   device     -    16G   -
```

2. In this case, it is the /dev/sda2 partition that we want to resize. We create a new qcow2 image and use the **virt-resize** command to write a resized copy of the original into the new image:

```
# qemu-img create -f qcow2 /data/images/win2012-50gb.qcow2 50G
# virt-resize --expand /dev/sda2 /data/images/win2012.qcow2 \
  /data/images/win2012-50gb.qcow2
Examining /data/images/win2012.qcow2 ...
**********

Summary of changes:

/dev/sda1: This partition will be left alone.

/dev/sda2: This partition will be resized from 15.7G to 49.7G.  The
    filesystem ntfs on /dev/sda2 will be expanded using the
    'ntfsresize' method.

**********
Setting up initial partition table on /data/images/win2012-50gb.qcow2 ...
Copying /dev/sda1 ...
 100% [                                                                ]␣
↪00:00
Copying /dev/sda2 ...
 100% [                                                                ]␣
↪00:00
Expanding /dev/sda2 using the 'ntfsresize' method ...

Resize operation completed with no errors. Before deleting the old
disk, carefully check that the resized disk boots and works correctly.
```

2.5.4 Loop devices, kpartx, network block devices

If you do not have access to the libguestfs, you can mount image file systems directly in the host using loop devices, kpartx, and network block devices.

Warning

Mounting untrusted guest images using the tools described in this section is a security risk, always use libguestfs tools such as guestfish and guestmount if you have access to them. See A reminder why you should never mount guest disk images on the host OS by Daniel Berrangé for more details.

Mount a raw image (without LVM)

If you have a raw virtual machine image that is not using LVM to manage its partitions, use the **losetup** command to find an unused loop device.

```
# losetup -f
/dev/loop0
```

In this example, /dev/loop0 is free. Associate a loop device with the raw image:

```
# losetup /dev/loop0 fedora17.img
```

If the image only has a single partition, you can mount the loop device directly:

```
# mount /dev/loop0 /mnt
```

If the image has multiple partitions, use **kpartx** to expose the partitions as separate devices (for example, /dev/mapper/loop0p1), then mount the partition that corresponds to the root file system:

```
# kpartx -av /dev/loop0
```

If the image has, say three partitions (/boot, /, swap), there should be one new device created per partition:

```
$ ls -l /dev/mapper/loop0p*
brw-rw---- 1 root disk 43, 49 2012-03-05 15:32 /dev/mapper/loop0p1
brw-rw---- 1 root disk 43, 50 2012-03-05 15:32 /dev/mapper/loop0p2
brw-rw---- 1 root disk 43, 51 2012-03-05 15:32 /dev/mapper/loop0p3
```

To mount the second partition, as root:

```
# mkdir /mnt/image
# mount /dev/mapper/loop0p2 /mnt/image
```

Once you are done, to clean up:

```
# umount /mnt/image
# rmdir /mnt/image
# kpartx -d /dev/loop0
# losetup -d /dev/loop0
```

Mount a raw image (with LVM)

If your partitions are managed with LVM, use **losetup** and **kpartx** commands as in the previous example to expose the partitions to the host.

```
# losetup -f
/dev/loop0
```

(continues on next page)

(continued from previous page)

```
# losetup /dev/loop0 rhel62.img
# kpartx -av /dev/loop0
```

Next, you need to use the **vgscan** command to identify the LVM volume groups and then the **vgchange** command to expose the volumes as devices:

```
# vgscan
Reading all physical volumes. This may take a while...
Found volume group "vg_rhel62x8664" using metadata type lvm2
# vgchange -ay
2 logical volume(s) in volume group "vg_rhel62x8664" now active
# mount /dev/vg_rhel62x8664/lv_root /mnt
```

Clean up when you are done:

```
# umount /mnt
# vgchange -an vg_rhel62x8664
# kpartx -d /dev/loop0
# losetup -d /dev/loop0
```

Mount a qcow2 image (without LVM)

You need the nbd (network block device) kernel module loaded to mount qcow2 images. This will load it with support for 16 block devices, which is fine for our purposes. As root:

```
# modprobe nbd max_part=16
```

Assuming the first block device (/dev/nbd0) is not currently in use, we can expose the disk partitions using the **qemu-nbd** and **partprobe** commands. As root:

```
# qemu-nbd -c /dev/nbd0 image.qcow2
# partprobe /dev/nbd0
```

If the image has, say three partitions (/boot, /, swap), there should be one new device created for each partition:

```
$ ls -l /dev/nbd0*
brw-rw---- 1 root disk 43, 48 2012-03-05 15:32 /dev/nbd0
brw-rw---- 1 root disk 43, 49 2012-03-05 15:32 /dev/nbd0p1
brw-rw---- 1 root disk 43, 50 2012-03-05 15:32 /dev/nbd0p2
brw-rw---- 1 root disk 43, 51 2012-03-05 15:32 /dev/nbd0p3
```

Note

If the network block device you selected was already in use, the initial **qemu-nbd** command will fail silently, and the /dev/nbd0p{1,2,3} device files will not be created.

If the image partitions are not managed with LVM, they can be mounted directly:

```
# mkdir /mnt/image
# mount /dev/nbd0p2 /mnt/image
```

When you are done, clean up:

```
# umount /mnt/image
# rmdir /mnt/image
# qemu-nbd -d /dev/nbd0
```

Mount a qcow2 image (with LVM)

If the image partitions are managed with LVM, after you use **qemu-nbd** and **partprobe**, you must use **vgscan** and **vgchange -ay** in order to expose the LVM partitions as devices that can be mounted:

```
# modprobe nbd max_part=16
# qemu-nbd -c /dev/nbd0 image.qcow2
# partprobe /dev/nbd0
# vgscan
Reading all physical volumes. This may take a while...
Found volume group "vg_rhel62x8664" using metadata type lvm2
# vgchange -ay
2 logical volume(s) in volume group "vg_rhel62x8664" now active
# mount /dev/vg_rhel62x8664/lv_root /mnt
```

When you are done, clean up:

```
# umount /mnt
# vgchange -an vg_rhel62x8664
# qemu-nbd -d /dev/nbd0
```

2.6 Create images manually

Creating a new image is a step done outside of your OpenStack installation. You create the new image manually on your own system and then upload the image to your cloud.

To create a new image, you will need the installation CD or DVD ISO file for the guest operating system. You will also need access to a virtualization tool. You can use KVM for this. Or, if you have a GUI desktop virtualization tool (such as, VMware Fusion or VirtualBox), you can use that instead. Convert the file to raw once you are done.

When you create a new virtual machine image, you will need to connect to the graphical console of the hypervisor, which acts as the virtual machines display and allows you to interact with the guest operating systems installer using your keyboard and mouse. KVM can expose the graphical console using the VNC (Virtual Network Computing) protocol or the newer SPICE protocol. We will use the VNC protocol here, since you are more likely to find a VNC client that works on your local desktop.

To create an image for the Database service, see Building Guest Images for OpenStack Trove.

2.6.1 Tools

Tools: libvirt and virsh/virt-manager

Contents

- *Tools: libvirt and virsh/virt-manager*
  - *Prerequisites*
    - *Verify the libvirt default network is running*
  - *Use the virt-manager X11 GUI*
  - *Use virt-install and connect by using a local VNC client*

Prerequisites

Verify the libvirt default network is running

Before starting a virtual machine with libvirt, verify that the libvirt `default` network has started. This network must be active for your virtual machine to be able to connect out to the network. Starting this network will create a Linux bridge (usually called `virbr0`), iptables rules, and a dnsmasq process that will serve as a DHCP server.

To verify that the libvirt `default` network is enabled, use the **`virsh net-list`** command and verify that the `default` network is active:

```
# virsh net-list
Name                 State      Autostart
----------------------------------------
default              active     yes
```

If the network is not active, start it by doing:

```
# virsh net-start default
```

Use the virt-manager X11 GUI

If you plan to create a virtual machine image on a machine that can run X11 applications, the simplest way to do so is to use the **`virt-manager`** GUI, which is installable as the `virt-manager` package on both Fedora-based and Debian-based systems. This GUI has an embedded VNC client that will let you view and interact with the guests graphical console.

If you are building the image on a headless server, and you have an X server on your local machine, you can launch **`virt-manager`** using ssh X11 forwarding to access the GUI. Since virt-manager interacts directly with libvirt, you typically need to be root to access it. If you can ssh directly in as root (or with a user that has permissions to interact with libvirt), do:

```
$ ssh -X root@server virt-manager
```

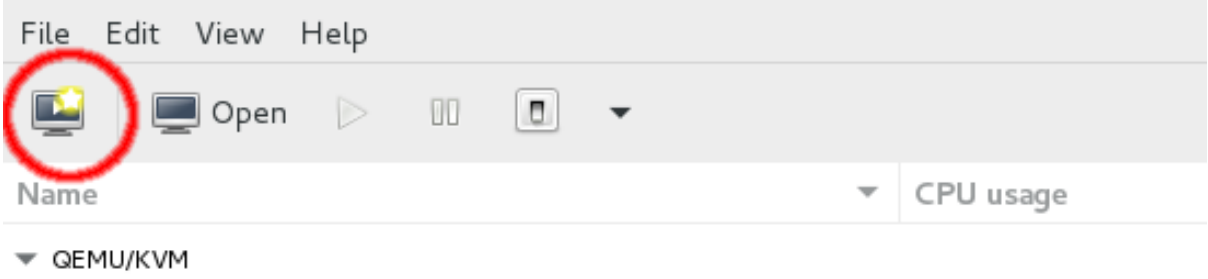
If the account you use to ssh into your server does not have permissions to run libvirt, but has sudo privileges, do:

```
$ ssh -X user@server
$ sudo virt-manager
```

Note

The `-X` flag passed to ssh will enable X11 forwarding over ssh. If this does not work, try replacing it with the `-Y` flag.

Click the *Create a new virtual machine* button at the top-left, or go to *File New Virtual Machine*. Then, follow the instructions.



You will be shown a series of dialog boxes that will allow you to specify information about the virtual machine.

Note

When using qcow2 format images, you should check the option `Customize configuration before install`, go to disk properties and explicitly select the *qcow2* format. This ensures the virtual machine disk size will be correct.

Use virt-install and connect by using a local VNC client

If you do not wish to use **`virt-manager`** (for example, you do not want to install the dependencies on your server, you do not have an X server running locally, the X11 forwarding over SSH is not working), you can use the **`virt-install`** tool to boot the virtual machine through libvirt and connect to the graphical console from a VNC client installed on your local machine.

Because VNC is a standard protocol, there are multiple clients available that implement the VNC spec, including TigerVNC (multiple platforms), TightVNC (multiple platforms), RealVNC (multiple platforms), Chicken (Mac OS X), Krde (KDE), Vinagre (GNOME).

The following example shows how to use the **`qemu-img`** command to create an empty image file, and **`virt-install`** command to start up a virtual machine using that image file. As root:

```
# qemu-img create -f qcow2 /tmp/centos.qcow2 10G
# virt-install --virt-type kvm --name centos --ram 1024 \
  --disk /tmp/centos.qcow2,format=qcow2 \
  --network network=default \
  --graphics vnc,listen=0.0.0.0 --noautoconsole \
  --os-type=linux --os-variant=centos7.0 \
  --location=/data/isos/CentOS-7-x86_64-NetInstall-1611.iso
```

(continues on next page)

(continued from previous page)

```
Starting install...
Creating domain...                        |    0 B     00:00
Domain installation still in progress. You can reconnect to
the console to complete the installation process.
```

The KVM hypervisor starts the virtual machine with the libvirt name, `centos`, with 1024 MB of RAM. The virtual machine also has a virtual CD-ROM drive associated with the `/data/isos/CentOS-7-x86_64-NetInstall-1611.iso` file and a local 10 GB hard disk in qcow2 format that is stored in the host at `/tmp/centos.qcow2`. It configures networking to use libvirt default network. There is a VNC server that is listening on all interfaces, and libvirt will not attempt to launch a VNC client automatically nor try to display the text console (`--no-autoconsole`). Finally, libvirt will attempt to optimize the configuration for a Linux guest running a CentOS 7 distribution.

Note

When using the libvirt `default` network, libvirt will connect the virtual machines interface to a bridge called `virbr0`. There is a dnsmasq process managed by libvirt that will hand out an IP address on the 192.168.122.0/24 subnet, and libvirt has iptables rules for doing NAT for IP addresses on this subnet.

Run the **`osinfo-query os`** command to see a range of allowed `--os-variant` options.

Use the **`virsh vncdisplay vm-name`** command to get the VNC port number.

```
# virsh vncdisplay centos
:1
```

In the example above, the guest `centos` uses VNC display `:1`, which corresponds to TCP port `5901`. You should be able to connect a VNC client running on your local machine to display `:1` on the remote machine and step through the installation process.

2.6.2 Examples

Example: CentOS Stream image

This example shows you how to install a CentOS Stream image and focuses mainly on CentOS Stream 9. Because the CentOS installation process might differ across versions, the installation steps might differ if you use a different version of CentOS.

Contents

- *Example: CentOS Stream image*
  - *Download a CentOS install ISO*
  - *Start the installation process*
    - *virt-install*
    - *virt-manager*
  - *Step through the installation*

        * *DHCP and hostname*
        * *Select installation option*
        * *Create a working user*
        * *Detach the CD-ROM and reboot*
    - *Finalize installation*
        * *Install the ACPI service*
        * *Configure to fetch metadata*
        * *Cloud-init alternatives*
        * *Allow partitions to resize*
        * *Configure console*
        * *Customize*
        * *Shut down the instance*
    - *Clean up*
    - *Resize the image*
    - *Image is complete*
    - *Undefine the libvirt domain*

Note

This is just an example, adjust paths and commands according to your environment

Download a CentOS install ISO

- Navigate to the CentOS mirrors page.
- Choose one of the mirrors and navigate to `9-stream/BaseOS/x86_64/iso`. Download a ISO, choose `boot` to download packages during install, otherwise choose `dvd`

Start the installation process

Start the installation process using either the **`virt-manager`** or the **`virt-install`** command as described in the Tools: libvirt and virsh/virt-manager page.

virt-install

If you use the **`virt-install`** command, do not forget to connect your VNC client to the virtual machine.

The command should look something like this:

```
$ sudo virt-install --virt-type kvm --name my-centos --ram 2048 \
  --network network=default \
  --graphics vnc,listen=0.0.0.0 --noautoconsole \
```

(continues on next page)

(continued from previous page)

```
  --os-type=linux --os-variant=centos-stream9 \
  --location=~/Downloads/CentOS-Stream-9-20240819.0-x86_64-dvd1.iso
```

virt-manager

Note

See here for libvirt wiki about new VM creation in virt-manager

When creating a new VM from the downloaded ISO, **`virt-manager`** should automatically detect the OS. If it fails, manually select CentOS Stream as OS. Default settings should be fine.

Step through the installation

Note

This guide focuses on the steps specific to create a OpenStack image, for a general overview of installation process see CentOS Documentation

In `Installation Summary` follow the instructions below.

DHCP and hostname

In `Network & Host Name` ensure that `Ethernet` is on and in `Configure.../IPv4 Settings` the `Method` is set to `Automatic (DHCP)`. The same page allows for host name selection - leave it to default as the `cloud-init` package will be installed later.

Select installation option

In `Software Selection` choose what to install, the default is `Server with GUI`, the smallest choice is `Minimal Install`.

Create a working user

Configure a root password in `Root Password`, as this will be needed later to finalize the installation. By default it will be then blocked by cloud-init. It is also possible to create an adiministrator user, as it will be later deleted by virt-sysprep.

Detach the CD-ROM and reboot

virt-install

To eject a disk by using the **`virsh`** command, libvirt requires that you attach an empty disk at the same target that the CD-ROM was previously attached, which may be `hda`. You can confirm the appropriate target using the **`virsh dumpxml vm-image`** command.

```
$ sudo virsh dumpxml my-centos
<domain type='kvm' id='19'>
  <name>centos</name>
...
    <disk type='block' device='cdrom'>
      <driver name='qemu' type='raw'/>
      <target dev='hda' bus='ide'/>
      <readonly/>
      <address type='drive' controller='0' bus='1' target='0' unit='0'/>
    </disk>
...
</domain>
```

Run the following commands from the host to eject the disk and reboot using `virsh`, as root.

```
$ sudo virsh attach-disk --type cdrom --mode readonly my-centos "" hda
$ sudo virsh reboot my-centos
```

virt-manager

If you are using `virt-manager`, the commands above will work, but you can also use the GUI to detach and reboot.

Finalize installation

Install the ACPI service

To enable the hypervisor to reboot or shutdown an instance, you must install and run the `acpid` service on the guest system.

Log in to the CentOS guest and run the following commands to install the ACPI service and configure it to start when the system boots:

```
# dnf install acpid
# systemctl enable acpid
```

Configure to fetch metadata

> **Note**
>
> Check cloud-init documentation for more information

An instance must interact with the metadata service to perform several tasks on start up. For example, the instance must get the ssh public key and run the user data script. To ensure that the instance performs these tasks, install the `cloud-init` utility.

The `cloud-init` package automatically fetches the public key from the metadata server and places the key in an account. Install `cloud-init` inside the CentOS guest by running:

```
# dnf install cloud-init
```

The account varies by distribution. On CentOS Stream virtual machines, the account is called `cloud-user`.

You can change the name of the account used by `cloud-init` by editing the `/etc/cloud/cloud.cfg` file and adding a line with a different user. For example, to configure `cloud-init` to put the key in an account named `admin`, use the following syntax in the configuration file:

```
users:
  - name: admin
    (...)
```

Cloud-init alternatives

Warning

This method is not recommended as only gets the ssh public key from the metadata server. It does not get user data, which is optional data that can be passed by the user when requesting a new instance. User data is often used to run a custom script when an instance boots.

If you are not able to install the `cloud-init` package in your image, to fetch the ssh public key and add it to the root account, edit the `/etc/rc.d/rc.local` file and add the following lines before the line `touch /var/lock/subsys/local`:

```
if [ ! -d /root/.ssh ]; then
  mkdir -p /root/.ssh
  chmod 700 /root/.ssh
fi

# Fetch public key using HTTP
ATTEMPTS=30
FAILED=0
while [ ! -f /root/.ssh/authorized_keys ]; do
  curl -f http://169.254.169.254/latest/meta-data/public-keys/0/openssh-key \
    > /tmp/metadata-key 2>/dev/null
  if [ \$? -eq 0 ]; then
    cat /tmp/metadata-key >> /root/.ssh/authorized_keys
    chmod 0600 /root/.ssh/authorized_keys
    restorecon /root/.ssh/authorized_keys
    rm -f /tmp/metadata-key
    echo "Successfully retrieved public key from instance metadata"
    echo "*****************"
    echo "AUTHORIZED KEYS"
    echo "*****************"
    cat /root/.ssh/authorized_keys
    echo "*****************"
  fi
done
```

Note

Some VNC clients replace the colon (`:`) with a semicolon (`;`) and the underscore (_) with a hyphen (-). Make sure to specify `http:` and not `http;`. Make sure to specify `authorized_keys` and not `authorized-keys`.

Note

With this method, previous CentOS versions needed to disable the zeroconf route, this is not needed as of EL8.

Allow partitions to resize

In order for the root partition to properly resize, install the `cloud-utils-growpart` package, which contains the proper tools to allow the disk to resize using cloud-init.

```
# dnf install cloud-utils-growpart
```

Configure console

Warning

Actually this does not work even following https://access.redhat.com/solutions/3443551

For the **`openstack console log`** command to work properly, instance must be configured to sent output to a serial console.

1. Edit the `/etc/default/grub` file and append `console=tty0 console=ttyS0,115200n8` to `GRUB_CMDLINE_LINUX`.

   For example:

   ```
   ...
   GRUB_CMDLINE_LINUX="resume=/dev/mapper/cs-swap rd.lvm.lv=cs/root rd.lvm.
   ↪lv=cs/swap console=tty0 console=ttyS0,115200n8"
   ```

2. Then regenerate GRUB configuration:

   ```
   # grub2-mkconfig -o /boot/grub2/grub.cfg --update-bls-cmdline
   ```

Customize

If you are building an image manually, you are probably interested in customizing it, now its time to apply your edits.

Shut down the instance

Shutdown the VM:

```
# shutdown now
```

Clean up

The image needs to be cleaned up of details such as the MAC address - the tool used is virt-sysprep, part of libguestfs

```
sudo virt-sysprep -d my-centos
```

Resize the image

Resize and compress the image to remove unused space (the image will then use available space after initialization) using virt-sparsify, part of libguestfs

```
sudo virt-sparsify --compress /var/lib/libvirt/images/my-centos.qcow2 ~/
↪upload.qcow2
```

Image is complete

The resized image is now ready to be uploaded using **`openstack image create`**. For more information, see the python-openstackclient command list.

Undefine the libvirt domain

Now that you can upload the image to the Image service, you no longer need to have this virtual machine image managed by libvirt.

```
$ sudo virsh undefine my-centos
```

Or if you used **`virt-manager`**, delete it through the GUI.

Example: Fedora image

This example shows you how to install a Fedora image and focuses mainly on Fedora 25. Because the Fedora installation process might differ across versions, the installation steps might differ if you use a different version of Fedora.

Contents

- *Example: Fedora image*
  - *Download a Fedora install ISO*
  - *Start the installation process*
  - *Step through the installation*
  - *Install the ACPI service*

- *Configure cloud-init to fetch metadata*
- *Install cloud-utils-growpart to allow partitions to resize*
- *Disable the zeroconf route*
- *Configure console*
- *Shut down the instance*
- *Clean up (remove MAC address details)*
- *Undefine the libvirt domain*
- *Image is complete*

Download a Fedora install ISO

1. Visit the Fedora download site.
2. Navigate to the Download Fedora Server page for a Fedora Server ISO image.
3. Choose the ISO image you want to download.

   For example, the `Netinstall Image` is a good choice because it is a smaller image that downloads missing packages from the Internet during installation.

Start the installation process

Start the installation process using either the **virt-manager** or the **virt-install** command as described previously. If you use the **virt-install** command, do not forget to connect your VNC client to the virtual machine.

Assume that:

- The name of your virtual machine image is `fedora`; you need this name when you use **virsh** commands to manipulate the state of the image.
- You saved the netinstall ISO image to the `/tmp` directory.

If you use the **virt-install** command, the commands should look something like this:

```
# qemu-img create -f qcow2 /tmp/fedora.qcow2 10G
# virt-install --virt-type kvm --name fedora --ram 1024 \
  --disk /tmp/fedora.qcow2,format=qcow2 \
  --network network=default \
  --graphics vnc,listen=0.0.0.0 --noautoconsole \
  --os-type=linux --os-variant=fedora23 \
  --location=/tmp/Fedora-Server-netinst-x86_64-25-1.3.iso
```

Step through the installation

After the installation program starts, choose your preferred language and click *Continue* to get to the installation summary. Accept the defaults.

Review the Ethernet status

Ensure that the Ethernet setting is `ON`. Additionally, make sure that `IPv4 Settings' Method` is `Automatic (DHCP)`, which is the default.

Hostname

The installer allows you to choose a host name. The default (`localhost.localdomain`) is fine. You install the `cloud-init` package later, which sets the host name on boot when a new instance is provisioned using this image.

Partition the disks

There are different options for partitioning the disks. The default installation uses LVM partitions, and creates three partitions (`/boot`, `/`, `swap`), which works fine. Alternatively, you might want to create a single ext4 partition that is mounted to `/`, which also works fine.

If unsure, use the default partition scheme for the installer. While no scheme is inherently better than another, having the partition that you want to dynamically grow at the end of the list will allow it to grow without crossing another partitions boundary.

Select software to install

Step through the installation, using the default options. The simplest thing to do is to choose the `Minimal Install` install, which installs an SSH server.

Set the root password

During the installation, remember to set the root password when prompted.

Detach the CD-ROM and reboot

Wait until the installation is complete.

To eject a disk by using the **`virsh`** command, libvirt requires that you attach an empty disk at the same target that the CD-ROM was previously attached, which may be `hda`. You can confirm the appropriate target using the **`virsh dumpxml vm-image`** command.

```
# virsh dumpxml fedora
<domain type='kvm' id='30'>
  <name>fedora</name>
...
    <disk type='file' device='cdrom'>
      <driver name='qemu' type='raw'/>
      <source file='/tmp/Fedora-Server-netinst-x86_64-25-1.3.iso'/>
      <backingStore/>
      <target dev='hda' bus='ide'/>
      <readonly/>
      <alias name='ide0-0-0'/>
      <address type='drive' controller='0' bus='0' target='0' unit='0'/>
    </disk>
...
</domain>
```

Run the following commands from the host to eject the disk and reboot using `virsh`, as root. If you are using `virt-manager`, the commands below will work, but you can also use the GUI to detach and reboot it by manually stopping and starting.

```
# virsh attach-disk --type cdrom --mode readonly fedora "" hda
# virsh reboot fedora
```

Install the ACPI service

To enable the hypervisor to reboot or shutdown an instance, you must install and run the `acpid` service on the guest system.

Log in as root to the Fedora guest and run the following commands to install the ACPI service and configure it to start when the system boots:

```
# dnf install acpid
# systemctl enable acpid
```

Configure cloud-init to fetch metadata

An instance must interact with the metadata service to perform several tasks on start up. For example, the instance must get the ssh public key and run the user data script. To ensure that the instance performs these tasks, use the `cloud-init` package.

The `cloud-init` package automatically fetches the public key from the metadata server and places the key in an account. Install `cloud-init` inside the Fedora guest by running:

```
# dnf install cloud-init
```

The account varies by distribution. On Fedora-based virtual machines, the account is called `fedora`.

You can change the name of the account used by `cloud-init` by editing the `/etc/cloud/cloud.cfg` file and adding a line with a different user. For example, to configure `cloud-init` to put the key in an account named `admin`, use the following syntax in the configuration file:

```
users:
  - name: admin
    (...)
```

Install cloud-utils-growpart to allow partitions to resize

In order for the root partition to properly resize, install the `cloud-utils-growpart` package, which contains the proper tools to allow the disk to resize using cloud-init.

```
# dnf install cloud-utils-growpart
```

Disable the zeroconf route

For the instance to access the metadata service, you must disable the default zeroconf route:

```
# echo "NOZEROCONF=yes" >> /etc/sysconfig/network
```

Configure console

For the **nova console-log** command to work properly on Fedora, you might need to do the following steps:

1. Edit the `/etc/default/grub` file and configure the `GRUB_CMDLINE_LINUX` option. Delete the `rhgb quiet` and add `console=tty0 console=ttyS0,115200n8` to the option. For example:

```
...
GRUB_CMDLINE_LINUX="rd.lvm.lv=fedora/root rd.lvm.lv=fedora/swap␣
↪console=tty0 console=ttyS0,115200n8"
```

2. Run the following command to save the changes:

```
# grub2-mkconfig -o /boot/grub2/grub.cfg
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-4.10.10-200.fc25.x86_64
Found initrd image: /boot/initramfs-4.10.10-200.fc25.x86_64.img
Found linux image: /boot/vmlinuz-0-rescue-c613978614c7426ea3e550527f63710c
Found initrd image: /boot/initramfs-0-rescue-
↪c613978614c7426ea3e550527f63710c.img
done
```

Shut down the instance

From inside the instance, run as root:

```
# poweroff
```

Clean up (remove MAC address details)

The operating system records the MAC address of the virtual Ethernet card in locations such as `/etc/sysconfig/network-scripts/ifcfg-eth0` during the instance process. However, each time the image boots up, the virtual Ethernet card will have a different MAC address, so this information must be deleted from the configuration file.

There is a utility called **virt-sysprep**, that performs various cleanup tasks such as removing the MAC address references. It will clean up a virtual machine image in place:

```
# virt-sysprep -d fedora
```

Undefine the libvirt domain

Now that you can upload the image to the Image service, you no longer need to have this virtual machine image managed by libvirt. Use the **virsh undefine vm-image** command to inform libvirt:

```
# virsh undefine fedora
```

Image is complete

The underlying image file that you created with the **qemu-img create** command is ready to be uploaded. For example, you can upload the `/tmp/fedora.qcow2` image to the Image service by using the **openstack image create** command. For more information, see the python-openstackclient command list.

Example: FreeBSD image

This example creates a minimal FreeBSD image that is compatible with OpenStack and `bsd-cloudinit`. The `bsd-cloudinit` program is independently maintained and in active development. The best source of information on the current state of the project is at bsd-cloudinit.

KVM with virtio drivers is used as the virtualization platform because that is the most widely used among OpenStack operators. If you use a different platform for your cloud virtualization, use that same platform in the image creation step.

This example shows how to create a FreeBSD 10 image. To create a FreeBSD 9.2 image, follow these steps with the noted differences.

Contents

- *Example: FreeBSD image*
  - *Prerequisites*
  - *Installation*

Prerequisites

1. Make a virtual drive:

```
$ qemu-img create -f qcow2 freebsd.qcow2 1G
```

   The minimum supported disk size for FreeBSD is 1 GB. Because the goal is to make the smallest possible base image, the example uses that minimum size. This size is sufficient to include the optional `doc`, `games`, and `lib32` collections. To include the `ports` collection, add another 1 GB. To include `src`, add 512 MB.

2. Get the installer ISO:

```
$ curl ftp://ftp.freebsd.org/pub/FreeBSD/releases/amd64/amd64/ISO-IMAGES/
↪10.1/FreeBSD-10.1-RELEASE-amd64-bootonly.iso \
  > FreeBSD-10.1-RELEASE-amd64-bootonly.iso
```

3. Launch a VM on your local workstation. Use the same hypervisor, virtual disk, and virtual network drivers as you use in your production environment.

   The following command uses the minimum amount of RAM, which is 256 MB:

```
$ kvm -smp 1 -m 256 -cdrom FreeBSD-10.1-RELEASE-amd64-bootonly.iso \
  -drive if=virtio,file=freebsd.qcow2 \
  -net nic,model=virtio -net user
```

You can specify up to 1 GB additional RAM to make the installation process run faster.

This VM must also have Internet access to download packages.

Note

By using the same hypervisor, you can ensure that you emulate the same devices that exist in production. However, if you use full hardware virtualization instead of paravirtualization, you do not need to use the same hypervisor; you must use the same type of virtualized hardware because FreeBSD device names are related to their drivers. If the name of your root block device or primary network interface in production differs than the names used during image creation, errors can occur.

You now have a VM that boots from the downloaded install ISO and is connected to the blank virtual disk that you created previously.

Installation

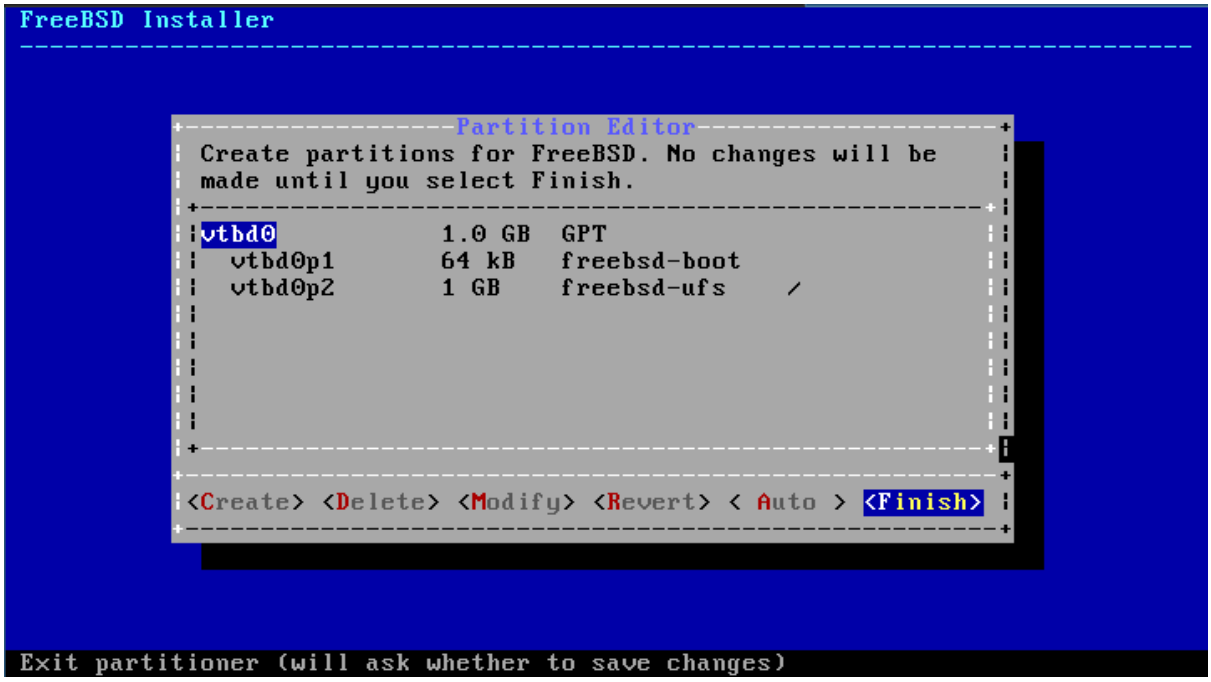
1. To install the operating system, complete the following steps inside the VM:
    1. When prompted, choose to run the ISO in *Install* mode.
    2. Accept the default keymap or select an appropriate mapping for your needs.
    3. Provide a host name for your image. If you use `bsd-cloudinit`, it overrides this value with the name provided by OpenStack when an instance boots from this image.
    4. When prompted about the optional `doc`, `games`, `lib32`, `ports`, and `src` system components, select only those that you need. It is possible to have a fully functional installation without selecting additional components selected. As noted previously, a minimal system with a 1 GB virtual disk supports `doc`, `games`, and `lib32` inclusive. The `ports` collection requires at least 1 GB additional space and possibly more if you plan to install many ports. The `src` collection requires an additional 512 MB.
    5. Configure the primary network interface to use DHCP. In this example, which uses a virtio network device, this interface is named `vtnet0`.
    6. Accept the default network mirror.
    7. Set up disk partitioning.

        Disk partitioning is a critical element of the image creation process and the auto-generated default partitioning scheme does not work with `bsd-cloudinit` at this time.

        Because the default does not work, you must select manual partitioning. The partition editor should list only one block device. If you use virtio for the disk device driver, it is named `vtbd0`. Select this device and run the **`create`** command three times:

        1. Select *Create* to create a partition table. This action is the default when no partition table exists. Then, select *GPT GUID Partition Table* from the list. This choice is the default.
        2. Create two partitions:
            - First partition: A 64 kB `freebsd-boot` partition with no mount point.
            - Second partition: A `freebsd-ufs` partition with a mount point of `/` with all remaining free space.

The following figure shows a completed partition table with a 1 GB virtual disk:



Select *Finish* and then *Commit* to commit your changes.

> **Note**
>
> If you modify this example, the root partition, which is mounted on `/`, must be the last partition on the drive so that it can expand at run time to the disk size that your instance type provides. Also note that `bsd-cloudinit` currently has a hard-coded assumption that this is the second partition.

2. Select a root password.
3. Select the CMOS time zone.

   The virtualized CMOS almost always stores its time in UTC, so unless you know otherwise, select UTC.
4. Select the time zone appropriate to your environment.
5. From the list of services to start on boot, you must select *ssh*. Optionally, select other services.
6. Optionally, add users.

   You do not need to add users at this time. The `bsd-cloudinit` program adds a `freebsd` user account if one does not exist. The `ssh` keys for this user are associated with OpenStack. To customize this user account, you can create it now. For example, you might want to customize the shell for the user.
7. Final config

   This menu enables you to update previous settings. Check that the settings are correct, and click *exit*.
8. After you exit, you can open a shell to complete manual configuration steps. Select *Yes* to make a few OpenStack-specific changes:

1. Set up the console:

```
# echo 'console="comconsole,vidconsole"' >> /boot/loader.conf
```

   This sets console output to go to the serial console, which is displayed by **nova consolelog**, and the video console for sites with VNC or Spice configured.

2. Minimize boot delay:

```
# echo 'autoboot_delay="1"' >> /boot/loader.conf
```

3. Download the latest `bsd-cloudinit-installer`. The download commands differ between FreeBSD 10.1 and 9.2 because of differences in how the **fetch** command handles HTTPS URLs.

   In FreeBSD 10.1 the **fetch** command verifies SSL peers by default, so you need to install the `ca_root_nss` package that contains certificate authority root certificates and tell **fetch** where to find them. For FreeBSD 10.1 run these commands:

```
# pkg install ca_root_nss
# fetch --ca-cert=/usr/local/share/certs/ca-root-nss.crt \
  https://raw.github.com/pellaeon/bsd-cloudinit-installer/master/
↪installer.sh
```

   FreeBSD 9.2 **fetch** does not support peer-verification for https. For FreeBSD 9.2, run this command:

```
# fetch https://raw.github.com/pellaeon/bsd-cloudinit-installer/
↪master/installer.sh
```

4. Run the installer:

```
# sh ./installer.sh
```

   Issue this command to download and install the latest `bsd-cloudinit` package, and install the necessary prerequisites.

5. Install `sudo` and configure the `freebsd` user to have passwordless access:

```
# pkg install sudo
# echo 'freebsd ALL=(ALL) NOPASSWD: ALL' > /usr/local/etc/sudoers.d/
↪10-cloudinit
```

9. Power off the system:

```
# shutdown -h now
```

Example: Microsoft Windows image

Contents

- *Example: Microsoft Windows image*

- *Prerequisites*
- *Installation*
- *Image ready*

This example creates a Windows Server 2012 qcow2 image, using the **virt-install** command and the KVM hypervisor.

Prerequisites

1. Follow these steps to prepare the installation:
    1. Download a Windows Server 2012 installation ISO. Evaluation images are available on the Microsoft website (registration required).
    2. Download the signed VirtIO drivers ISO from the Fedora website.
    3. Create a 15 GB qcow2 image:

```
$ qemu-img create -f qcow2 ws2012.qcow2 15G
```

Installation

1. Start the Windows Server 2012 installation with the **virt-install** command:

```
# virt-install --connect qemu:///system \
  --name ws2012 --ram 2048 --vcpus 2 \
  --network network=default,model=virtio \
  --disk path=ws2012.qcow2,format=qcow2,device=disk,bus=virtio \
  --cdrom /path/to/en_windows_server_2012_x64_dvd.iso \
  --disk path=/path/to/virtio-win-0.1-XX.iso,device=cdrom \
  --vnc --os-type windows --os-variant win2k12 \
  --os-distro windows --os-version 2012
```

   Use **virt-manager** or **virt-viewer** to connect to the VM and start the Windows installation.
2. Enable the VirtIO drivers. By default, the Windows installer does not detect the disk.
3. Load VirtIO SCSI drivers and network drivers by choosing an installation target when prompted. Click *Load driver* and browse the file system.
4. Select the `E:\virtio-win-0.1XX\viostor\2k12\amd64` folder. The Windows installer displays a list of drivers to install.
5. Select the VirtIO SCSI drivers.
6. Click *Load driver* and browse the file system, and select the `E:\NETKVM\2k12\amd64` folder.
7. Select the network drivers, and continue the installation. Once the installation is completed, the VM restarts.
8. Define a password for the administrator when prompted.
9. Log in as administrator and start a command window.
10. Complete the VirtIO drivers installation by running the following command:

```
C:\pnputil -i -a E:\virtio-win-0.1XX\viostor\2k12\amd64\*.INF
```

11. To allow the *Cloudbase-Init* to run scripts during an instance boot, set the PowerShell execution policy to be unrestricted:

```
C:\powershell
C:\Set-ExecutionPolicy Unrestricted
```

12. Download and install the `Cloudbase-Init`:

```
C:\Invoke-WebRequest -UseBasicParsing https://cloudbase.it/downloads/
↪CloudbaseInitSetup_Stable_x64.msi -OutFile cloudbaseinit.msi
C:\.\cloudbaseinit.msi
```

In the *configuration options* window, change the following settings:

- Username: `Administrator`
- Network adapter to configure: `Red Hat VirtIO Ethernet Adapter`
- Serial port for logging: `COM1`

When the installation is done, in the *Complete the Cloudbase-Init Setup Wizard* window, select the *Run Sysprep* and *Shutdown* check boxes and click *Finish*.

Wait for the machine shutdown.

Image ready

Your image is ready to upload to the Image service:

```
$ openstack image create --disk-format qcow2 --file ws2012.qcow2 WS2012
```

Example: Ubuntu image

This example installs an Ubuntu 18.04 (Bionic Beaver) image. To create an image for a different version of Ubuntu, follow these steps with the noted differences.

Contents

- *Example: Ubuntu image*
  - *Download an Ubuntu installation ISO*
  - *Start the installation process*
  - *Step through the installation*
  - *Log in to newly created image*
  - *Install cloud-init*
  - *Shut down the instance*
  - *Clean up (remove MAC address details)*

- *Undefine the libvirt domain*
- *Image is complete*

Download an Ubuntu installation ISO

Because the goal is to make the smallest possible base image, this example uses the network installation ISO. The Ubuntu 64-bit 18.04 network installation ISO is at the Ubuntu download page.

Start the installation process

Start the installation process by using either **virt-manager** or **virt-install** as described in the previous section. If you use **virt-install**, do not forget to connect your VNC client to the virtual machine.

Assume that the name of your virtual machine image is `ubuntu-18.04`, which you need to know when you use **virsh** commands to manipulate the state of the image.

If you are using **virt-manager**, the commands should look something like this:

```
# wget -O /var/lib/libvirt/boot/bionic-mini.iso \
  http://archive.ubuntu.com/ubuntu/dists/bionic/main/installer-amd64/current/
↪images/netboot/mini.iso
# chown libvirt-qemu:kvm /var/lib/libvirt/boot/bionic-mini.iso
# qemu-img create -f qcow2 /var/lib/libvirt/images/bionic.qcow2 10G
# chown libvirt-qemu:kvm /var/lib/libvirt/images/bionic.qcow2
# virt-install --virt-type kvm --name bionic --ram 1024 \
  --cdrom=/var/lib/libvirt/boot/bionic-mini.iso \
  --disk /var/lib/libvirt/images/bionic.qcow2,bus=virtio,size=10,format=qcow2␣
↪\
  --network network=default \
  --graphics vnc,listen=0.0.0.0 --noautoconsole \
  --os-type=linux --os-variant=ubuntu18.04
```

Step through the installation

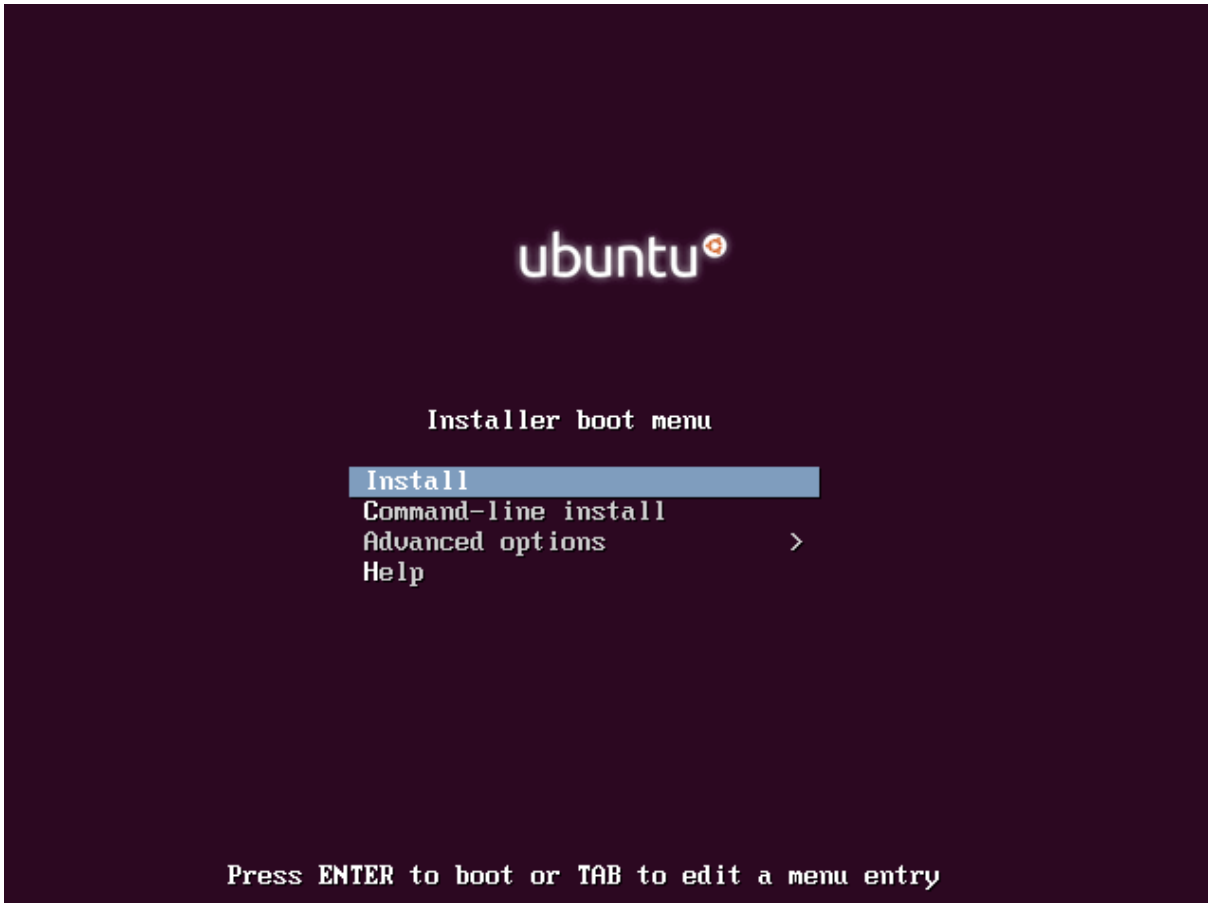
At the initial Installer boot menu, choose the *Install* option. Step through the installation prompts, the defaults should be fine.

Hostname

The installer may ask you to choose a host name. The default (`ubuntu`) is fine. We will install the cloud-init package later, which will set the host name on boot when a new instance is provisioned using this image.

Select a mirror

The default mirror proposed by the installer should be fine.



Step through the install

Step through the install, using the default options. When prompted for a user name, the default (`ubuntu`) is fine.

Partition the disks

There are different options for partitioning the disks. The default installation will use LVM partitions, and will create three partitions (`/boot`, `/`, swap), and this will work fine. Alternatively, you may wish to create a single ext4 partition, mounted to `/`, should also work fine.

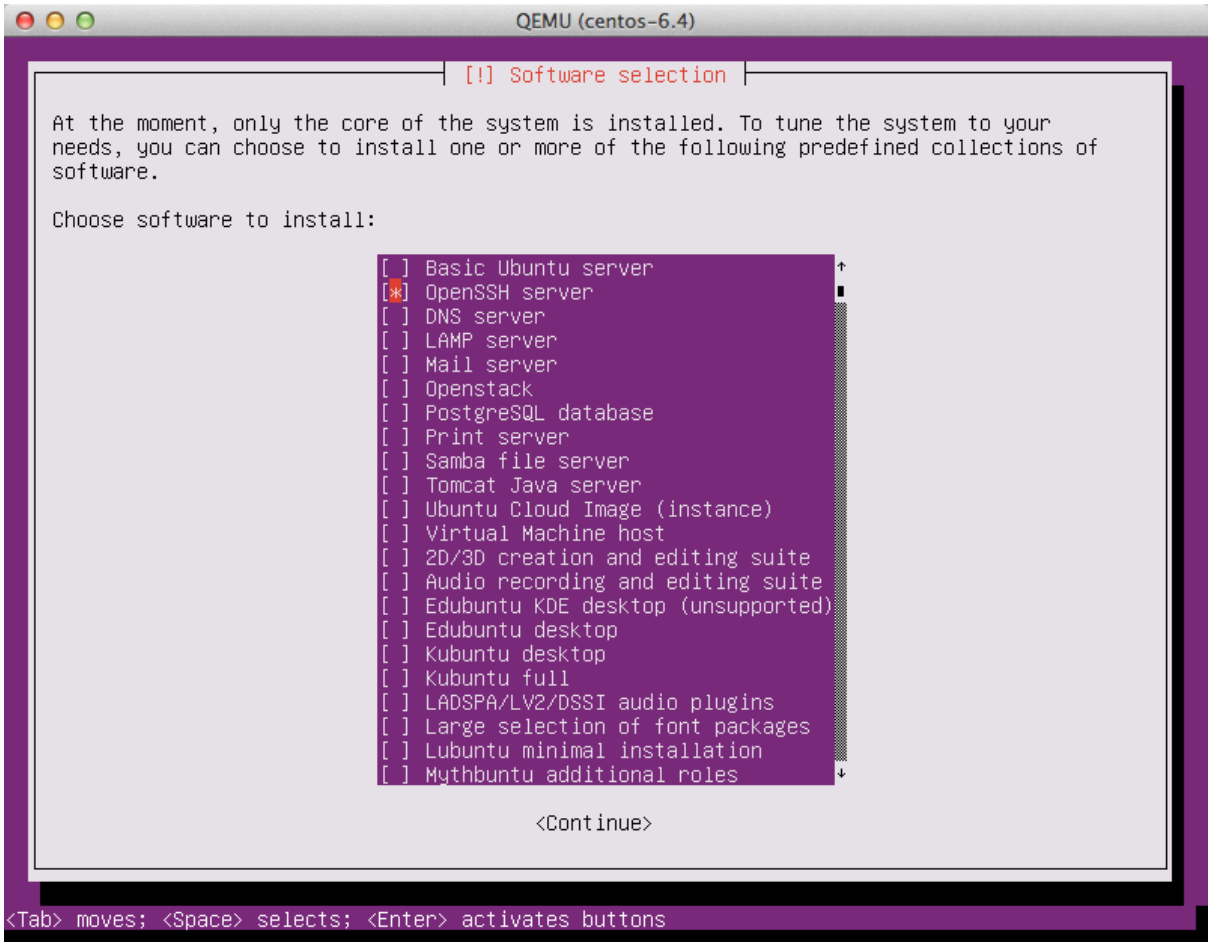
If unsure, we recommend you use the installers default partition scheme, since there is no clear advantage to one scheme or another.

Automatic updates

The Ubuntu installer will ask how you want to manage upgrades on your system. This option depends on your specific use case. If your virtual machine instances will be connected to the Internet, we recommend Install security updates automatically.

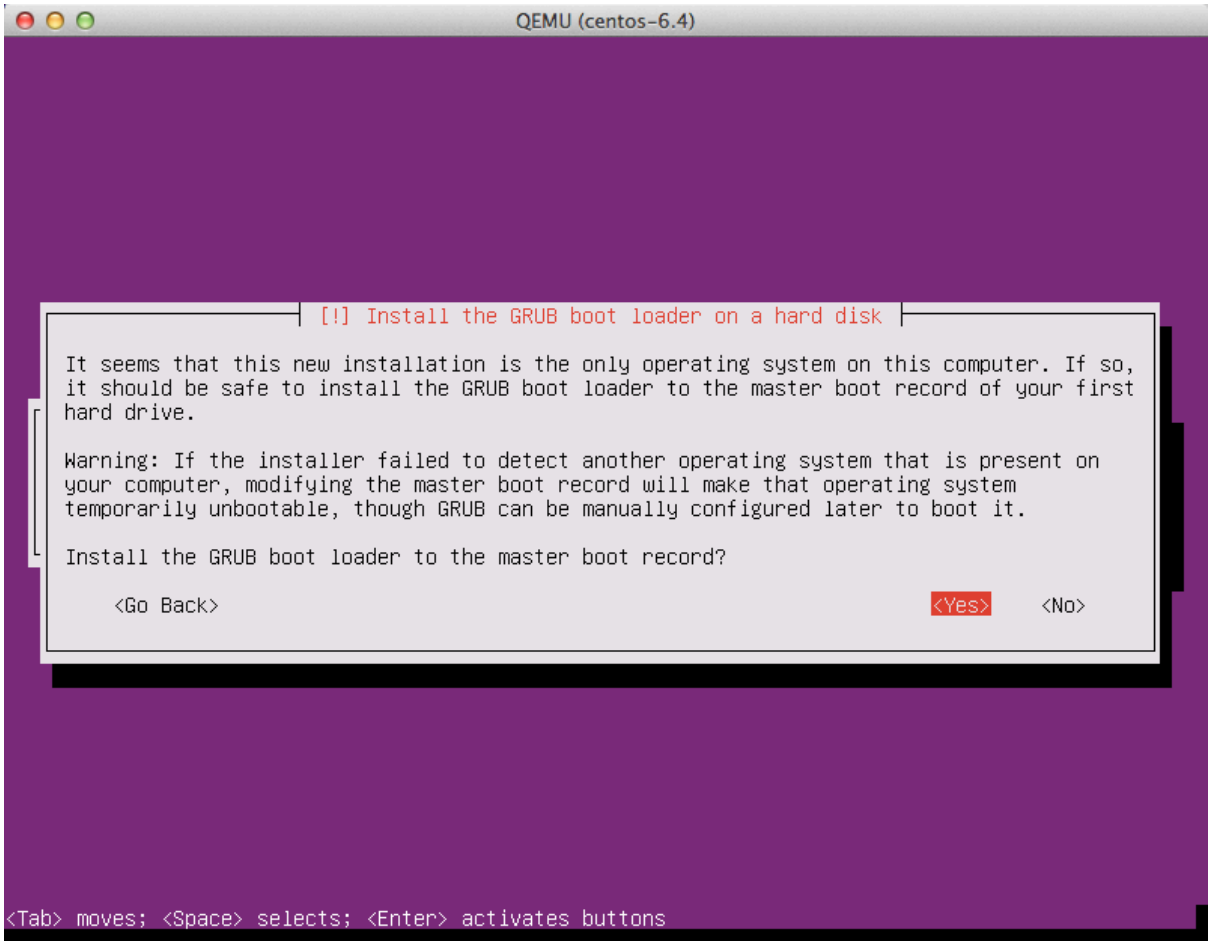
Software selection: OpenSSH server

Choose *OpenSSH server* so that you will be able to SSH into the virtual machine when it launches inside of an OpenStack cloud.


QEMU (centos-6.4)
[!] Software selection
At the moment, only the core of the system is installed. To tune the system to your needs, you can choose to install one or more of the following predefined collections of software.
Choose software to install:
[ ] Basic Ubuntu server
[*] OpenSSH server
[ ] DNS server
[ ] LAMP server
[ ] Mail server
[ ] Openstack
[ ] PostgreSQL database
[ ] Print server
[ ] Samba file server
[ ] Tomcat Java server
[ ] Ubuntu Cloud Image (instance)
[ ] Virtual Machine host
[ ] 2D/3D creation and editing suite
[ ] Audio recording and editing suite
[ ] Edubuntu KDE desktop (unsupported)
[ ] Edubuntu desktop
[ ] Kubuntu desktop
[ ] Kubuntu full
[ ] LADSPA/LV2/DSSI audio plugins
[ ] Large selection of font packages
[ ] Lubuntu minimal installation
[ ] Mythbuntu additional roles
<Continue>
<Tab> moves; <Space> selects; <Enter> activates buttons

Install GRUB boot loader

Select *Yes* when asked about installing the GRUB boot loader to the master boot record.



For more information on configuring Grub, see the section called *Ensure image writes boot log to console*.

Log in to newly created image

When you boot for the first time after install, it may ask you about authentication tools, you can just choose *Exit*. Then, log in as admin user using the password you specified.

Install cloud-init

The **cloud-init** script starts on instance boot and will search for a metadata provider to fetch a public key from. The public key will be placed in the default user account for the image.

Install the `cloud-init` package:

```
# apt install cloud-init
```

When building Ubuntu images **cloud-init** must be explicitly configured for the metadata source in use. The OpenStack metadata server emulates the EC2 metadata service used by images in Amazon EC2.

To set the metadata source to be used by the image run the **dpkg-reconfigure** command against the `cloud-init` package. When prompted select the *EC2* data source:

```
# dpkg-reconfigure cloud-init
```

The account varies by distribution. On Ubuntu-based virtual machines, the account is called `ubuntu`. On Fedora-based virtual machines, the account is called `ec2-user`.

You can change the name of the account used by `cloud-init` by editing the `/etc/cloud/cloud.cfg` file and adding a line with a different user. For example, to configure `cloud-init` to put the key in an account named `admin`, use the following syntax in the configuration file:

```
users:
  - name: admin
    (...)
```

Shut down the instance

From inside the instance, as root:

```
# /sbin/shutdown -h now
```

Clean up (remove MAC address details)

The operating system records the MAC address of the virtual Ethernet card in locations such as `/etc/udev/rules.d/70-persistent-net.rules` during the installation process. However, each time the image boots up, the virtual Ethernet card will have a different MAC address, so this information must be deleted from the configuration file.

There is a utility called **`virt-sysprep`**, that performs various cleanup tasks such as removing the MAC address references. It will clean up a virtual machine image in place:

```
# virt-sysprep -d bionic
```

Undefine the libvirt domain

Now that the image is ready to be uploaded to the Image service, you no longer need to have this virtual machine image managed by libvirt. Use the **`virsh undefine vm-image`** command to inform libvirt:

```
# virsh undefine bionic
```

Image is complete

The underlying image file that you created with the **`qemu-img create`** command, such as `/var/lib/libvirt/images/bionic.qcow2`, is now ready for uploading to the Image service by using the **`openstack image create`** command. For more information, see the Glance User Guide.

2.7 Tools to automate image creation

There are several tools that are designed to automate image creation.

Contents

- *Tools to automate image creation*
  - *OpenStack tools*
    - *Diskimage-builder*
  - *External tools*
    - *image-bootstrap*
    - *KIWI*
    - *openstack-debian-images*
    - *Oz*
    - *Packer*
    - *virt-builder*
    - *windows-openstack-imaging-tools*

2.7.1 OpenStack tools

Diskimage-builder

Diskimage-builder is an automated disk image creation tool that supports a variety of distributions and architectures. Diskimage-builder (DIB) can build images for Fedora, Red Hat Enterprise Linux, Ubuntu, Debian, CentOS, and openSUSE. DIB is organized in a series of elements that build on top of each other to create specific images.

To build an image, call the following script:

```
# disk-image-create ubuntu vm
```

This example creates a generic, bootable Ubuntu image of the latest release.

Further customization could be accomplished by setting environment variables or adding elements to the command-line:

```
# disk-image-create -a armhf ubuntu vm
```

This example creates the image as before, but for arm architecture. More elements are available in the git source directory and documented in the diskimage-builder elements documentation.

2.7.2 External tools

image-bootstrap

image-bootstrap is a command line tool that generates bootable virtual machine images with support for Arch, Debian, Gentoo and Ubuntu, and is prepared for use with OpenStack.

KIWI

The KIWI appliance builder provides an operating system image builder for various Linux supported hardware platforms as well as for virtualization and cloud systems. It allows building of images based on openSUSE, SUSE Linux Enterprise, and Red Hat Enterprise Linux. The KIWI NG Documentation explains how to use it.

openstack-debian-images

openstack-debian-images is the tool Debian uses to create its official OpenStack image. It is made of a single very simple shell script that is easy to understand and modify. It supports Grub and Syslinux, BIOS or EFI, amd64 and arm64 arch.

openstack-debian-images can also be used to create a bootable image directly on a hard disk, instead of using the Debian installer.

To build an image, type this:

```
# build-openstack-debian-image --release stretch
```

More parameters can be added to further customize the image:

```
# build-openstack-debian-image --release stretch \
  --hook-script /root/my-hook-script.sh \
  --debootstrap-url http://ftp.fr.debian.org/debian \
  --sources.list-mirror http://ftp.fr.debian.org/debian \
  --login myusername \
  --extra-packages vim,emacs
```

The file `/root/my-hook-script.sh` will recieve 2 environment variable: `BODI_CHROOT_PATH` path where the image is mounted, and `BODI_RELEASE` which is the name of the Debian release that is being bootstraped. Heres an example for customizing the motd:

```
# #!/bin/sh
  set -e
  echo "My message" >${BODI_CHROOT_PATH}/etc/motd
```

This hook script will conveniently be called at the correct moment of the build process, when everything is installed, but before unmounting the partition.

Oz

Oz is a command-line tool that automates the process of creating a virtual machine image file. Oz is a Python app that interacts with KVM to step through the process of installing a virtual machine.

It uses a predefined set of kickstart (Red Hat-based systems) and preseed files (Debian-based systems) for operating systems that it supports, and it can also be used to create Microsoft Windows images.

A full treatment of Oz is beyond the scope of this document, but we will provide an example. You can find additional examples of Oz template files on GitHub at rcbops/oz-image-build/tree/master/templates. Heres how you would create a CentOS 6.4 image with Oz.

Create a template file called `centos64.tdl` with the following contents. The only entry you will need to change is the <`rootpw`> contents.

```
<template>
  <name>centos64</name>
  <os>
    <name>CentOS-6</name>
    <version>4</version>
    <arch>x86_64</arch>
    <install type='iso'>
      <iso>http://mirror.rackspace.com/CentOS/6/isos/x86_64/CentOS-6.4-x86_64-
↪bin-DVD1.iso</iso>
    </install>
    <rootpw>CHANGE THIS TO YOUR ROOT PASSWORD</rootpw>
  </os>
  <description>CentOS 6.4 x86_64</description>
  <repositories>
    <repository name='epel-6'>
      <url>http://download.fedoraproject.org/pub/epel/6/$basearch</url>
      <signed>no</signed>
    </repository>
  </repositories>
  <packages>
    <package name='epel-release'/>
    <package name='cloud-utils'/>
    <package name='cloud-init'/>
  </packages>
  <commands>
    <command name='update'>
yum -y update
yum clean all
sed -i '/^HWADDR/d' /etc/sysconfig/network-scripts/ifcfg-eth0
echo -n > /etc/udev/rules.d/70-persistent-net.rules
echo -n > /lib/udev/rules.d/75-persistent-net-generator.rules
    </command>
  </commands>
</template>
```

This Oz template specifies where to download the Centos 6.4 install ISO. Oz will use the version information to identify which kickstart file to use. In this case, it will be RHEL6.auto. It adds EPEL as a repository and install the `epel-release`, `cloud-utils`, and `cloud-init` packages, as specified in the `packages` section of the file.

After Oz completes the initial OS install using the kickstart file, it customizes the image with an update. It also removes any reference to the eth0 device that libvirt creates while Oz does the customizing, as specified in the `command` section of the XML file.

To run this:

```
# oz-install -d3 -u centos64.tdl -x centos64-libvirt.xml
```

- The `-d3` flag tells Oz to show status information as it runs.
- The `-u` tells Oz to do the customization (install extra packages, run the commands) once it does the initial install.

- The `-x` flag tells Oz what filename to use to write out a libvirt XML file (otherwise it will default to something like `centos64Apr_03_2013-12:39:42`).

If you leave out the `-u` flag, or you want to edit the file to do additional customizations, you can use the **`oz-customize`** command, using the libvirt XML file that **`oz-install`** creates. For example:

```
# oz-customize -d3 centos64.tdl centos64-libvirt.xml
```

Oz will invoke libvirt to boot the image inside of KVM, then Oz will ssh into the instance and perform the customizations.

Packer

Packer is a tool for creating machine images for multiple platforms from a single source configuration.

virt-builder

Virt-builder is a tool for quickly building new virtual machines. You can build a variety of VMs for local or cloud use, usually within a few minutes or less. Virt-builder also has many ways to customize these VMs. Everything is run from the command line and nothing requires root privileges, so automation and scripting is simple.

To build an image, call the following script:

```
# virt-builder fedora-23 -o image.qcow2 --format qcow2 \
  --update --selinux-relabel --size 20G
```

To list the operating systems available to install:

```
$ virt-builder --list
```

To import it into libvirt with **`virsh`**:

```
# virt-install --name fedora --ram 2048 \
  --disk path=image.qcow2,format=qcow2 --import
```

windows-openstack-imaging-tools

windows-openstack-imaging-tools is a PowerShell module that automates the Windows image creation for OpenStack and supports building VHDX, QCOW2, RAW and VMDK image types.

For easier installation, the tool is published as a PowerShellGallery module WindowsImageBuilder.

Windows image build example:

```
# Install and import WindowsImageBuilder module from PowerShellGallery
Install-Module WindowsImageBuilder -Force
Import-Module WindowsImageBuilder

New-WindowsImageConfig -ConfigFilePath ".\windows-image-config.ini"
# Update the configuration file to fit your specific use case
# Extensive information for all the configuration can be found here:
# https://github.com/cloudbase/windows-openstack-imaging-tools/blob/master/
↪Config.psm1#L21
```

(continues on next page)

(continued from previous page)

```
# Generate the Windows image
New-WindowsOnlineImage -ConfigFilePath ".\windows-image-config.ini"
```

Requirements:

- A Windows host, with Hyper-V virtualization enabled, PowerShell >=v4 support and Windows Assessment and Deployment Kit (ADK).
- A Windows installation ISO or DVD.
- Windows compatible drivers, if required by the target environment. For example, VirtIO, network card, or storage adapter drivers.
- Git environment.

2.8 Converting between image formats

Converting images from one format to another is generally straightforward.

2.8.1 qemu-img convert: raw, qcow2, qed, vdi, vmdk, vhd

The **`qemu-img convert`** command can do conversion between multiple formats, including `qcow2`, `qed`, `raw`, `vdi`, `vhd`, and `vmdk`.

Table 2: qemu-img format strings

Image format	Argument to qemu-img
QCOW2 (KVM, Xen)	`qcow2`
QED (KVM)	`qed`
raw	`raw`
VDI (VirtualBox)	`vdi`
VHD (Hyper-V)	`vpc`
VMDK (VMware)	`vmdk`

This example will convert a raw image file named `image.img` to a qcow2 image file.

```
$ qemu-img convert -f raw -O qcow2 image.img image.qcow2
```

Run the following command to convert a vmdk image file to a raw image file.

```
$ qemu-img convert -f vmdk -O raw image.vmdk image.img
```

Run the following command to convert a vmdk image file to a qcow2 image file.

```
$ qemu-img convert -f vmdk -O qcow2 image.vmdk image.qcow2
```

Note

The `-f format` flag is optional. If omitted, `qemu-img` will try to infer the image format.

When converting an image file with Windows, ensure the virtio driver is installed. Otherwise, you will get a blue screen when launching the image due to lack of the virtio driver. Another option is to set the image properties as below when you update the image in the Image service to avoid this issue, but it will reduce virtual machine performance significantly.

```
$ openstack image set --property hw_disk_bus='ide' image_name_or_id
```

2.8.2 VBoxManage: VDI (VirtualBox) to raw

If youve created a VDI image using VirtualBox, you can convert it to raw format using the `VBoxManage` command-line tool that ships with VirtualBox. On Mac OS X, and Linux, VirtualBox stores images by default in the `~/VirtualBox VMs/` directory. The following example creates a raw image in the current directory from a VirtualBox VDI image.

```
$ VBoxManage clonehd ~/VirtualBox\ VMs/image.vdi image.img --format raw
```

2.9 Image sharing

Image producers and consumers are both OpenStack users, or projects. Image producers create and share images with image consumers, allowing the consumers to use the shared image when booting a server. The producer shares an image with the consumer by making the consumer a member of that image. The consumer then accepts or rejects the image by changing the image member status. After it is accepted, the image appears in the consumers image list. As long as the consumer is a member of the image, the consumer can use the image, regardless of the image member status, if the consumer knows the image ID.

Note

In the OpenStack Image API, the image member status serves three purposes:

- The member status controls whether image appears in the consumers image list. If the image member status is `accepted`, the image appears in the consumers image list. Otherwise, the image does not appear in the image list. The image may still be used as long as the consumer knows the image ID.
- The member status can be used to filter the consumers image list.
- The member status lets the producer know whether the consumer has seen and acted on the shared image. If the status is `accepted` or `rejected`, the consumer has definitely seen the shared image. If the status is `pending`, the consumer may not be aware that an image was shared.

Image producers and consumers have different abilities and responsibilities regarding image sharing, which the following list shows.

- Image producers add members to images, or remove members from images, but they may not modify the member status of an image member.
- Image producers and consumers view the status of image members. When listing image members, the producers see all the image members, and the consumers see only themselves.
- Image consumers change their own member status, but they may not add or remove themselves as an image member.

- Image consumers can boot from any image shared by the image producer, regardless of the member status, as long as the consumer knows the image ID.

2.9.1 Sharing an image

The following procedure is a workflow for image sharing after image creation.

Communications between the image producer and the consumer, such as those described in this example, must be arranged independently of the OpenStack Image API. The consumer and producer can send notifications by using email, phone, Twitter, or other channels.

1. The producer posts the availability of specific images for consumers to review.
2. A potential consumer provides the producer with the consumers project ID. Optionally, the producer might request the consumers email address for notification purposes, but this is outside the scope of the API.
3. The producer shares the image with the consumer, by using the `Create image member` API operation.
4. Optionally, the producer notifies the consumer that the image has been shared and provides the images ID (UUID).
5. If the consumer wants the image to appear in the image list, the consumer uses the OpenStack Image API to change the image member status to `accepted`, by using the `Update image member` API operation.
6. If the consumer subsequently wants to hide the image, the consumer uses the OpenStack Image API to change the image member status to `rejected`. If the consumer wants to hide the image, but is open to the possibility of being reminded by the producer that the image is available, the consumer uses the OpenStack Image API to change the image member status back to `pending`, by using the `Update image member` API operation.

2.10 Appendix

2.10.1 Community support

The following resources are available to help you run and use OpenStack. The OpenStack community constantly improves and adds to the main features of OpenStack, but if you have any questions, do not hesitate to ask. Use the following resources to get OpenStack support and troubleshoot your installations.

Documentation

For the available OpenStack documentation, see docs.openstack.org.

The following guides explain how to install a Proof-of-Concept OpenStack cloud and its associated components:

- 2025.2 Flamingo Installation Guides

The following books explain how to configure and run an OpenStack cloud:

- Architecture Design Guide
- Administrator Guides
- Configuration Guides
- Networking Guide

- High Availability Guide
- Security Guide
- Virtual Machine Image Guide

The following book explains how to use the command-line clients:

- API Bindings

The following documentation provides reference and guidance information for the OpenStack APIs:

- API Documentation

The following guide provides information on how to contribute to OpenStack documentation:

- Documentation Contributor Guide

The OpenStack wiki

The OpenStack wiki contains a broad range of topics but some of the information can be difficult to find or is a few pages deep. Fortunately, the wiki search feature enables you to search by title or content. If you search for specific information, such as about networking or OpenStack Compute, you can find a large amount of relevant material. More is being added all the time, so be sure to check back often. You can find the search box in the upper-right corner of any OpenStack wiki page.

The Launchpad bugs area

The OpenStack community values your set up and testing efforts and wants your feedback. To log a bug, you must sign up for a Launchpad account. You can view existing bugs and report bugs in the Launchpad Bugs area. Use the search feature to determine whether the bug has already been reported or already been fixed. If it still seems like your bug is unreported, fill out a bug report.

Some tips:

- Give a clear, concise summary.
- Provide as much detail as possible in the description. Paste in your command output or stack traces, links to screen shots, and any other information which might be useful.
- Be sure to include the software and package versions that you are using, especially if you are using a development branch, such as, `"Kilo release" vs git commit bc79c3ecc55929bac585d04a03475b72e06a3208`.
- Any deployment-specific information is helpful, such as whether you are using CentOS Stream 10 or are performing a multi-node installation.

The following Launchpad Bugs areas are available:

- Bugs: OpenStack Block Storage (cinder)
- Bugs: OpenStack Compute (nova)
- Bugs: OpenStack Dashboard (horizon)
- Bugs: OpenStack Identity (keystone)
- Bugs: OpenStack Image service (glance)
- Bugs: OpenStack Networking (neutron)
- Bugs: OpenStack Object Storage (swift)

- Bugs: OpenStack Placement (placement)
- Bugs: Bare metal service (ironic)
- Bugs: Container Infrastructure Management service (magnum)
- Bugs: Database service (trove)
- Bugs: DNS service (designate)
- Bugs: Key Manager Service (barbican)
- Bugs: Orchestration (heat)
- Bugs: Rating (cloudkitty)
- Bugs: Shared file systems (manila)
- Bugs: Telemetry (ceilometer)
- Bugs: Alarming (aodh)
- Bugs: Workflow service (mistral)
- Bugs: Messaging service (zaqar)
- Bugs: Container service (zun)
- Bugs: Backup and Restore Service platform (freezer)
- Bugs: OpenStack Modern Dashboard (skyline-console)
- Bugs: OpenStack Skyline APIServer (skyline-apiserver)
- Bugs: OpenStack API Documentation (developer.openstack.org)
- Bugs: OpenStack Documentation (docs.openstack.org)

Documentation feedback

To provide feedback on documentation, join our IRC channel `#openstack-doc` on the OFTC IRC network, or report a bug in Launchpad and choose the particular project that the documentation is a part of.

The OpenStack IRC channel

The OpenStack community lives in the #openstack IRC channel on the OFTC network. You can hang out, ask questions, or get immediate feedback for urgent and pressing issues. To install an IRC client or use a browser-based client, go to https://webchat.oftc.net/. You can also use Colloquy (Mac OS X), mIRC (Windows), or XChat (Linux). When you are in the IRC channel and want to share code or command output, the generally accepted method is to use a Paste Bin. The OpenStack project has one at Paste. Just paste your longer amounts of text or logs in the web form and you get a URL that you can paste into the channel. The OpenStack IRC channel is `#openstack` on `irc.oftc.net`. You can find a list of all OpenStack IRC channels on the IRC page on the wiki.

OpenStack mailing lists

A great way to get answers and insights is to post your question or problematic scenario to the OpenStack mailing list. You can learn from and help others who might have similar issues. To subscribe or view the archives, go to the general OpenStack mailing list. If you are interested in the other mailing lists for specific projects or development, refer to Mailing Lists.

OpenStack distribution packages

The following Linux distributions provide community-supported packages for OpenStack:

- **CentOS Stream and Red Hat Enterprise Linux:** https://www.rdoproject.org/
- **Ubuntu:** https://wiki.ubuntu.com/OpenStack/CloudArchive

2.10.2 Glossary

This glossary offers a list of terms and definitions to define a vocabulary for OpenStack-related concepts.

To add to OpenStack glossary, clone the openstack/openstack-manuals repository and update the source file `doc/common/glossary.rst` through the OpenStack contribution process.

0-9

2023.1 Antelope
The code name for the twenty seventh release of OpenStack. This release is the first release based on the new release identification process which is formed after *year.release count within the year* and the name Antelope, a swift and gracious animal, also a type of steam locomotive.

2023.2 Bobcat
The code name for the twenty eighth release of OpenStack.

2024.1 Caracal
The code name for the twenty ninth release of OpenStack.

2024.2 Dalmatian
The code name for the thirtieth release of OpenStack.

2025.1 Epoxy
The code name for the thirty first release of OpenStack.

2025.2 Flamingo
The code name for the thirty second release of OpenStack.

2026.1 Gazpacho
The code name for the thirty third release of OpenStack.

2026.2 Hibiscus
The code name for the thirty fourth release of OpenStack.

6to4
A mechanism that allows IPv6 packets to be transmitted over an IPv4 network, providing a strategy for migrating to IPv6.

A

absolute limit
Impassable limits for guest VMs. Settings include total RAM size, maximum number of vCPUs, and maximum disk size.

access control list (ACL)
A list of permissions attached to an object. An ACL specifies which users or system processes have access to objects. It also defines which operations can be performed on specified objects. Each entry in a typical ACL specifies a subject and an operation. For instance, the ACL entry `(Alice, delete)` for a file gives Alice permission to delete the file.

access key
Alternative term for an Amazon EC2 access key. See *EC2 access key*.

account
The Object Storage context of an account. Do not confuse with a user account from an authentication service, such as Active Directory, /etc/passwd, OpenLDAP, OpenStack Identity, and so on.

account auditor
Checks for missing replicas and incorrect or corrupted objects in a specified Object Storage account by running queries against the back-end SQLite database.

account database
A SQLite database that contains Object Storage accounts and related metadata and that the accounts server accesses.

account reaper
An Object Storage worker that scans for and deletes account databases and that the account server has marked for deletion.

account server
Lists containers in Object Storage and stores container information in the account database.

account service
An Object Storage component that provides account services such as list, create, modify, and audit. Do not confuse with OpenStack Identity service, OpenLDAP, or similar user-account services.

accounting
The Compute service provides accounting information through the event notification and system usage data facilities.

Active Directory
Authentication and identity service by Microsoft, based on LDAP. Supported in OpenStack.

active/active configuration
In a high-availability setup with an active/active configuration, several systems share the load together and if one fails, the load is distributed to the remaining systems.

active/passive configuration
In a high-availability setup with an active/passive configuration, systems are set up to bring additional resources online to replace those that have failed.

address pool
A group of fixed and/or floating IP addresses that are assigned to a project and can be used by or assigned to the VM instances in a project.

Address Resolution Protocol (ARP)
The protocol by which layer-3 IP addresses are resolved into layer-2 link local addresses.

admin (Role)
A role that grants highest privileges in an OpenStack deployment. In many cases, admin rights can imply broader system access, depending on deployment policies.

admin API
A subset of API calls that are accessible to authorized administrators and are generally not accessible to end users or the public Internet. They can exist as a separate service (keystone) or can be a subset of another API (nova).

admin server
In the context of the Identity service, the worker process that provides access to the admin API.

administrator
The person responsible for installing, configuring, and managing an OpenStack cloud.

Advanced Message Queuing Protocol (AMQP)
The open standard messaging protocol used by OpenStack components for intra-service communications, provided by RabbitMQ, or Qpid.

Advanced RISC Machine (ARM)
Lower power consumption CPU often found in mobile and embedded devices. Supported by OpenStack.

alert
The Compute service can send alerts through its notification system, which includes a facility to create custom notification drivers. Alerts can be sent to and displayed on the dashboard.

allocate
The process of taking a floating IP address from the address pool so it can be associated with a fixed IP on a guest VM instance.

AlmaLinux
A free, open-source RHEL-compatible Linux distribution.

Amazon Kernel Image (AKI)
Both a VM container format and disk format. Supported by Image service.

Amazon Machine Image (AMI)
Both a VM container format and disk format. Supported by Image service.

Amazon Ramdisk Image (ARI)
Both a VM container format and disk format. Supported by Image service.

Ansible
Ansible is an open-source automation tool that can be used to deploy, configure, and manage OpenStack environments using simple, repeatable playbooks.

aodh
Part of the OpenStack *Telemetry service*; provides alarming functionality.

Apache
The Apache Software Foundation supports the Apache community of open-source software projects. These projects provide software products for the public good.

Apache License 2.0
All OpenStack core projects are provided under the terms of the Apache License 2.0 license.

Apache Web Server
The most common web server software currently used on the Internet.

API endpoint
The daemon, worker, or service that a client communicates with to access an API. API endpoints can provide any number of services, such as authentication, sales data, performance meters, Compute VM commands, census data, and so on.

API extension
Custom modules that extend some OpenStack core APIs.

API extension plug-in
Alternative term for a Networking plug-in or Networking API extension.

API key
Alternative term for an API token.

API server
Any node running a daemon or worker that provides an API endpoint.

API token
Passed to API requests and used by OpenStack to verify that the client is authorized to run the requested operation.

API version
In OpenStack, the API version for a project is part of the URL. For example, `example.com/nova/v1/foobar`.

applet
A Java program that can be embedded into a web page.

Application Programming Interface (API)
A collection of specifications used to access a service, application, or program. Includes service calls, required parameters for each call, and the expected return values.

application server
A piece of software that makes available another piece of software over a network.

Application Service Provider (ASP)
Companies that rent specialized applications that help businesses and organizations provide additional services with lower cost.

arptables
Tool used for maintaining Address Resolution Protocol packet filter rules in the Linux kernel firewall modules. Used along with iptables, ebtables, and ip6tables in Compute to provide firewall services for VMs.

associate
The process associating a Compute floating IP address with a fixed IP address.

Asynchronous JavaScript and XML (AJAX)
A group of interrelated web development techniques used on the client-side to create asynchronous web applications. Used extensively in horizon.

ATA over Ethernet (AoE)
A disk storage protocol tunneled within Ethernet.

attach
The process of connecting a VIF or vNIC to a L2 network in Networking. In the context of Compute, this process connects a storage volume to an instance.

attachment (network)
Association of an interface ID to a logical port. Plugs an interface into a port.

auditing
Provided in Compute through the system usage data facility.

auditor
A worker process that verifies the integrity of Object Storage objects, containers, and accounts.

Auditors is the collective term for the Object Storage account auditor, container auditor, and object auditor.

Austin
The code name for the initial release of OpenStack. The first design summit took place in Austin, Texas, US.

auth node
Alternative term for an Object Storage authorization node.

authentication
The process that confirms that the user, process, or client is really who they say they are through private key, secret token, password, fingerprint, or similar method.

authentication token
A string of text provided to the client after authentication. Must be provided by the user or process in subsequent requests to the API endpoint.

AuthN
The Identity service component that provides authentication services.

authorization
The act of verifying that a user, process, or client is authorized to perform an action.

authorization node
An Object Storage node that provides authorization services.

AuthZ
The Identity component that provides high-level authorization services.

Auto ACK
Configuration setting within RabbitMQ that enables or disables message acknowledgment. Enabled by default.

auto declare
A Compute RabbitMQ setting that determines whether a message exchange is automatically created when the program starts.

availability zone
An Availability Zone (AZ) is a logical subdivision of cloud storage, compute and network services. It provides a way for cloud operators to logically segment their compute based on arbitrary factors like location (country, data center, rack), network layout and/or power source.

AWS CloudFormation template
AWS CloudFormation allows Amazon Web Services (AWS) users to create and manage a collection of related resources. The Orchestration service supports a CloudFormation-compatible format (CFN).

B

back end
Interactions and processes that are obfuscated from the user, such as Compute volume mount, data transmission to an iSCSI target by a daemon, or Object Storage object integrity checks.

back-end catalog
The storage method used by the Identity service catalog service to store and retrieve information about API endpoints that are available to the client. Examples include an SQL database, LDAP database, or KVS back end.

back-end store
The persistent data store used to save and retrieve information for a service, such as lists of Object Storage objects, current state of guest VMs, lists of user names, and so on. Also, the method that the Image service uses to get and store VM images. Options include Object Storage, locally mounted file system, RADOS block devices, VMware datastore, and HTTP.

Backup and Restore service (freezer)
The project that provides integrated tooling for backing up and restoring instances or database backups.

bandwidth
The amount of available data used by communication resources, such as the Internet. Represents the amount of data that is used to download things or the amount of data available to download.

barbican
Code name of the *Key Manager service*.

bare
An Image service container format that indicates that no container exists for the VM image.

Bare Metal service (ironic)
The OpenStack service that provides a service and associated libraries capable of managing and provisioning physical machines in a security-aware and fault-tolerant manner.

base image
An OpenStack-provided image.

Bell-LaPadula model
A security model that focuses on data confidentiality and controlled access to classified information. This model divides the entities into subjects and objects. The clearance of a subject is compared to the classification of the object to determine if the subject is authorized for the specific access mode. The clearance or classification scheme is expressed in terms of a lattice.

Benchmark service (rally)
OpenStack project that provides a framework for performance analysis and benchmarking of individual OpenStack components as well as full production OpenStack cloud deployments.

Bexar
A grouped release of projects related to OpenStack that came out in February of 2011. It included only Compute (nova) and Object Storage (swift). Bexar is the code name for the second release of OpenStack. The design summit took place in San Antonio, Texas, US, which is the county seat for Bexar county.

binary
Information that consists solely of ones and zeroes, which is the language of computers.

bit
A bit is a single digit number that is in base of 2 (either a zero or one). Bandwidth usage is measured in bits per second.

bits per second (BPS)
The universal measurement of how quickly data is transferred from place to place.

block device
A device that moves data in the form of blocks. These device nodes interface the devices, such as hard disks, CD-ROM drives, flash drives, and other addressable regions of memory.

block migration
A method of VM live migration used by KVM to evacuate instances from one host to another with very little downtime during a user-initiated switchover. Does not require shared storage. Supported by Compute.

Block Storage API
An API on a separate endpoint for attaching, detaching, and creating block storage for compute VMs.

Block Storage service (cinder)
The OpenStack service that implements services and libraries to provide on-demand, self-service access to Block Storage resources via abstraction and automation on top of other block storage devices.

BMC (Baseboard Management Controller)
The intelligence in the IPMI architecture, which is a specialized micro-controller that is embedded on the motherboard of a computer and acts as a server. Manages the interface between system management software and platform hardware.

bootable disk image
A type of VM image that exists as a single, bootable file.

Bootstrap Protocol (BOOTP)
A network protocol used by a network client to obtain an IP address from a configuration server. Provided in Compute through the dnsmasq daemon when using either the FlatDHCP manager or VLAN manager network manager.

Border Gateway Protocol (BGP)
The Border Gateway Protocol is a dynamic routing protocol that connects autonomous systems. Considered the backbone of the Internet, this protocol connects disparate networks to form a larger network.

browser
Any client software that enables a computer or device to access the Internet.

builder file
Contains configuration information that Object Storage uses to reconfigure a ring or to re-create it from scratch after a serious failure.

bursting
The practice of utilizing a secondary environment to elastically build instances on-demand when the primary environment is resource constrained.

button class
A group of related button types within horizon. Buttons to start, stop, and suspend VMs are in one class. Buttons to associate and disassociate floating IP addresses are in another class, and so on.

byte
Set of bits that make up a single character; there are usually 8 bits to a byte.

C

cache pruner
A program that keeps the Image service VM image cache at or below its configured maximum size.

Cactus
An OpenStack grouped release of projects that came out in the spring of 2011. It included Compute

(nova), Object Storage (swift), and the Image service (glance). Cactus is a city in Texas, US and is the code name for the third release of OpenStack. When OpenStack releases went from three to six months long, the code name of the release changed to match a geography nearest the previous summit.

CALL
One of the RPC primitives used by the OpenStack message queue software. Sends a message and waits for a response.

capability
Defines resources for a cell, including CPU, storage, and networking. Can apply to the specific services within a cell or a whole cell.

capacity cache
A Compute back-end database table that contains the current workload, amount of free RAM, and number of VMs running on each host. Used to determine on which host a VM starts.

capacity updater
A notification driver that monitors VM instances and updates the capacity cache as needed.

CAST
One of the RPC primitives used by the OpenStack message queue software. Sends a message and does not wait for a response.

catalog
A list of API endpoints that are available to a user after authentication with the Identity service.

catalog service
An Identity service that lists API endpoints that are available to a user after authentication with the Identity service.

ceilometer
Part of the OpenStack *Telemetry service*; gathers and stores metrics from other OpenStack services.

cell
Provides logical partitioning of Compute resources in a child and parent relationship. Requests are passed from parent cells to child cells if the parent cannot provide the requested resource.

cell forwarding
A Compute option that enables parent cells to pass resource requests to child cells if the parent cannot provide the requested resource.

cell manager
The Compute component that contains a list of the current capabilities of each host within the cell and routes requests as appropriate.

CentOS
A Linux distribution that is compatible with OpenStack.

Ceph
Massively scalable distributed storage system that consists of an object store, block store, and POSIX-compatible distributed file system. Compatible with OpenStack.

CephFS
The POSIX-compliant file system provided by Ceph.

certificate authority (CA)
In cryptography, an entity that issues digital certificates. The digital certificate certifies the ownership of a public key by the named subject of the certificate. This enables others (relying parties) to

rely upon signatures or assertions made by the private key that corresponds to the certified public key. In this model of trust relationships, a CA is a trusted third party for both the subject (owner) of the certificate and the party relying upon the certificate. CAs are characteristic of many public key infrastructure (PKI) schemes. In OpenStack, a simple certificate authority is provided by Compute for cloudpipe VPNs and VM image decryption.

Challenge-Handshake Authentication Protocol (CHAP)
An iSCSI authentication method supported by Compute.

chance scheduler
A scheduling method used by Compute that randomly chooses an available host from the pool.

changes since
A Compute API parameter that allows downloading changes to the requested item since your last request, instead of downloading a new, fresh set of data and comparing it against the old data.

child cell
If a requested resource such as CPU time, disk storage, or memory is not available in the parent cell, the request is forwarded to its associated child cells. If the child cell can fulfill the request, it does. Otherwise, it attempts to pass the request to any of its children.

cinder
Codename for *Block Storage service*.

CirrOS
A minimal Linux distribution designed for use as a test image on clouds such as OpenStack.

Cisco neutron plug-in
A Networking plug-in for Cisco devices and technologies, including UCS and Nexus.

cloud architect
A person who plans, designs, and oversees the creation of clouds.

Cloud Auditing Data Federation (CADF)
Cloud Auditing Data Federation (CADF) is a specification for audit event data. CADF is supported by OpenStack Identity.

cloud computing
A model that enables access to a shared pool of configurable computing resources, such as networks, servers, storage, applications, and services, that can be rapidly provisioned and released with minimal management effort or service provider interaction.

cloud computing infrastructure
The hardware and software components such as servers, storage, and network and virtualization software that are needed to support the computing requirements of a cloud computing model.

cloud computing platform software
The delivery of different services through the Internet. These resources include tools and applications like data storage, servers, databases, networking, and software. As long as an electronic device has access to the web, it has access to the data and the software programs to run it.

cloud computing service architecture
Cloud service architecture defines the overall cloud computing services and solutions that are implemented in and across the boundaries of an enterprise business network. Considers the core business requirements and matches them with a possible cloud solution.

cloud controller
Collection of Compute components that represent the global state of the cloud; talks to services,

such as Identity authentication, Object Storage, and node/storage workers through a queue.

cloud controller node
A node that runs network, volume, API, scheduler, and image services. Each service may be broken out into separate nodes for scalability or availability.

Cloud Data Management Interface (CDMI)
SINA standard that defines a RESTful API for managing objects in the cloud, currently unsupported in OpenStack.

Cloud Infrastructure Management Interface (CIMI)
An in-progress specification for cloud management. Currently unsupported in OpenStack.

cloud technology
Clouds are tools of virtual sources orchestrated by management and automation softwares. This includes, raw processing power, memory, network, storage of cloud based applications.

cloud-init
A package commonly installed in VM images that performs initialization of an instance after boot using information that it retrieves from the metadata service, such as the SSH public key and user data.

Cloudbase-Init
A Windows project providing guest initialization features, similar to cloud-init.

command filter
Lists allowed commands within the Compute rootwrap facility.

Command-Line Interface (CLI)
A text-based client that helps you create scripts to interact with OpenStack clouds.

Common Internet File System (CIFS)
A file sharing protocol. It is a public or open variation of the original Server Message Block (SMB) protocol developed and used by Microsoft. Like the SMB protocol, CIFS runs at a higher level and uses the TCP/IP protocol.

Common Libraries (oslo)
The project that produces a set of python libraries containing code shared by OpenStack projects. The APIs provided by these libraries should be high quality, stable, consistent, documented and generally applicable.

community project
A project that is not officially endorsed by the OpenStack Technical Commitee. If the project is successful enough, it might be elevated to an incubated project and then to a core project, or it might be merged with the main code trunk.

compression
Reducing the size of files by special encoding, the file can be decompressed again to its original content. OpenStack supports compression at the Linux file system level but does not support compression for things such as Object Storage objects or Image service VM images.

Compute API (nova API)
The nova-api daemon provides access to nova services. Can communicate with other APIs, such as the Amazon EC2 API.

compute controller
The Compute component that chooses suitable hosts on which to start VM instances.

compute host
Physical host dedicated to running compute nodes.

compute node
A node that runs the nova-compute daemon that manages VM instances that provide a wide range of services, such as web applications and analytics.

Compute service (nova)
The OpenStack core project that implements services and associated libraries to provide massively-scalable, on-demand, self-service access to compute resources, including bare metal, virtual machines, and containers.

compute worker
The Compute component that runs on each compute node and manages the VM instance lifecycle, including run, reboot, terminate, attach/detach volumes, and so on. Provided by the nova-compute daemon.

concatenated object
A set of segment objects that Object Storage combines and sends to the client.

conductor
In Compute, conductor is the process that proxies database requests from the compute process. Using conductor improves security because compute nodes do not need direct access to the database.

consistency window
The amount of time it takes for a new Object Storage object to become accessible to all clients.

console log
Contains the output from a Linux VM console in Compute.

container
Organizes and stores objects in Object Storage. Similar to the concept of a Linux directory but cannot be nested. Alternative term for an Image service container format.

container auditor
Checks for missing replicas or incorrect objects in specified Object Storage containers through queries to the SQLite back-end database.

container database
A SQLite database that stores Object Storage containers and container metadata. The container server accesses this database.

container format
A wrapper used by the Image service that contains a VM image and its associated metadata, such as machine state, OS disk size, and so on.

Container Infrastructure Management service (magnum)
The project which provides a set of services for provisioning, scaling, and managing container orchestration engines.

container server
An Object Storage server that manages containers.

container service
The Object Storage component that provides container services, such as create, delete, list, and so on.

content delivery network (CDN)
A content delivery network is a specialized network that is used to distribute content to clients,

typically located close to the client for increased performance.

continuous delivery
A software engineering approach in which teams produce software in short cycles, ensuring that the software can be reliably released at any time and, when releasing the software, doing so manually.

continuous deployment
A software release process that uses automated testing to validate if changes to a codebase are correct and stable for immediate autonomous deployment to a production environment.

continuous integration
The practice of merging all developers working copies to a shared mainline several times a day.

controller node
Alternative term for a cloud controller node.

core API
Depending on context, the core API is either the OpenStack API or the main API of a specific core project, such as Compute, Networking, Image service, and so on.

cost
Under the Compute distributed scheduler, this is calculated by looking at the capabilities of each host relative to the flavor of the VM instance being requested.

credentials
Data that is only known to or accessible by a user and used to verify that the user is who he says he is. Credentials are presented to the server during authentication. Examples include a password, secret key, digital certificate, and fingerprint.

CRL
A Certificate Revocation List (CRL) in a PKI model is a list of certificates that have been revoked. End entities presenting these certificates should not be trusted.

Cross-Origin Resource Sharing (CORS)
A mechanism that allows many resources (for example, fonts, JavaScript) on a web page to be requested from another domain outside the domain from which the resource originated. In particular, JavaScripts AJAX calls can use the XMLHttpRequest mechanism.

Crowbar
An open source community project by SUSE that aims to provide all necessary services to quickly deploy and manage clouds.

current workload
An element of the Compute capacity cache that is calculated based on the number of build, snapshot, migrate, and resize operations currently in progress on a given host.

customer
Alternative term for project.

customization module
A user-created Python module that is loaded by horizon to change the look and feel of the dashboard.

D

daemon
A process that runs in the background and waits for requests. May or may not listen on a TCP or UDP port. Do not confuse with a worker.

Dashboard (horizon)
OpenStack project which provides an extensible, unified, web-based user interface for all OpenStack services.

data encryption
Both Image service and Compute support encrypted virtual machine (VM) images (but not instances). In-transit data encryption is supported in OpenStack using technologies such as HTTPS, SSL, TLS, and SSH. Object Storage does not support object encryption at the application level but may support storage that uses disk encryption.

Data loss prevention (DLP) software
Software programs used to protect sensitive information and prevent it from leaking outside a network boundary through the detection and denying of the data transportation.

data store
A database engine supported by the Database service.

database ID
A unique ID given to each replica of an Object Storage database.

database replicator
An Object Storage component that copies changes in the account, container, and object databases to other nodes.

Database service (trove)
An integrated project that provides scalable and reliable Cloud Database-as-a-Service functionality for both relational and non-relational database engines.

deallocate
The process of removing the association between a floating IP address and a fixed IP address. Once this association is removed, the floating IP returns to the address pool.

Debian
A Linux distribution that is compatible with OpenStack.

deduplication
The process of finding duplicate data at the disk block, file, and/or object level to minimize storage usecurrently unsupported within OpenStack.

default panel
The default panel that is displayed when a user accesses the dashboard.

default project
New users are assigned to this project if no project is specified when a user is created.

default token
An Identity service token that is not associated with a specific project and is exchanged for a scoped token.

delayed delete
An option within Image service so that an image is deleted after a predefined number of seconds instead of immediately.

delivery mode
Setting for the Compute RabbitMQ message delivery mode; can be set to either transient or persistent.

denial of service (DoS)
Denial of service (DoS) is a short form for denial-of-service attack. This is a malicious attempt to prevent legitimate users from using a service.

deprecated auth
An option within Compute that enables administrators to create and manage users through the `nova-manage` command as opposed to using the Identity service.

designate
Code name for the *DNS service*.

Desktop-as-a-Service
A platform that provides a suite of desktop environments that users access to receive a desktop experience from any location. This may provide general use, development, or even homogeneous testing environments.

device ID
Maps Object Storage partitions to physical storage devices.

device weight
Distributes partitions proportionately across Object Storage devices based on the storage capacity of each device.

DevStack
Community project that uses shell scripts to quickly build complete OpenStack development environments.

DHCP agent
OpenStack Networking agent that provides DHCP services for virtual networks.

Diablo
A grouped release of projects related to OpenStack that came out in the fall of 2011, the fourth release of OpenStack. It included Compute (nova 2011.3), Object Storage (swift 1.4.3), and the Image service (glance). Diablo is the code name for the fourth release of OpenStack. The design summit took place in the Bay Area near Santa Clara, California, US and Diablo is a nearby city.

direct consumer
An element of the Compute RabbitMQ that comes to life when a RPC call is executed. It connects to a direct exchange through a unique exclusive queue, sends the message, and terminates.

direct exchange
A routing table that is created within the Compute RabbitMQ during RPC calls; one is created for each RPC call that is invoked.

direct publisher
Element of RabbitMQ that provides a response to an incoming MQ message.

disassociate
The process of removing the association between a floating IP address and fixed IP and thus returning the floating IP address to the address pool.

Discretionary Access Control (DAC)
Governs the ability of subjects to access objects, while enabling users to make policy decisions and

assign security attributes. The traditional UNIX system of users, groups, and read-write-execute permissions is an example of DAC.

disk encryption
The ability to encrypt data at the file system, disk partition, or whole-disk level. Supported within Compute VMs.

disk format
The underlying format that a disk image for a VM is stored as within the Image service back-end store. For example, AMI, ISO, QCOW2, VMDK, and so on.

dispersion
In Object Storage, tools to test and ensure dispersion of objects and containers to ensure fault tolerance.

distributed virtual router (DVR)
Mechanism for highly available multi-host routing when using OpenStack Networking (neutron).

Django
A web framework used extensively in horizon.

DNS record
A record that specifies information about a particular domain and belongs to the domain.

DNS service (designate)
OpenStack project that provides scalable, on demand, self service access to authoritative DNS services, in a technology-agnostic manner.

dnsmasq
Daemon that provides DNS, DHCP, BOOTP, and TFTP services for virtual networks.

domain
An Identity API v3 entity. Represents a collection of projects, groups and users that defines administrative boundaries for managing OpenStack Identity entities. On the Internet, separates a website from other sites. Often, the domain name has two or more parts that are separated by dots. For example, yahoo.com, usa.gov, harvard.edu, or mail.yahoo.com. Also, a domain is an entity or container of all DNS-related information containing one or more records.

Domain Name System (DNS)
A system by which Internet domain name-to-address and address-to-name resolutions are determined. DNS helps navigate the Internet by translating the IP address into an address that is easier to remember. For example, translating 111.111.111.1 into www.yahoo.com. All domains and their components, such as mail servers, utilize DNS to resolve to the appropriate locations. DNS servers are usually set up in a master-slave relationship such that failure of the master invokes the slave. DNS servers might also be clustered or replicated such that changes made to one DNS server are automatically propagated to other active servers. In Compute, the support that enables associating DNS entries with floating IP addresses, nodes, or cells so that hostnames are consistent across reboots.

download
The transfer of data, usually in the form of files, from one computer to another.

durable exchange
The Compute RabbitMQ message exchange that remains active when the server restarts.

durable queue
A Compute RabbitMQ message queue that remains active when the server restarts.

Dynamic Host Configuration Protocol (DHCP)
A network protocol that configures devices that are connected to a network so that they can communicate on that network by using the Internet Protocol (IP). The protocol is implemented in a client-server model where DHCP clients request configuration data, such as an IP address, a default route, and one or more DNS server addresses from a DHCP server. A method to automatically configure networking for a host at boot time. Provided by both Networking and Compute.

Dynamic HyperText Markup Language (DHTML)
Pages that use HTML, JavaScript, and Cascading Style Sheets to enable users to interact with a web page or show simple animation.

E

east-west traffic
Network traffic between servers in the same cloud or data center. See also *north-south traffic*.

EBS boot volume
An Amazon EBS storage volume that contains a bootable VM image, currently unsupported in OpenStack.

ebtables
Filtering tool for a Linux bridging firewall, enabling filtering of network traffic passing through a Linux bridge. Used in Compute along with arptables, iptables, and ip6tables to ensure isolation of network communications.

EC2
The Amazon commercial compute product, similar to Compute.

EC2 access key
Used along with an EC2 secret key to access the Compute EC2 API.

EC2 API
OpenStack supports accessing the Amazon EC2 API through Compute.

EC2 Compatibility API
A Compute component that enables OpenStack to communicate with Amazon EC2.

EC2 secret key
Used along with an EC2 access key when communicating with the Compute EC2 API; used to digitally sign each request.

edge computing
Running fewer processes in the cloud and moving those processes to local places.

Elastic Block Storage (EBS)
The Amazon commercial block storage product.

encapsulation
The practice of placing one packet type within another for the purposes of abstracting or securing data. Examples include GRE, MPLS, or IPsec.

encryption
OpenStack supports encryption technologies such as HTTPS, SSH, SSL, TLS, digital certificates, and data encryption.

endpoint
See *API endpoint*.

endpoint registry
Alternative term for an Identity service catalog.

endpoint template
A list of URL and port number endpoints that indicate where a service, such as Object Storage, Compute, Identity, and so on, can be accessed.

enterprise cloud computing
A computing environment residing behind a firewall that delivers software, infrastructure and platform services to an enterprise.

entity
Any piece of hardware or software that wants to connect to the network services provided by Networking, the network connectivity service. An entity can make use of Networking by implementing a VIF.

ephemeral image
A VM image that does not save changes made to its volumes and reverts them to their original state after the instance is terminated.

ephemeral volume
Volume that does not save the changes made to it and reverts to its original state when the current user relinquishes control.

Essex
A grouped release of projects related to OpenStack that came out in April 2012, the fifth release of OpenStack. It included Compute (nova 2012.1), Object Storage (swift 1.4.8), Image (glance), Identity (keystone), and Dashboard (horizon). Essex is the code name for the fifth release of OpenStack. The design summit took place in Boston, Massachusetts, US and Essex is a nearby city.

ESXi
An OpenStack-supported hypervisor.

ETag
MD5 hash of an object within Object Storage, used to ensure data integrity.

euca2ools
A collection of command-line tools for administering VMs; most are compatible with OpenStack.

Eucalyptus Kernel Image (EKI)
Used along with an ERI to create an EMI.

Eucalyptus Machine Image (EMI)
VM image container format supported by Image service.

Eucalyptus Ramdisk Image (ERI)
Used along with an EKI to create an EMI.

evacuate
The process of migrating one or all virtual machine (VM) instances from one host to another, compatible with both shared storage live migration and block migration.

exchange
Alternative term for a RabbitMQ message exchange.

exchange type
A routing algorithm in the Compute RabbitMQ.

exclusive queue
Connected to by a direct consumer in RabbitMQCompute, the message can be consumed only by the current connection.

extended attributes (xattr)
File system option that enables storage of additional information beyond owner, group, permissions, modification time, and so on. The underlying Object Storage file system must support extended attributes.

extension
Alternative term for an API extension or plug-in. In the context of Identity service, this is a call that is specific to the implementation, such as adding support for OpenID.

external network
A network segment typically used for instance Internet access.

extra specs
Specifies additional requirements when Compute determines where to start a new instance. Examples include a minimum amount of network bandwidth or a GPU.

F

FakeLDAP
An easy method to create a local LDAP directory for testing Identity and Compute. Requires Redis.

fan-out exchange
Within RabbitMQ and Compute, it is the messaging interface that is used by the scheduler service to receive capability messages from the compute, volume, and network nodes.

federated identity
A method to establish trusts between identity providers and the OpenStack cloud.

Fedora
A Linux distribution compatible with OpenStack.

Fibre Channel
Storage protocol similar in concept to TCP/IP; encapsulates SCSI commands and data.

Fibre Channel over Ethernet (FCoE)
The fibre channel protocol tunneled within Ethernet.

fill-first scheduler
The Compute scheduling method that attempts to fill a host with VMs rather than starting new VMs on a variety of hosts.

filter
The step in the Compute scheduling process when hosts that cannot run VMs are eliminated and not chosen.

firewall
Used to restrict communications between hosts and/or nodes, implemented in Compute using iptables, arptables, ip6tables, and ebtables.

FireWall-as-a-Service (FWaaS)
A Networking extension that provides perimeter firewall functionality.

fixed IP address
An IP address that is associated with the same instance each time that instance boots, is generally not accessible to end users or the public Internet, and is used for management of the instance.

Flat Manager
The Compute component that gives IP addresses to authorized nodes and assumes DHCP, DNS, and routing configuration and services are provided by something else.

flat mode injection
A Compute networking method where the OS network configuration information is injected into the VM image before the instance starts.

flat network
Virtual network type that uses neither VLANs nor tunnels to segregate project traffic. Each flat network typically requires a separate underlying physical interface defined by bridge mappings. However, a flat network can contain multiple subnets.

FlatDHCP Manager
The Compute component that provides dnsmasq (DHCP, DNS, BOOTP, TFTP) and radvd (routing) services.

flavor
Alternative term for a VM instance type.

flavor ID
UUID for each Compute or Image service VM flavor or instance type.

floating IP address
An IP address that a project can associate with a VM so that the instance has the same public IP address each time that it boots. You create a pool of floating IP addresses and assign them to instances as they are launched to maintain a consistent IP address for maintaining DNS assignment.

Folsom
A grouped release of projects related to OpenStack that came out in the fall of 2012, the sixth release of OpenStack. It includes Compute (nova), Object Storage (swift), Identity (keystone), Networking (neutron), Image service (glance), and Volumes or Block Storage (cinder). Folsom is the code name for the sixth release of OpenStack. The design summit took place in San Francisco, California, US and Folsom is a nearby city.

FormPost
Object Storage middleware that uploads (posts) an image through a form on a web page.

freezer
Code name for the *Backup and Restore service*.

front end
The point where a user interacts with a service; can be an API endpoint, the dashboard, or a command-line tool.

G

gateway
An IP address, typically assigned to a router, that passes network traffic between different networks.

generic network virtualization encapsulation (Geneve)
A flexible network protocol that adapts to the changing needs and capabilities of devices in virtualized networks. It provides a tunneling framework without being prescriptive, supporting evolving network requirements. Geneve is predominantly used for OVN projects networks.

generic receive offload (GRO)
Feature of certain network interface drivers that combines many smaller received packets into a large packet before delivery to the kernel IP stack.

generic routing encapsulation (GRE)
Protocol that encapsulates a wide variety of network layer protocols inside virtual point-to-point links.

glance
Codename for the *Image service*.

glance API server
Alternative name for the *Image API*.

glance registry
Alternative term for the Image service *image registry*.

global endpoint template
The Identity service endpoint template that contains services available to all projects.

GlusterFS
A file system designed to aggregate NAS hosts, compatible with OpenStack.

gnocchi
Part of the OpenStack *Telemetry service*; provides an indexer and time-series database.

golden image
A method of operating system installation where a finalized disk image is created and then used by all nodes without modification.

Graphic Interchange Format (GIF)
A type of image file that is commonly used for animated images on web pages.

Graphics Processing Unit (GPU)
Choosing a host based on the existence of a GPU is currently unsupported in OpenStack.

Green Threads
The cooperative threading model used by Python; reduces race conditions and only context switches when specific library calls are made. Each OpenStack service is its own thread.

Grizzly
The code name for the seventh release of OpenStack. The design summit took place in San Diego, California, US and Grizzly is an element of the state flag of California.

Group
An Identity v3 API entity. Represents a collection of users that is owned by a specific domain.

guest OS
An operating system instance running under the control of a hypervisor.

H

Hadoop
Apache Hadoop is an open source software framework that supports data-intensive distributed applications.

Hadoop Distributed File System (HDFS)
A distributed, highly fault-tolerant file system designed to run on low-cost commodity hardware.

handover
An object state in Object Storage where a new replica of the object is automatically created due to a drive failure.

HAProxy
Provides a load balancer for TCP and HTTP-based applications that spreads requests across multiple servers.

hard reboot
A type of reboot where a physical or virtual power button is pressed as opposed to a graceful, proper shutdown of the operating system.

Havana
The code name for the eighth release of OpenStack. The design summit took place in Portland, Oregon, US and Havana is an unincorporated community in Oregon.

health monitor
Determines whether back-end members of a VIP pool can process a request. A pool can have several health monitors associated with it. When a pool has several monitors associated with it, all monitors check each member of the pool. All monitors must declare a member to be healthy for it to stay active.

heat
Codename for the *Orchestration service*.

Heat Orchestration Template (HOT)
Heat input in the format native to OpenStack.

high availability (HA)
A high availability system design approach and associated service implementation ensures that a prearranged level of operational performance will be met during a contractual measurement period. High availability systems seek to minimize system downtime and data loss.

horizon
Codename for the *Dashboard*.

horizon plug-in
A plug-in for the OpenStack Dashboard (horizon).

host
A physical computer, not a VM instance (node).

host aggregate
A method to further subdivide availability zones into hypervisor pools, a collection of common hosts.

Host Bus Adapter (HBA)
Device plugged into a PCI slot, such as a fibre channel or network card.

hybrid cloud
A hybrid cloud is a composition of two or more clouds (private, community or public) that remain distinct entities but are bound together, offering the benefits of multiple deployment models. Hybrid cloud can also mean the ability to connect colocation, managed and/or dedicated services with cloud resources.

hybrid cloud computing
A mix of on-premises, private cloud and third-party, public cloud services with orchestration between the two platforms.

hyperlink
Any kind of text that contains a link to some other site, commonly found in documents where clicking on a word or words opens up a different website.

Hypertext Transfer Protocol (HTTP)
An application protocol for distributed, collaborative, hypermedia information systems. It is the foundation of data communication for the World Wide Web. Hypertext is structured text that uses logical links (hyperlinks) between nodes containing text. HTTP is the protocol to exchange or transfer hypertext.

Hypertext Transfer Protocol Secure (HTTPS)
An encrypted communications protocol for secure communication over a computer network, with especially wide deployment on the Internet. Technically, it is not a protocol in and of itself; rather, it is the result of simply layering the Hypertext Transfer Protocol (HTTP) on top of the TLS or SSL protocol, thus adding the security capabilities of TLS or SSL to standard HTTP communications. Most OpenStack API endpoints and many inter-component communications support HTTPS communication.

hypervisor
Software that arbitrates and controls VM access to the actual underlying hardware.

hypervisor pool
A collection of hypervisors grouped together through host aggregates.

I

Icehouse
The code name for the ninth release of OpenStack. The design summit took place in Hong Kong and Ice House is a street in that city.

ID number
Unique numeric ID associated with each user in Identity, conceptually similar to a Linux or LDAP UID.

Identity API
Alternative term for the Identity service API.

Identity back end
The source used by Identity service to retrieve user information; an OpenLDAP server, for example.

identity provider
A directory service, which allows users to login with a user name and password. It is a typical source of authentication tokens.

Identity service (keystone)
The project that facilitates API client authentication, service discovery, distributed multi-project authorization, and auditing. It provides a central directory of users mapped to the OpenStack services they can access. It also registers endpoints for OpenStack services and acts as a common authentication system.

Identity service API
The API used to access the OpenStack Identity service provided through keystone.

IETF
Internet Engineering Task Force (IETF) is an open standards organization that develops Internet standards, particularly the standards pertaining to TCP/IP.

image
A collection of files for a specific operating system (OS) that you use to create or rebuild a server. OpenStack provides pre-built images. You can also create custom images, or snapshots, from

servers that you have launched. Custom images can be used for data backups or as gold images for additional servers.

Image API
The Image service API endpoint for management of VM images. Processes client requests for VMs, updates Image service metadata on the registry server, and communicates with the store adapter to upload VM images from the back-end store.

image cache
Used by Image service to obtain images on the local host rather than re-downloading them from the image server each time one is requested.

image ID
Combination of a URI and UUID used to access Image service VM images through the image API.

image membership
A list of projects that can access a given VM image within Image service.

image owner
The project who owns an Image service virtual machine image.

image registry
A list of VM images that are available through Image service.

Image service (glance)
The OpenStack service that provides services and associated libraries to store, browse, share, distribute and manage bootable disk images, other data closely associated with initializing compute resources, and metadata definitions.

image status
The current status of a VM image in Image service, not to be confused with the status of a running instance.

image store
The back-end store used by Image service to store VM images, options include Object Storage, locally mounted file system, RADOS block devices, VMware datastore, or HTTP.

image UUID
UUID used by Image service to uniquely identify each VM image.

incubated project
A community project may be elevated to this status and is then promoted to a core project.

Infrastructure Optimization service (watcher)
OpenStack project that aims to provide a flexible and scalable resource optimization service for multi-project OpenStack-based clouds.

Infrastructure-as-a-Service (IaaS)
IaaS is a provisioning model in which an organization outsources physical components of a data center, such as storage, hardware, servers, and networking components. A service provider owns the equipment and is responsible for housing, operating and maintaining it. The client typically pays on a per-use basis. IaaS is a model for providing cloud services.

ingress filtering
The process of filtering incoming network traffic. Supported by Compute.

INI format
The OpenStack configuration files use an INI format to describe options and their values. It consists of sections and key value pairs.

injection
The process of putting a file into a virtual machine image before the instance is started.

Input/Output Operations Per Second (IOPS)
IOPS are a common performance measurement used to benchmark computer storage devices like hard disk drives, solid state drives, and storage area networks.

instance
A running VM, or a VM in a known state such as suspended, that can be used like a hardware server.

instance ID
Alternative term for instance UUID.

instance state
The current state of a guest VM image.

instance tunnels network
A network segment used for instance traffic tunnels between compute nodes and the network node.

instance type
Describes the parameters of the various virtual machine images that are available to users; includes parameters such as CPU, storage, and memory. Alternative term for flavor.

instance type ID
Alternative term for a flavor ID.

instance UUID
Unique ID assigned to each guest VM instance.

Intelligent Platform Management Interface (IPMI)
IPMI is a standardized computer system interface used by system administrators for out-of-band management of computer systems and monitoring of their operation. In laymans terms, it is a way to manage a computer using a direct network connection, whether it is turned on or not; connecting to the hardware rather than an operating system or login shell.

interface
A physical or virtual device that provides connectivity to another device or medium.

interface ID
Unique ID for a Networking VIF or vNIC in the form of a UUID.

Internet Control Message Protocol (ICMP)
A network protocol used by network devices for control messages. For example, **`ping`** uses ICMP to test connectivity.

Internet protocol (IP)
Principal communications protocol in the internet protocol suite for relaying datagrams across network boundaries.

Internet Service Provider (ISP)
Any business that provides Internet access to individuals or businesses.

Internet Small Computer System Interface (iSCSI)
Storage protocol that encapsulates SCSI frames for transport over IP networks. Supported by Compute, Object Storage, and Image service.

IO
The abbreviation for input and output.

IP address
Number that is unique to every computer system on the Internet. Two versions of the Internet Protocol (IP) are in use for addresses: IPv4 and IPv6.

IP Address Management (IPAM)
The process of automating IP address allocation, deallocation, and management. Currently provided by Compute, melange, and Networking.

ip6tables
Tool used to set up, maintain, and inspect the tables of IPv6 packet filter rules in the Linux kernel. In OpenStack Compute, ip6tables is used along with arptables, ebtables, and iptables to create firewalls for both nodes and VMs.

ipset
Extension to iptables that allows creation of firewall rules that match entire sets of IP addresses simultaneously. These sets reside in indexed data structures to increase efficiency, particularly on systems with a large quantity of rules.

iptables
Used along with arptables and ebtables, iptables create firewalls in Compute. iptables are the tables provided by the Linux kernel firewall (implemented as different Netfilter modules) and the chains and rules it stores. Different kernel modules and programs are currently used for different protocols: iptables applies to IPv4, ip6tables to IPv6, arptables to ARP, and ebtables to Ethernet frames. Requires root privilege to manipulate.

ironic
Codename for the *Bare Metal service*.

iSCSI Qualified Name (IQN)
IQN is the format most commonly used for iSCSI names, which uniquely identify nodes in an iSCSI network. All IQNs follow the pattern iqn.yyyy-mm.domain:identifier, where yyyy-mm is the year and month in which the domain was registered, domain is the reversed domain name of the issuing organization, and identifier is an optional string which makes each IQN under the same domain unique. For example, iqn.2015-10.org.openstack.408ae959bce1.

ISO9660
One of the VM image disk formats supported by Image service.

J

Java
A programming language that is used to create systems that involve more than one computer by way of a network.

JavaScript
A scripting language that is used to build web pages.

JavaScript Object Notation (JSON)
One of the supported response formats in OpenStack.

jumbo frame
Feature in modern Ethernet networks that supports frames up to approximately 9000 bytes.

Juno
The code name for the tenth release of OpenStack. The design summit took place in Atlanta, Georgia, US and Juno is an unincorporated community in Georgia.

K

Kerberos
A network authentication protocol which works on the basis of tickets. Kerberos allows nodes communication over a non-secure network, and allows nodes to prove their identity to one another in a secure manner.

kernel-based VM (KVM)
An OpenStack-supported hypervisor. KVM is a full virtualization solution for Linux on x86 hardware containing virtualization extensions (Intel VT or AMD-V), ARM, IBM Power, and IBM zSeries. It consists of a loadable kernel module, that provides the core virtualization infrastructure and a processor specific module.

Key Manager service (barbican)
The project that produces a secret storage and generation system capable of providing key management for services wishing to enable encryption features.

keystone
Codename of the *Identity service*.

Kickstart
A tool to automate system configuration and installation on Red Hat, Fedora, and CentOS-based Linux distributions.

Kilo
The code name for the eleventh release of OpenStack. The design summit took place in Paris, France. Due to delays in the name selection, the release was known only as K. Because `k` is the unit symbol for kilo and the kilogram reference artifact is stored near Paris in the Pavillon de Breteuil in Sèvres, the community chose Kilo as the release name.

L

large object
An object within Object Storage that is larger than 5ăGB.

Launchpad
The collaboration site for OpenStack.

Layer-2 (L2) agent
OpenStack Networking agent that provides layer-2 connectivity for virtual networks.

Layer-2 network
Term used in the OSI network architecture for the data link layer. The data link layer is responsible for media access control, flow control and detecting and possibly correcting errors that may occur in the physical layer.

Layer-3 (L3) agent
OpenStack Networking agent that provides layer-3 (routing) services for virtual networks.

Layer-3 network
Term used in the OSI network architecture for the network layer. The network layer is responsible for packet forwarding including routing from one node to another.

Liberty
The code name for the twelfth release of OpenStack. The design summit took place in Vancouver, Canada and Liberty is the name of a village in the Canadian province of Saskatchewan.

libvirt
Virtualization API library used by OpenStack to interact with many of its supported hypervisors.

Lightweight Directory Access Protocol (LDAP)
An application protocol for accessing and maintaining distributed directory information services over an IP network.

Linux
Unix-like computer operating system assembled under the model of free and open-source software development and distribution.

Linux bridge
Software that enables multiple VMs to share a single physical NIC within Compute.

Linux Bridge neutron plug-in
Enables a Linux bridge to understand a Networking port, interface attachment, and other abstractions.

This driver was removed in the 2025.1 (Epoxy) release of OpenStack, it is recommended to migrate to the OVN driver.

Linux containers (LXC)
An OpenStack-supported hypervisor.

live migration
The ability within Compute to move running virtual machine instances from one host to another with only a small service interruption during switchover.

load balancer
A load balancer is a logical device that belongs to a cloud account. It is used to distribute workloads between multiple back-end systems or services, based on the criteria defined as part of its configuration.

load balancing
The process of spreading client requests between two or more nodes to improve performance and availability.

Load-Balancer-as-a-Service (LBaaS)
Enables Networking to distribute incoming requests evenly between designated instances.

Load-balancing service (octavia)
The project that aims to provide scalable, on demand, self service access to load-balancer services, in technology-agnostic manner.

Logical Volume Manager (LVM)
Provides a method of allocating space on mass-storage devices that is more flexible than conventional partitioning schemes.

M

magnum
Code name for the *Containers Infrastructure Management service*.

management API
Alternative term for an admin API.

management network
A network segment used for administration, not accessible to the public Internet.

manager
Logical groupings of related code, such as the Block Storage volume manager or network manager.

manager (Role)
A role positioned between the admin and member roles, providing limited identity management. It has a narrower scope and purpose, focused on managing identity-related resources. When assigned at the domain scope, the manager role allows users to manage users, projects, groups, and role assignments within an entire domain.

manifest
Used to track segments of a large object within Object Storage.

manifest object
A special Object Storage object that contains the manifest for a large object.

manila
Codename for OpenStack *Shared File Systems service*.

manila-share
Responsible for managing Shared File System Service devices, specifically the back-end devices.

maximum transmission unit (MTU)
Maximum frame or packet size for a particular network medium. Typically 1500 bytes for Ethernet networks.

mechanism driver
A driver for the Modular Layer 2 (ML2) neutron plug-in that provides layer-2 connectivity for virtual instances. A single OpenStack installation can use multiple mechanism drivers.

melange
Project name for OpenStack Network Information Service. To be merged with Networking.

member (Role)
A role that provides intermediate privileges between the admin and reader roles. The member role offers a way to introduce more granular access control and flexibility across different scopes.

membership
The association between an Image service VM image and a project. Enables images to be shared with specified projects.

membership list
A list of projects that can access a given VM image within Image service.

memcached
A distributed memory object caching system that is used by Object Storage for caching.

memory overcommit
The ability to start new VM instances based on the actual memory usage of a host, as opposed to basing the decision on the amount of RAM each running instance thinks it has available. Also known as RAM overcommit.

message broker
The software package used to provide AMQP messaging capabilities within Compute. Default package is RabbitMQ.

message bus
The main virtual communication line used by all AMQP messages for inter-cloud communications within Compute.

message queue
Passes requests from clients to the appropriate workers and returns the output to the client after the job completes.

Message service (zaqar)
The project that provides a messaging service that affords a variety of distributed application patterns in an efficient, scalable, and highly available manner, and to create and maintain associated Python libraries and documentation.

Meta-Data Server (MDS)
Stores CephFS metadata.

Metadata agent
OpenStack Networking agent that provides metadata services for instances.

migration
The process of moving a VM instance from one host to another.

mistral
Code name for *Workflow service*.

Mitaka
The code name for the thirteenth release of OpenStack. The design summit took place in Tokyo, Japan. Mitaka is a city in Tokyo.

Modular Layer 2 (ML2) neutron plug-in
Can concurrently use multiple layer-2 networking technologies, such as 802.1Q and VXLAN, in Networking.

monasca
Codename for OpenStack *Monitoring*.

Monitor (LBaaS)
LBaaS feature that provides availability monitoring using the `ping` command, TCP, and HTTP/HTTPS GET.

Monitor (Mon)
A Ceph component that communicates with external clients, checks data state and consistency, and performs quorum functions.

Monitoring (monasca)
The OpenStack service that provides a multi-project, highly scalable, performant, fault-tolerant monitoring-as-a-service solution for metrics, complex event processing and logging. To build an extensible platform for advanced monitoring services that can be used by both operators and projects to gain operational insight and visibility, ensuring availability and stability.

multi-cloud computing
The use of multiple cloud computing and storage services in a single network architecture.

multi-cloud SDKs
SDKs that provide a multi-cloud abstraction layer and include support for OpenStack. These SDKs are excellent for writing applications that need to consume more than one type of cloud provider, but may expose a more limited set of features.

multi-factor authentication
Authentication method that uses two or more credentials, such as a password and a private key. Currently not supported in Identity.

multi-host
High-availability mode for legacy (nova) networking. Each compute node handles NAT and DHCP and acts as a gateway for all of the VMs on it. A networking failure on one compute node doesnt affect VMs on other compute nodes.

multinic
Facility in Compute that allows each virtual machine instance to have more than one VIF connected to it.

N

Nebula
Released as open source by NASA in 2010 and is the basis for Compute.

NetApp volume driver
Enables Compute to communicate with NetApp storage devices through the NetApp OnCommand Provisioning Manager.

network
A virtual network that provides connectivity between entities. For example, a collection of virtual ports that share network connectivity. In Networking terminology, a network is always a layer-2 network.

Network Address Translation (NAT)
Process of modifying IP address information while in transit. Supported by Compute and Networking.

network controller
A Compute daemon that orchestrates the network configuration of nodes, including IP addresses, VLANs, and bridging. Also manages routing for both public and private networks.

Network File System (NFS)
A method for making file systems available over the network. Supported by OpenStack.

network ID
Unique ID assigned to each network segment within Networking. Same as network UUID.

network manager
The Compute component that manages various network components, such as firewall rules, IP address allocation, and so on.

network namespace
Linux kernel feature that provides independent virtual networking instances on a single host with separate routing tables and interfaces. Similar to virtual routing and forwarding (VRF) services on physical network equipment.

network node
Any compute node that runs the network worker daemon.

network segment
Represents a virtual, isolated OSI layer-2 subnet in Networking.

Network Service Header (NSH)
Provides a mechanism for metadata exchange along the instantiated service path.

Network Time Protocol (NTP)
Method of keeping a clock for a host or node correct via communication with a trusted, accurate time source.

network UUID
Unique ID for a Networking network segment.

network worker
The `nova-network` worker daemon; provides services such as giving an IP address to a booting nova instance.

Networking API (Neutron API)
API used to access OpenStack Networking. Provides an extensible architecture to enable custom plug-in creation.

Networking service (neutron)
The OpenStack project which implements services and associated libraries to provide on-demand, scalable, and technology-agnostic network abstraction.

neutron
Codename for OpenStack *Networking service*.

neutron API
An alternative name for *Networking API*.

neutron manager
Enables Compute and Networking integration, which enables Networking to perform network management for guest VMs.

neutron plug-in
Interface within Networking that enables organizations to create custom plug-ins for advanced features, such as QoS, ACLs, or IDS.

Newton
The code name for the fourteenth release of OpenStack. The design summit took place in Austin, Texas, US. The release is named after Newton House which is located at 1013 E. Ninth St., Austin, TX. which is listed on the National Register of Historic Places.

Nexenta volume driver
Provides support for NexentaStor devices in Compute.

NFV Orchestration service (tacker)
OpenStack service that aims to implement Network Function Virtualization (NFV) orchestration services and libraries for end-to-end life-cycle management of network services and Virtual Network Functions (VNFs).

Nginx
An HTTP and reverse proxy server, a mail proxy server, and a generic TCP/UDP proxy server.

No ACK
Disables server-side message acknowledgment in the Compute RabbitMQ. Increases performance but decreases reliability.

node
A VM instance that runs on a host.

non-durable exchange
Message exchange that is cleared when the service restarts. Its data is not written to persistent storage.

non-durable queue
Message queue that is cleared when the service restarts. Its data is not written to persistent storage.

non-persistent volume
Alternative term for an ephemeral volume.

north-south traffic
Network traffic between a user or client (north) and a server (south), or traffic into the cloud (south) and out of the cloud (north). See also east-west traffic.

nova
Codename for OpenStack *Compute service*.

Nova API
Alternative term for the *Compute API*.

O

object
A BLOB of data held by Object Storage; can be in any format.

object auditor
Opens all objects for an object server and verifies the MD5 hash, size, and metadata for each object.

object expiration
A configurable option within Object Storage to automatically delete objects after a specified amount of time has passed or a certain date is reached.

object hash
Unique ID for an Object Storage object.

object path hash
Used by Object Storage to determine the location of an object in the ring. Maps objects to partitions.

object replicator
An Object Storage component that copies an object to remote partitions for fault tolerance.

object server
An Object Storage component that is responsible for managing objects.

Object Storage API
API used to access OpenStack *Object Storage*.

Object Storage Device (OSD)
The Ceph storage daemon.

Object Storage service (swift)
The OpenStack core project that provides eventually consistent and redundant storage and retrieval of fixed digital content.

object versioning
Allows a user to set a flag on an *Object Storage* container so that all objects within the container are versioned.

Ocata
The code name for the fifteenth release of OpenStack. The design summit took place in Barcelona, Spain. Ocata is a beach north of Barcelona.

Octavia
Code name for the *Load-balancing service*.

Oldie
Term for an *Object Storage* process that runs for a long time. Can indicate a hung process.

Open Cloud Computing Interface (OCCI)
A standardized interface for managing compute, data, and network resources, currently unsupported in OpenStack.

Open Virtual Network (OVN)
OVN (Open Virtual Network) is a set of services that convert virtual network setups into OpenFlow rules, and apply them into Open vSwitch. OVN provides a more abstract layer than Open vSwitch, allowing working with logical routers and logical switches, instead of flows.

Open Virtualization Format (OVF)
Standard for packaging VM images. Supported in OpenStack.

Open vSwitch
Open vSwitch is a production quality, multilayer virtual switch licensed under the open source Apache 2.0 license. It is designed to enable massive network automation through programmatic extension, while still supporting standard management interfaces and protocols (for example NetFlow, sFlow, SPAN, RSPAN, CLI, LACP, 802.1ag).

Open vSwitch (OVS) agent
Provides an interface to the underlying Open vSwitch service for the Networking plug-in.

Open vSwitch neutron plug-in
Provides support for Open vSwitch in Networking.

OpenDev
OpenDev is a space for collaborative Open Source software development.

OpenDevs mission is to provide project hosting, continuous integration tooling, and virtual collaboration spaces for Open Source software projects. OpenDev is itself self hosted on this set of tools including code review, continuous integration, etherpad, wiki, code browsing and so on. This means that OpenDev itself is run like an open source project, you can join us and help run the system. Additionally, all of the services run are Open Source software themselves.

The OpenStack project is the largest project using OpenDev.

OpenLDAP
An open source LDAP server. Supported by both Compute and Identity.

OpenStack
OpenStack is a cloud operating system that controls large pools of compute, storage, and networking resources throughout a data center, all managed through a dashboard that gives administrators control while empowering their users to provision resources through a web interface. OpenStack is an open source project licensed under the Apache License 2.0.

OpenStack code name
Each OpenStack release has a code name. Code names ascend in alphabetical order: Austin, Bexar, Cactus, Diablo, Essex, Folsom, Grizzly, Havana, Icehouse, Juno, Kilo, Liberty, Mitaka, Newton, Ocata, Pike, Queens, Rocky, Stein, Train, Ussuri, Victoria, Wallaby, Xena, Yoga, Zed.

Wallaby was the first code name choosen by a new policy: Code names are choosen by the community following the alphabet, for details see release name criteria.

The Victoria name was the last name where code names are cities or counties near where the corresponding OpenStack design summit took place. An exception, called the Waldon exception,

was granted to elements of the state flag that sound especially cool. Code names are chosen by popular vote.

At the same time as OpenStack releases run out of alphabet the Technical Committee changed the naming process to have release number and a release name as an identification code. The release number will be the primary identifier: *year.release count within the year* and the name will be used mostly for marketing purposes. The first such release is 2023.1 Antelope. Followed by, respectively, 2023.2 Bobcat, 2024.1 Caracal, 2024.2 Dalmatian, 2025.1 Epoxy, 2025.2 Flamingo.

OpenStack-Ansible
OpenStack-Ansible (OSA) provides Ansible playbooks and roles for deploying and configuring an OpenStack environment, enabling scalable and automated cloud infrastructure management.

operator
The person responsible for planning and maintaining an OpenStack installation.

optional service
An official OpenStack service defined as optional by Interop Working Group. Currently, consists of Dashboard (horizon), Telemetry service (Telemetry), Orchestration service (heat), Database service (trove), Bare Metal service (ironic), and so on.

Orchestration service (heat)
The OpenStack service which orchestrates composite cloud applications using a declarative template format through an OpenStack-native REST API.

orphan
In the context of Object Storage, this is a process that is not terminated after an upgrade, restart, or reload of the service.

oslo
Codename for the *Common Libraries project*. Its a collection of tools and libraries that help OpenStack services share common features like logging, configuration, and messaging.

P

parent cell
If a requested resource, such as CPU time, disk storage, or memory, is not available in the parent cell, the request is forwarded to associated child cells.

partition
A unit of storage within Object Storage used to store objects. It exists on top of devices and is replicated for fault tolerance.

partition index
Contains the locations of all Object Storage partitions within the ring.

partition shift value
Used by Object Storage to determine which partition data should reside on.

path MTU discovery (PMTUD)
Mechanism in IP networks to detect end-to-end MTU and adjust packet size accordingly.

pause
A VM state where no changes occur (no changes in memory, network communications stop, etc); the VM is frozen but not shut down.

PCI passthrough
Gives guest VMs exclusive access to a PCI device. Currently supported in OpenStack Havana and

later releases.

persistent message
A message that is stored both in memory and on disk. The message is not lost after a failure or restart.

persistent volume
Changes to these types of disk volumes are saved.

personality file
A file used to customize a Compute instance. It can be used to inject SSH keys or a specific network configuration.

Pike
The code name for the sixteenth release of OpenStack. The OpenStack summit took place in Boston, Massachusetts, US. The release is named after the Massachusetts Turnpike, abbreviated commonly as the Mass Pike, which is the easternmost stretch of Interstate 90.

Platform-as-a-Service (PaaS)
Provides to the consumer an operating system and, often, a language runtime and libraries (collectively, the platform) upon which they can run their own application code, without providing any control over the underlying infrastructure. Examples of Platform-as-a-Service providers include Cloud Foundry and OpenShift.

plug-in
Software component providing the actual implementation for Networking APIs, or for Compute APIs, depending on the context.

policy service
Component of Identity that provides a rule-management interface and a rule-based authorization engine.

policy-based routing (PBR)
Provides a mechanism to implement packet forwarding and routing according to the policies defined by the network administrator.

pool
A logical set of devices, such as web servers, that you group together to receive and process traffic. The load balancing function chooses which member of the pool handles the new requests or connections received on the VIP address. Each VIP has one pool.

pool member
An application that runs on the back-end server in a load-balancing system.

port
A virtual network port within Networking; VIFs / vNICs are connected to a port.

port UUID
Unique ID for a Networking port.

preseed
A tool to automate system configuration and installation on Debian-based Linux distributions.

private cloud
Computing resources used exclusively by one business or organization.

private image
An Image service VM image that is only available to specified projects.

private IP address
An IP address used for management and administration, not available to the public Internet.

private network
The Network Controller provides virtual networks to enable compute servers to interact with each other and with the public network. All machines must have a public and private network interface. A private network interface can be a flat or VLAN network interface. A flat network interface is controlled by the flat_interface with flat managers. A VLAN network interface is controlled by the `vlan_interface` option with VLAN managers.

project
Projects represent the base unit of ownership in OpenStack, in that all resources in OpenStack should be owned by a specific project. In OpenStack Identity, a project must be owned by a specific domain.

project ID
Unique ID assigned to each project by the Identity service.

project VPN
Alternative term for a cloudpipe.

promiscuous mode
Causes the network interface to pass all traffic it receives to the host rather than passing only the frames addressed to it.

protected property
Generally, extra properties on an Image service image to which only cloud administrators have access. Limits which user roles can perform CRUD operations on that property. The cloud administrator can configure any image property as protected.

provider
An administrator who has access to all hosts and instances.

proxy node
A node that provides the Object Storage proxy service.

proxy server
Users of Object Storage interact with the service through the proxy server, which in turn looks up the location of the requested data within the ring and returns the results to the user.

public API
An API endpoint used for both service-to-service communication and end-user interactions.

public cloud
Data centers available to many users over the Internet.

public image
An Image service VM image that is available to all projects.

public IP address
An IP address that is accessible to end-users.

public key authentication
Authentication method that uses keys rather than passwords.

public network
The Network Controller provides virtual networks to enable compute servers to interact with each other and with the public network. All machines must have a public and private network interface. The public network interface is controlled by the `public_interface` option.

Puppet
An operating system configuration-management tool supported by OpenStack.

Python
Programming language used extensively in OpenStack.

Q

QEMU Copy On Write 2 (QCOW2)
One of the VM image disk formats supported by Image service.

Qpid
Message queue software supported by OpenStack; an alternative to RabbitMQ.

Quality of Service (QoS)
The ability to guarantee certain network or storage requirements to satisfy a Service Level Agreement (SLA) between an application provider and end users. Typically includes performance requirements like networking bandwidth, latency, jitter correction, and reliability as well as storage performance in Input/Output Operations Per Second (IOPS), throttling agreements, and performance expectations at peak load.

quarantine
If Object Storage finds objects, containers, or accounts that are corrupt, they are placed in this state, are not replicated, cannot be read by clients, and a correct copy is re-replicated.

Queens
The code name for the seventeenth release of OpenStack. The OpenStack summit took place in Sydney, Australia. The release is named after the Queens Pound river in the South Coast region of New South Wales.

Quick EMUlator (QEMU)
QEMU is a generic and open source machine emulator and virtualizer. One of the hypervisors supported by OpenStack, generally used for development purposes.

quota
In Compute and Block Storage, the ability to set resource limits on a per-project basis.

R

RabbitMQ
The default message queue software used by OpenStack.

Rackspace Cloud Files
Released as open source by Rackspace in 2010; the basis for Object Storage.

RADOS Block Device (RBD)
Ceph component that enables a Linux block device to be striped over multiple distributed data stores.

radvd
The router advertisement daemon, used by the Compute VLAN manager and FlatDHCP manager to provide routing services for VM instances.

rally
Codename for the *Benchmark service*.

RAM filter
The Compute setting that enables or disables RAM overcommitment.

RAM overcommit
The ability to start new VM instances based on the actual memory usage of a host, as opposed to basing the decision on the amount of RAM each running instance thinks it has available. Also known as memory overcommit.

rate limit
Configurable option within Object Storage to limit database writes on a per-account and/or per-container basis.

raw
One of the VM image disk formats supported by Image service; an unstructured disk image.

reader (Role)
A role that grants read-only access to resources within a specific scope system, domain, or project. The behavior of the reader role depends on its assigned scope. For example, a system-level reader can view all projects in the deployment, while a domain-level reader can only view projects within their domain. This scope-based distinction affects how APIs behave for different users with the reader role.

rebalance
The process of distributing Object Storage partitions across all drives in the ring; used during initial ring creation and after ring reconfiguration.

reboot
Either a soft or hard reboot of a server. With a soft reboot, the operating system is signaled to restart, which enables a graceful shutdown of all processes. A hard reboot is the equivalent of power cycling the server. The virtualization platform should ensure that the reboot action has completed successfully, even in cases in which the underlying domain/VM is paused or halted/stopped.

rebuild
Removes all data on the server and replaces it with the specified image. Server ID and IP addresses remain the same.

Recon
An Object Storage component that collects meters.

record
Belongs to a particular domain and is used to specify information about the domain. There are several types of DNS records. Each record type contains particular information used to describe the purpose of that record. Examples include mail exchange (MX) records, which specify the mail server for a particular domain; and name server (NS) records, which specify the authoritative name servers for a domain.

record ID
A number within a database that is incremented each time a change is made. Used by Object Storage when replicating.

Red Hat Enterprise Linux (RHEL)
A Linux distribution that is compatible with OpenStack.

reference architecture
A recommended architecture for an OpenStack cloud.

Region
A region in OpenStack represents a complete OpenStack cluster that has a dedicated control plane and set of API endpoints. It is common for operators of large clouds to offer their users several

OpenStack regions, which differ by their geographical location or purpose. In order to easily navigate in a multi-region environment, cloud users need a way to distinguish clusters by their names.

registry
Alternative term for the Image service registry.

registry server
An Image service that provides VM image metadata information to clients.

Reliable, Autonomic Distributed Object Store
(RADOS)

A collection of components that provides object storage within Ceph. Similar to OpenStack Object Storage.

Remote Procedure Call (RPC)
The method used by the Compute RabbitMQ for intra-service communications.

replica
Provides data redundancy and fault tolerance by creating copies of Object Storage objects, accounts, and containers so that they are not lost when the underlying storage fails.

replica count
The number of replicas of the data in an Object Storage ring.

replication
The process of copying data to a separate physical device for fault tolerance and performance.

replicator
The Object Storage back-end process that creates and manages object replicas.

request ID
Unique ID assigned to each request sent to Compute.

rescue image
A special type of VM image that is booted when an instance is placed into rescue mode. Allows an administrator to mount the file systems for an instance to correct the problem.

resize
Converts an existing server to a different flavor, which scales the server up or down. The original server is saved to enable rollback if a problem occurs. All resizes must be tested and explicitly confirmed, at which time the original server is removed.

RESTful
A kind of web service API that uses REST, or Representational State Transfer. REST is the style of architecture for hypermedia systems that is used for the World Wide Web.

ring
An entity that maps Object Storage data to partitions. A separate ring exists for each service, such as account, object, and container.

ring builder
Builds and manages rings within Object Storage, assigns partitions to devices, and pushes the configuration to other storage nodes.

Rocky
The code name for the eightteenth release of OpenStack. The OpenStack summit took place in Vancouver, Canada. The release is named after the Rocky Mountains.

Rocky Linux
A Linux distribution that is compatible with OpenStack.

role
A personality that a user assumes to perform a specific set of operations. A role includes a set of rights and privileges. A user assuming that role inherits those rights and privileges.

Role Based Access Control (RBAC)
Provides a predefined list of actions that the user can perform, such as start or stop VMs, reset passwords, and so on. Supported in both Identity and Compute and can be configured using the dashboard.

role ID
Alphanumeric ID assigned to each Identity service role.

Root Cause Analysis (RCA) service (Vitrage)
OpenStack project that aims to organize, analyze and visualize OpenStack alarms and events, yield insights regarding the root cause of problems and deduce their existence before they are directly detected.

rootwrap
A feature of Compute that allows the unprivileged nova user to run a specified list of commands as the Linux root user.

round-robin scheduler
Type of Compute scheduler that evenly distributes instances among available hosts.

router
A physical or virtual network device that passes network traffic between different networks.

routing key
The Compute direct exchanges, fanout exchanges, and topic exchanges use this key to determine how to process a message; processing varies depending on exchange type.

RPC driver
Modular system that allows the underlying message queue software of Compute to be changed. For example, from RabbitMQ to Qpid.

rsync
Used by Object Storage to push object replicas.

RXTX cap
Absolute limit on the amount of network traffic a Compute VM instance can send and receive.

RXTX quota
Soft limit on the amount of network traffic a Compute VM instance can send and receive.

S

SAML assertion
Contains information about a user as provided by the identity provider. It is an indication that a user has been authenticated.

Sandbox
A virtual space in which new or untested software can be run securely.

scheduler manager
A Compute component that determines where VM instances should start. Uses modular design to support a variety of scheduler types.

scoped token
An Identity service API access token that is associated with a single scope of operation. This token provides access based on defined scopes, which can vary depending on the level of access required. Scopes can include system-level access, domain-level access, or project-specific access.

scrubber
Checks for and deletes unused VMs; the component of Image service that implements delayed delete.

secret key
String of text known only by the user; used along with an access key to make requests to the Compute API.

secure boot
Process whereby the system firmware validates the authenticity of the code involved in the boot process.

secure shell (SSH)
Open source tool used to access remote hosts through an encrypted communications channel, SSH key injection is supported by Compute.

security group
A set of network traffic filtering rules that are applied to a Compute instance.

segmented object
An Object Storage large object that has been broken up into pieces. The re-assembled object is called a concatenated object.

self-service
For IaaS, ability for a regular (non-privileged) account to manage a virtual infrastructure component such as networks without involving an administrator.

SELinux
Linux kernel security module that provides the mechanism for supporting access control policies.

server
Computer that provides explicit services to the client software running on that system, often managing a variety of computer operations. A server is a VM instance in the Compute system. Flavor and image are requisite elements when creating a server.

server image
Alternative term for a VM image.

server UUID
Unique ID assigned to each guest VM instance.

service
An OpenStack service, such as Compute, Object Storage, or Image service. Provides one or more endpoints through which users can access resources and perform operations.

service (Role)
A role reserved for service-to-service communication. It allows one service to interact with another and be granted only the necessary elevated privileges by the receiving service. The service role was introduced to replace the previous practice of assigning the overly privileged admin role to service users. With this role, service-to-service APIs can now default to using the service role, ensuring more secure and limited access tailored to inter-service needs.

service catalog
Alternative term for the Identity service catalog.

Service Function Chain (SFC)
For a given service, SFC is the abstracted view of the required service functions and the order in which they are to be applied.

service ID
Unique ID assigned to each service that is available in the Identity service catalog.

Service Level Agreement (SLA)
Contractual obligations that ensure the availability of a service.

service project
Special project that contains all services that are listed in the catalog.

service provider
A system that provides services to other system entities. In case of federated identity, OpenStack Identity is the service provider.

service registration
An Identity service feature that enables services, such as Compute, to automatically register with the catalog.

service token
An administrator-defined token used by Compute to communicate securely with the Identity service.

session back end
The method of storage used by horizon to track client sessions, such as local memory, cookies, a database, or memcached.

session persistence
A feature of the load-balancing service. It attempts to force subsequent connections to a service to be redirected to the same node as long as it is online.

session storage
A horizon component that stores and tracks client session information. Implemented through the Django sessions framework.

share
A remote, mountable file system in the context of the *Shared File Systems service*. You can mount a share to, and access a share from, several hosts by several users at a time.

share network
An entity in the context of the *Shared File Systems service* that encapsulates interaction with the Networking service. If the driver you selected runs in the mode requiring such kind of interaction, you need to specify the share network to create a share.

Shared File Systems API
A Shared File Systems service that provides a stable RESTful API. The service authenticates and routes requests throughout the Shared File Systems service. There is python-manilaclient to interact with the API.

Shared File Systems service (manila)
The service that provides a set of services for management of shared file systems in a multi-project cloud environment, similar to how OpenStack provides block-based storage management through the OpenStack *Block Storage service* project. With the Shared File Systems service, you can create

a remote file system and mount the file system on your instances. You can also read and write data from your instances to and from your file system.

shared IP address
An IP address that can be assigned to a VM instance within the shared IP group. Public IP addresses can be shared across multiple servers for use in various high-availability scenarios. When an IP address is shared to another server, the cloud network restrictions are modified to enable each server to listen to and respond on that IP address. You can optionally specify that the target server network configuration be modified. Shared IP addresses can be used with many standard heartbeat facilities, such as keepalive, that monitor for failure and manage IP failover.

shared IP group
A collection of servers that can share IPs with other members of the group. Any server in a group can share one or more public IPs with any other server in the group. With the exception of the first server in a shared IP group, servers must be launched into shared IP groups. A server may be a member of only one shared IP group.

shared storage
Block storage that is simultaneously accessible by multiple clients, for example, NFS.

Sheepdog
Distributed block storage system for QEMU, supported by OpenStack.

Simple Cloud Identity Management (SCIM)
Specification for managing identity in the cloud, currently unsupported by OpenStack.

Simple Protocol for Independent Computing Environments (SPICE)
SPICE provides remote desktop access to guest virtual machines. It is an alternative to VNC. SPICE is supported by OpenStack.

Single-root I/O Virtualization (SR-IOV)
A specification that, when implemented by a physical PCIe device, enables it to appear as multiple separate PCIe devices. This enables multiple virtualized guests to share direct access to the physical device, offering improved performance over an equivalent virtual device. Currently supported in OpenStack Havana and later releases.

Skyline
An OpenStack dashboard with optimized modern user interface (UI) and user experience (UX). It offers an improved performance, supporting higher concurrency compared to previous dashboard solutions.

snapshot
A point-in-time copy of an OpenStack storage volume or image. Use storage volume snapshots to back up volumes. Use image snapshots to back up data, or as gold images for additional servers.

soft reboot
A controlled reboot where a VM instance is properly restarted through operating system commands.

Software Development Kit (SDK)
Contains code, examples, and documentation that you use to create applications in the language of your choice.

Software Development Lifecycle Automation service (solum)
OpenStack project that aims to make cloud services easier to consume and integrate with application development process by automating the source-to-image process, and simplifying app-centric deployment.

Software-defined networking (SDN)
Provides an approach for network administrators to manage computer network services through abstraction of lower-level functionality.

SolidFire Volume Driver
The Block Storage driver for the SolidFire iSCSI storage appliance.

solum
Code name for the *Software Development Lifecycle Automation service*.

spread-first scheduler
The Compute VM scheduling algorithm that attempts to start a new VM on the host with the least amount of load.

SQLAlchemy
An open source SQL toolkit for Python, used in OpenStack.

SQLite
A lightweight SQL database, used as the default persistent storage method in many OpenStack services.

stack
A set of OpenStack resources created and managed by the Orchestration service according to a given template (either an AWS CloudFormation template or a Heat Orchestration Template (HOT)).

StackTach
Community project that captures Compute AMQP communications; useful for debugging.

static IP address
Alternative term for a fixed IP address.

StaticWeb
WSGI middleware component of Object Storage that serves container data as a static web page.

Stein
The code name for the nineteenth release of OpenStack. The OpenStack Summit took place in Berlin, Germany. The release is named after the street SteinstraSSe in Berlin.

storage back end
The method that a service uses for persistent storage, such as iSCSI, NFS, or local disk.

storage manager
A XenAPI component that provides a pluggable interface to support a wide variety of persistent storage back ends.

storage manager back end
A persistent storage method supported by XenAPI, such as iSCSI or NFS.

storage node
An Object Storage node that provides container services, account services, and object services; controls the account databases, container databases, and object storage.

storage services
Collective name for the Object Storage object services, container services, and account services.

strategy
Specifies the authentication source used by Image service or Identity. In the Database service, it refers to the extensions implemented for a data store.

subdomain
A domain within a parent domain. Subdomains cannot be registered. Subdomains enable you to delegate domains. Subdomains can themselves have subdomains, so third-level, fourth-level, fifth-level, and deeper levels of nesting are possible.

subnet
Logical subdivision of an IP network.

suspend
The VM instance is paused and its state is saved to disk of the host.

swap
Disk-based virtual memory used by operating systems to provide more memory than is actually available on the system.

swift
Codename for OpenStack *Object Storage service*.

swift All in One (SAIO)
Creates a full Object Storage development environment within a single VM.

swift middleware
Collective term for Object Storage components that provide additional functionality.

swift proxy server
Acts as the gatekeeper to Object Storage and is responsible for authenticating the user.

swift storage node
A node that runs Object Storage account, container, and object services.

sync point
Point in time since the last container and accounts database sync among nodes within Object Storage.

system usage
A Compute component that, along with the notification system, collects meters and usage information. This information can be used for billing.

T

tacker
Code name for the *NFV Orchestration service*

Telemetry service (telemetry)
The OpenStack project which collects measurements of the utilization of the physical and virtual resources comprising deployed clouds, persists this data for subsequent retrieval and analysis, and triggers actions when defined criteria are met.

TempAuth
An authentication facility within Object Storage that enables Object Storage itself to perform authentication and authorization. Frequently used in testing and development.

Tempest
Automated software test suite designed to run against the trunk of the OpenStack core project.

TempURL
An Object Storage middleware component that enables creation of URLs for temporary object access.

tenant
A group of users; used to isolate access to Compute resources. An alternative term for a project.

Tenant API
An API that is accessible to projects.

tenant endpoint
An Identity service API endpoint that is associated with one or more projects.

tenant ID
An alternative term for *project ID*.

token
An alpha-numeric string of text used to access OpenStack APIs and resources.

token services
An Identity service component that manages and validates tokens after a user or project has been authenticated.

tombstone
Used to mark Object Storage objects that have been deleted; ensures that the object is not updated on another node after it has been deleted.

topic publisher
A process that is created when a RPC call is executed; used to push the message to the topic exchange.

Torpedo
Community project used to run automated tests against the OpenStack API.

Train
The code name for the twentieth release of OpenStack. The OpenStack Infrastructure Summit took place in Denver, Colorado, US.

Two Project Team Gathering meetings in Denver were held at a hotel next to the train line from downtown to the airport. The crossing signals there had some sort of malfunction in the past causing them to not stop the cars when a train was coming properly. As a result the trains were required to blow their horns when passing through that area. Obviously staying in a hotel, by trains that are blowing their horns 24/7 was less than ideal. As a result, many jokes popped up about Denver and trains - and thus the release is called train.

transaction ID
Unique ID assigned to each Object Storage request; used for debugging and tracing.

transient
Alternative term for non-durable.

transient exchange
Alternative term for a non-durable exchange.

transient message
A message that is stored in memory and is lost after the server is restarted.

transient queue
Alternative term for a non-durable queue.

trove
Codename for OpenStack *Database service*.

trusted platform module (TPM)
Specialized microprocessor for incorporating cryptographic keys into devices for authenticating and securing a hardware platform.

U

Ubuntu
A Debian-based Linux distribution.

unscoped token
Alternative term for an Identity service default token.

updater
Collective term for a group of Object Storage components that processes queued and failed updates for containers and objects.

user
In OpenStack Identity, entities represent individual API consumers and are owned by a specific domain. In OpenStack Compute, a user can be associated with roles, projects, or both.

user data
A blob of data that the user can specify when they launch an instance. The instance can access this data through the metadata service or config drive. Commonly used to pass a shell script that the instance runs on boot.

User Mode Linux (UML)
An OpenStack-supported hypervisor.

Ussuri
The code name for the twenty first release of OpenStack. The OpenStack Infrastructure Summit took place in Shanghai, Peoples Republic of China. The release is named after the Ussuri river.

V

Victoria
The code name for the twenty second release of OpenStack. The OpenDev + PTG was planned to take place in Vancouver, British Columbia, Canada. The release is named after Victoria, the capital city of British Columbia.

The in-person event was cancelled due to COVID-19. The event is being virtualized instead.

VIF UUID
Unique ID assigned to each Networking VIF.

Virtual Central Processing Unit (vCPU)
Subdivides physical CPUs. Instances can then use those divisions.

Virtual Disk Image (VDI)
One of the VM image disk formats supported by Image service.

Virtual Extensible LAN (VXLAN)
A network virtualization technology that attempts to reduce the scalability problems associated with large cloud computing deployments. It uses a VLAN-like encapsulation technique to encapsulate Ethernet frames within UDP packets.

Virtual Hard Disk (VHD)
One of the VM image disk formats supported by Image service.

virtual IP address (VIP)
An Internet Protocol (IP) address configured on the load balancer for use by clients connecting to a service that is load balanced. Incoming connections are distributed to back-end nodes based on the configuration of the load balancer.

virtual machine (VM)
An operating system instance that runs on top of a hypervisor. Multiple VMs can run at the same time on the same physical host.

virtual network
An L2 network segment within Networking.

Virtual Network Computing (VNC)
Open source GUI and CLI tools used for remote console access to VMs. Supported by Compute.

Virtual Network InterFace (VIF)
An interface that is plugged into a port in a Networking network. Typically a virtual network interface belonging to a VM.

virtual networking
A generic term for virtualization of network functions such as switching, routing, load balancing, and security using a combination of VMs and overlays on physical network infrastructure.

virtual port
Attachment point where a virtual interface connects to a virtual network.

virtual private network (VPN)
Provided by Compute in the form of cloudpipes, specialized instances that are used to create VPNs on a per-project basis.

virtual server
Alternative term for a VM or guest.

virtual switch (vSwitch)
Software that runs on a host or node and provides the features and functions of a hardware-based network switch.

virtual VLAN
Alternative term for a virtual network.

Vitrage
Code name for the *Root Cause Analysis service*.

VLAN manager
A Compute component that provides dnsmasq and radvd and sets up forwarding to and from cloudpipe instances.

VLAN network
The Network Controller provides virtual networks to enable compute servers to interact with each other and with the public network. All machines must have a public and private network interface. A VLAN network is a private network interface, which is controlled by the `vlan_interface` option with VLAN managers.

VM disk (VMDK)
One of the VM image disk formats supported by Image service.

VM image
Alternative term for an image.

VM Remote Control (VMRC)
Method to access VM instance consoles using a web browser. Supported by Compute.

VMware API
Supports interaction with VMware products in Compute.

VMware NSX Neutron plug-in
Provides support for VMware NSX in Neutron.

VNC proxy
A Compute component that provides users access to the consoles of their VM instances through VNC or VMRC.

volume
Disk-based data storage generally represented as an iSCSI target with a file system that supports extended attributes; can be persistent or ephemeral.

Volume API
Alternative name for the Block Storage API.

volume controller
A Block Storage component that oversees and coordinates storage volume actions.

volume driver
Alternative term for a volume plug-in.

volume ID
Unique ID applied to each storage volume under the Block Storage control.

volume manager
A Block Storage component that creates, attaches, and detaches persistent storage volumes.

volume node
A Block Storage node that runs the cinder-volume daemon.

volume plug-in
Provides support for new and specialized types of back-end storage for the Block Storage volume manager.

volume worker
A cinder component that interacts with back-end storage to manage the creation and deletion of volumes and the creation of compute volumes, provided by the cinder-volume daemon.

vSphere
An OpenStack-supported hypervisor.

W

Wallaby
The code name for the twenty third release of OpenStack. Wallabies are native to Australia, which at the start of this naming period was experiencing unprecedented wild fires.

Watcher
Code name for the *Infrastructure Optimization service*.

weight
Used by Object Storage devices to determine which storage devices are suitable for the job. Devices are weighted by size.

weighted cost
The sum of each cost used when deciding where to start a new VM instance in Compute.

weighting
A Compute process that determines the suitability of the VM instances for a job for a particular host. For example, not enough RAM on the host, too many CPUs on the host, and so on.

worker
A daemon that listens to a queue and carries out tasks in response to messages. For example, the cinder-volume worker manages volume creation and deletion on storage arrays.

Workflow service (mistral)
The OpenStack service that provides a simple YAML-based language to write workflows (tasks and transition rules) and a service that allows to upload them, modify, run them at scale and in a highly available manner, manage and monitor workflow execution state and state of individual tasks.

X

X.509
X.509 is the most widely used standard for defining digital certificates. It is a data structure that contains the subject (entity) identifiable information such as its name along with its public key. The certificate can contain a few other attributes as well depending upon the version. The most recent and standard version of X.509 is v3.

Xena
The code name for the twenty fourth release of OpenStack. The release is named after a fictional warrior princess.

XFS
High-performance 64-bit file system created by Silicon Graphics. Excels in parallel I/O operations and data consistency.

Y

Yoga
The code name for the twenty fifth release of OpenStack. The release is named after a philosophical school with mental and physical practices from India.

Z

zaqar
Codename for the *Message service*.

Zed
The code name for the twenty sixth release of OpenStack. The release is named after the pronunciation of the letter Z.

Zuul
Zuul is an open source CI/CD platform specializing in gating changes across multiple systems and applications before landing a single patch.

Zuul is used for OpenStack development to ensure that only tested code gets merged.

INDEX

Non-alphabetical

6to4,
2023.1 Antelope,
2023.2 Bobcat,
2024.1 Caracal,
2024.2 Dalmatian,
2025.1 Epoxy,
2025.2 Flamingo,
2026.1 Gazpacho,
2026.2 Hibiscus,

A

absolute limit,
access control list (*ACL*),
access key,
account,
account auditor,
account database,
account reaper,
account server,
account service,
accounting,
Active Directory,
active/active configuration,
active/passive configuration,
address pool,
Address Resolution Protocol (*ARP*),
admin (*Role*),
admin API,
admin server,
administrator,
Advanced Message Queuing Protocol (*AMQP*),
Advanced RISC Machine (*ARM*),
alert,
allocate,
AlmaLinux,
Amazon Kernel Image (*AKI*),
Amazon Machine Image (*AMI*),
Amazon Ramdisk Image (*ARI*),
Ansible,
aodh,
Apache,
Apache License 2.0,
Apache Web Server,
API endpoint,
API extension,
API extension plug-in,
API key,
API server,
API token,
API version,
applet,
Application Programming Interface (*API*),
application server,
Application Service Provider (*ASP*),
arptables,
associate,
Asynchronous JavaScript and XML (*AJAX*),
ATA over Ethernet (*AoE*),
attach,
attachment (*network*),
auditing,
auditor,
Austin,
auth node,
authentication,
authentication token,
AuthN,
authorization,
authorization node,
AuthZ,
Auto ACK,
auto declare,
availability zone,
AWS CloudFormation template,

B

back end,
back-end catalog,
back-end store,
Backup and Restore service (*freezer*),

bandwidth,
barbican,
bare,
Bare Metal service (*ironic*),
base image,
Bell-LaPadula model,
Benchmark service (*rally*),
Bexar,
binary,
bit,
bits per second (*BPS*),
block device,
block migration,
Block Storage API,
Block Storage service (*cinder*),
BMC (*Baseboard Management Controller*),
bootable disk image,
Bootstrap Protocol (*BOOTP*),
Border Gateway Protocol (*BGP*),
browser,
builder file,
bursting,
button class,
byte,

C

cache pruner,
Cactus,
CALL,
capability,
capacity cache,
capacity updater,
CAST,
catalog,
catalog service,
ceilometer,
cell,
cell forwarding,
cell manager,
CentOS,
Ceph,
CephFS,
certificate authority (*CA*),
Challenge-Handshake Authentication Protocol (*CHAP*),
chance scheduler,
changes since,
child cell,
cinder,
CirrOS,
Cisco neutron plug-in,
cloud architect,
Cloud Auditing Data Federation (*CADF*),
cloud computing,
cloud computing infrastructure,
cloud computing platform software,
cloud computing service architecture,
cloud controller,
cloud controller node,
Cloud Data Management Interface (*CDMI*),
Cloud Infrastructure Management Interface (*CIMI*),
cloud technology,
cloud-init,
Cloudbase-Init,
command filter,
Command-Line Interface (*CLI*),
Common Internet File System (*CIFS*),
Common Libraries (*oslo*),
community project,
compression,
Compute API (*nova API*),
compute controller,
compute host,
compute node,
Compute service (*nova*),
compute worker,
concatenated object,
conductor,
consistency window,
console log,
container,
container auditor,
container database,
container format,
Container Infrastructure Management service (*magnum*),
container server,
container service,
content delivery network (*CDN*),
continuous delivery,
continuous deployment,
continuous integration,
controller node,
core API,
cost,
credentials,
CRL,
Cross-Origin Resource Sharing (*CORS*),
Crowbar,
current workload,
customer,

customization module,

D

daemon,
Dashboard (*horizon*),
data encryption,
Data loss prevention (DLP) software,
data store,
database ID,
database replicator,
Database service (*trove*),
deallocate,
Debian,
deduplication,
default panel,
default project,
default token,
delayed delete,
delivery mode,
denial of service (*DoS*),
deprecated auth,
designate,
Desktop-as-a-Service,
device ID,
device weight,
DevStack,
DHCP agent,
Diablo,
direct consumer,
direct exchange,
direct publisher,
disassociate,
Discretionary Access Control (*DAC*),
disk encryption,
disk format,
dispersion,
distributed virtual router (*DVR*),
Django,
DNS record,
DNS service (*designate*),
dnsmasq,
domain,
Domain Name System (*DNS*),
download,
durable exchange,
durable queue,
Dynamic Host Configuration Protocol (*DHCP*),
Dynamic HyperText Markup Language (*DHTML*),

E

east-west traffic,
EBS boot volume,
ebtables,
EC2,
EC2 access key,
EC2 API,
EC2 Compatibility API,
EC2 secret key,
edge computing,
Elastic Block Storage (*EBS*),
encapsulation,
encryption,
endpoint,
endpoint registry,
endpoint template,
enterprise cloud computing,
entity,
ephemeral image,
ephemeral volume,
Essex,
ESXi,
ETag,
euca2ools,
Eucalyptus Kernel Image (*EKI*),
Eucalyptus Machine Image (*EMI*),
Eucalyptus Ramdisk Image (*ERI*),
evacuate,
exchange,
exchange type,
exclusive queue,
extended attributes (*xattr*),
extension,
external network,
extra specs,

F

FakeLDAP,
fan-out exchange,
federated identity,
Fedora,
Fibre Channel,
Fibre Channel over Ethernet (*FCoE*),
fill-first scheduler,
filter,
firewall,
FireWall-as-a-Service (*FWaaS*),
fixed IP address,
Flat Manager,
flat mode injection,
flat network,

FlatDHCP Manager,
flavor,
flavor ID,
floating IP address,
Folsom,
FormPost,
freezer,
front end,

G

gateway,
generic network virtualization encapsulation (*Geneve*),
generic receive offload (*GRO*),
generic routing encapsulation (*GRE*),
glance,
glance API server,
glance registry,
global endpoint template,
GlusterFS,
gnocchi,
golden image,
Graphic Interchange Format (*GIF*),
Graphics Processing Unit (*GPU*),
Green Threads,
Grizzly,
Group,
guest OS,

H

Hadoop,
Hadoop Distributed File System (*HDFS*),
handover,
HAProxy,
hard reboot,
Havana,
health monitor,
heat,
Heat Orchestration Template (*HOT*),
high availability (*HA*),
horizon,
horizon plug-in,
host,
host aggregate,
Host Bus Adapter (*HBA*),
hybrid cloud,
hybrid cloud computing,
hyperlink,
Hypertext Transfer Protocol (*HTTP*),
Hypertext Transfer Protocol Secure (*HTTPS*),
hypervisor,
hypervisor pool,

I

Icehouse,
ID number,
Identity API,
Identity back end,
identity provider,
Identity service (*keystone*),
Identity service API,
IETF,
image,
Image API,
image cache,
image ID,
image membership,
image owner,
image registry,
Image service (*glance*),
image status,
image store,
image UUID,
incubated project,
Infrastructure Optimization service (*watcher*),
Infrastructure-as-a-Service (*IaaS*),
ingress filtering,
INI format,
injection,
Input/Output Operations Per Second (*IOPS*),
instance,
instance ID,
instance state,
instance tunnels network,
instance type,
instance type ID,
instance UUID,
Intelligent Platform Management Interface (*IPMI*),
interface,
interface ID,
Internet Control Message Protocol (*ICMP*),
Internet protocol (*IP*),
Internet Service Provider (*ISP*),
Internet Small Computer System Interface (*iSCSI*),
IO,
IP address,
IP Address Management (*IPAM*),

ip6tables,
ipset,
iptables,
ironic,
iSCSI Qualified Name (*IQN*),
ISO9660,

J

Java,
JavaScript,
JavaScript Object Notation (*JSON*),
jumbo frame,
Juno,

K

Kerberos,
kernel-based VM (*KVM*),
Key Manager service (*barbican*),
keystone,
Kickstart,
Kilo,

L

large object,
Launchpad,
Layer-2 (L2) agent,
Layer-2 network,
Layer-3 (L3) agent,
Layer-3 network,
Liberty,
libvirt,
Lightweight Directory Access Protocol (*LDAP*),
Linux,
Linux bridge,
Linux Bridge neutron plug-in,
Linux containers (*LXC*),
live migration,
load balancer,
load balancing,
Load-Balancer-as-a-Service (*LBaaS*),
Load-balancing service (*octavia*),
Logical Volume Manager (*LVM*),

M

magnum,
management API,
management network,
manager,
manager (*Role*),
manifest,
manifest object,
manila,
manila-share,
maximum transmission unit (*MTU*),
mechanism driver,
melange,
member (*Role*),
membership,
membership list,
memcached,
memory overcommit,
message broker,
message bus,
message queue,
Message service (*zaqar*),
Meta-Data Server (*MDS*),
Metadata agent,
migration,
mistral,
Mitaka,
Modular Layer 2 (ML2) neutron plug-in,
monasca,
Monitor (*LBaaS*),
Monitor (*Mon*),
Monitoring (*monasca*),
multi-cloud computing,
multi-cloud SDKs,
multi-factor authentication,
multi-host,
multinic,

N

Nebula,
NetApp volume driver,
network,
Network Address Translation (*NAT*),
network controller,
Network File System (*NFS*),
network ID,
network manager,
network namespace,
network node,
network segment,
Network Service Header (*NSH*),
Network Time Protocol (*NTP*),
network UUID,
network worker,
Networking API (*Neutron API*),
Networking service (*neutron*),
neutron,
neutron API,
neutron manager,

neutron plug-in,
Newton,
Nexenta volume driver,
NFV Orchestration service (*tacker*),
Nginx,
No ACK,
node,
non-durable exchange,
non-durable queue,
non-persistent volume,
north-south traffic,
nova,
Nova API,

O

object,
object auditor,
object expiration,
object hash,
object path hash,
object replicator,
object server,
Object Storage API,
Object Storage Device (*OSD*),
Object Storage service (*swift*),
object versioning,
Ocata,
Octavia,
Oldie,
Open Cloud Computing Interface (*OCCI*),
Open Virtual Network (*OVN*),
Open Virtualization Format (*OVF*),
Open vSwitch,
Open vSwitch (OVS) agent,
Open vSwitch neutron plug-in,
OpenDev,
OpenLDAP,
OpenStack,
OpenStack code name,
OpenStack-Ansible,
operator,
optional service,
Orchestration service (*heat*),
orphan,
oslo,

P

parent cell,
partition,
partition index,
partition shift value,
path MTU discovery (*PMTUD*),
pause,
PCI passthrough,
persistent message,
persistent volume,
personality file,
Pike,
Platform-as-a-Service (*PaaS*),
plug-in,
policy service,
policy-based routing (*PBR*),
pool,
pool member,
port,
port UUID,
preseed,
private cloud,
private image,
private IP address,
private network,
project,
project ID,
project VPN,
promiscuous mode,
protected property,
provider,
proxy node,
proxy server,
public API,
public cloud,
public image,
public IP address,
public key authentication,
public network,
Puppet,
Python,

Q

QEMU Copy On Write 2 (*QCOW2*),
Qpid,
Quality of Service (*QoS*),
quarantine,
Queens,
Quick EMUlator (*QEMU*),
quota,

R

RabbitMQ,
Rackspace Cloud Files,
RADOS Block Device (*RBD*),
radvd,
rally,
RAM filter,

RAM overcommit,
rate limit,
raw,
reader (*Role*),
rebalance,
reboot,
rebuild,
Recon,
record,
record ID,
Red Hat Enterprise Linux (*RHEL*),
reference architecture,
Region,
registry,
registry server,
Reliable, Autonomic Distributed Object Store,
Remote Procedure Call (*RPC*),
replica,
replica count,
replication,
replicator,
request ID,
rescue image,
resize,
RESTful,
ring,
ring builder,
Rocky,
Rocky Linux,
role,
Role Based Access Control (*RBAC*),
role ID,
Root Cause Analysis (RCA) service (*Vitrage*),
rootwrap,
round-robin scheduler,
router,
routing key,
RPC driver,
rsync,
RXTX cap,
RXTX quota,

S

SAML assertion,
Sandbox,
scheduler manager,
scoped token,
scrubber,
secret key,
secure boot,
secure shell (*SSH*),
security group,
segmented object,
self-service,
SELinux,
server,
server image,
server UUID,
service,
service (*Role*),
service catalog,
Service Function Chain (*SFC*),
service ID,
Service Level Agreement (*SLA*),
service project,
service provider,
service registration,
service token,
session back end,
session persistence,
session storage,
share,
share network,
Shared File Systems API,
Shared File Systems service (*manila*),
shared IP address,
shared IP group,
shared storage,
Sheepdog,
Simple Cloud Identity Management (*SCIM*),
Simple Protocol for Independent Computing Environments (*SPICE*),
Single-root I/O Virtualization (*SR-IOV*),
Skyline,
snapshot,
soft reboot,
Software Development Kit (*SDK*),
Software Development Lifecycle Automation service (*solum*),
Software-defined networking (*SDN*),
SolidFire Volume Driver,
solum,
spread-first scheduler,
SQLAlchemy,
SQLite,
stack,
StackTach,
static IP address,
StaticWeb,

Stein,
storage back end,
storage manager,
storage manager back end,
storage node,
storage services,
strategy,
subdomain,
subnet,
suspend,
swap,
swift,
swift All in One (*SAIO*),
swift middleware,
swift proxy server,
swift storage node,
sync point,
system usage,

T

tacker,
Telemetry service (*telemetry*),
TempAuth,
Tempest,
TempURL,
tenant,
Tenant API,
tenant endpoint,
tenant ID,
token,
token services,
tombstone,
topic publisher,
Torpedo,
Train,
transaction ID,
transient,
transient exchange,
transient message,
transient queue,
trove,
trusted platform module (*TPM*),

U

Ubuntu,
unscoped token,
updater,
user,
user data,
User Mode Linux (*UML*),
Ussuri,

V

Victoria,
VIF UUID,
Virtual Central Processing Unit (*vCPU*),
Virtual Disk Image (*VDI*),
Virtual Extensible LAN (*VXLAN*),
Virtual Hard Disk (*VHD*),
virtual IP address (*VIP*),
virtual machine (*VM*),
virtual network,
Virtual Network Computing (*VNC*),
Virtual Network InterFace (*VIF*),
virtual networking,
virtual port,
virtual private network (*VPN*),
virtual server,
virtual switch (*vSwitch*),
virtual VLAN,
Vitrage,
VLAN manager,
VLAN network,
VM disk (*VMDK*),
VM image,
VM Remote Control (*VMRC*),
VMware API,
VMware NSX Neutron plug-in,
VNC proxy,
volume,
Volume API,
volume controller,
volume driver,
volume ID,
volume manager,
volume node,
volume plug-in,
volume worker,
vSphere,

W

Wallaby,
Watcher,
weight,
weighted cost,
weighting,
worker,
Workflow service (*mistral*),

X

X.509,
Xena,
XFS,

Y

Yoga,

Z

zaqar,
Zed,
Zuul,